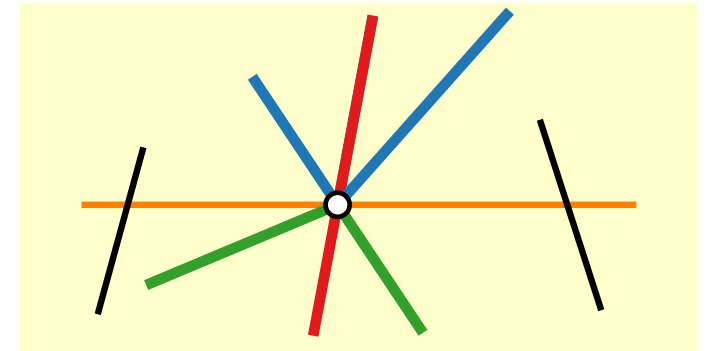
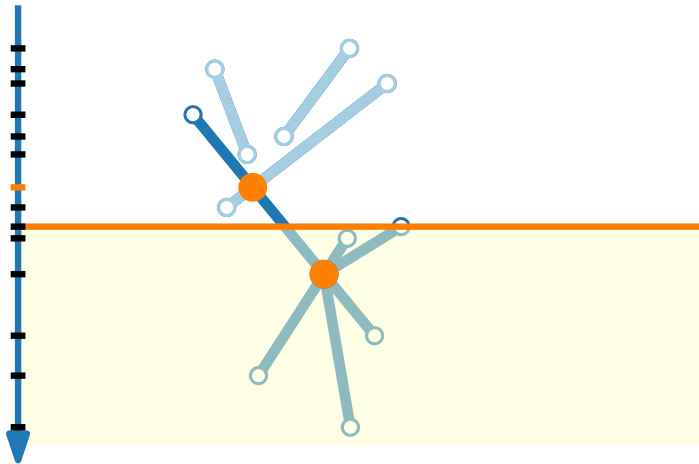


Fortgeschrittene Algorithmen

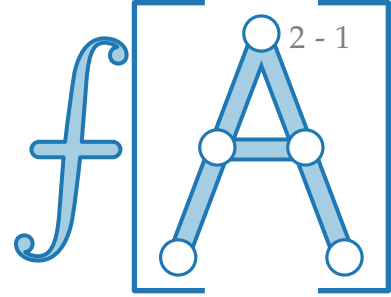
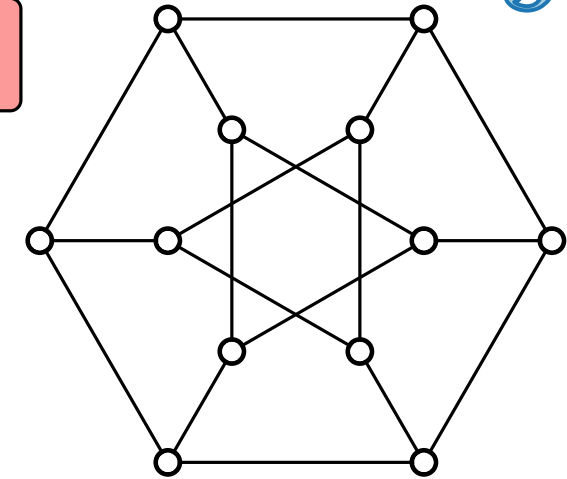
13. Vorlesung Algorithmische Geometrie II: Sweep-Line Algorithmus

Teil I: Problemstellung



Kreuzungen zählen

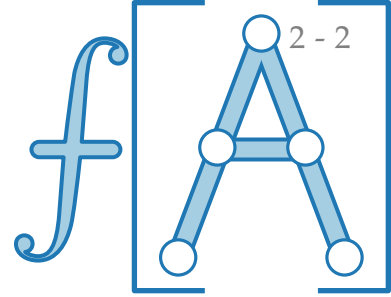
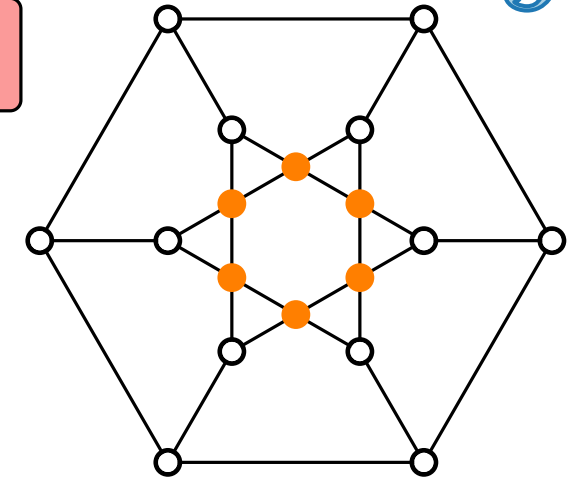
Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen



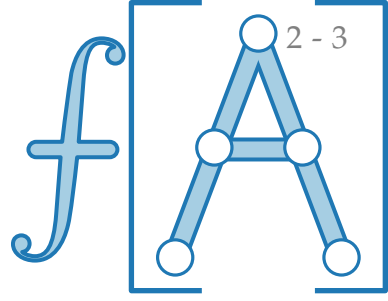
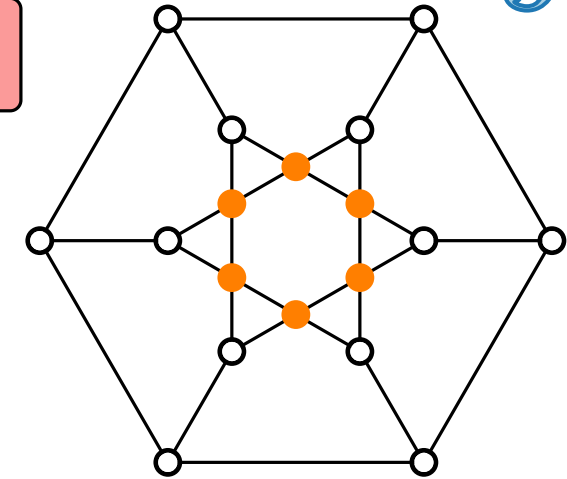
Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

Frage.

Wie finden wir alle Kreuzungen?



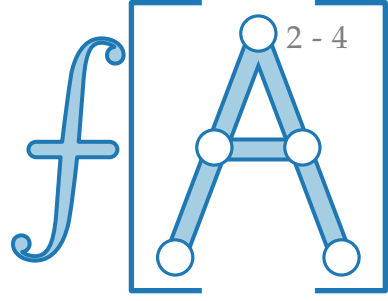
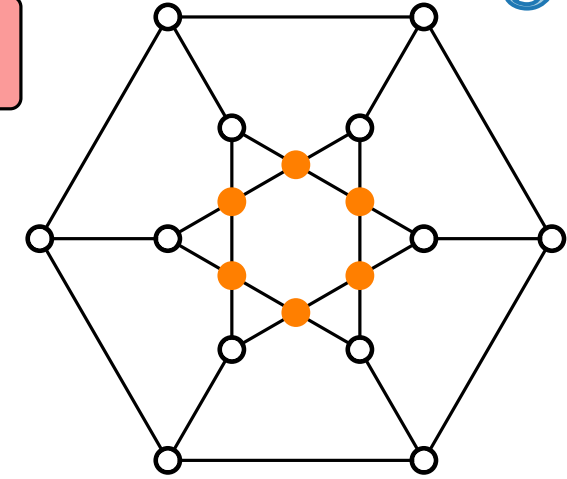
Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.



Kreuzungen zählen

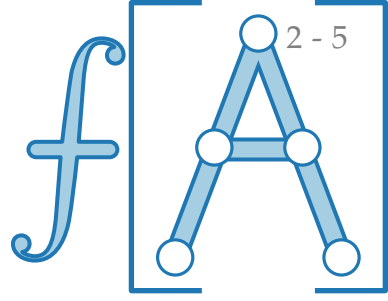
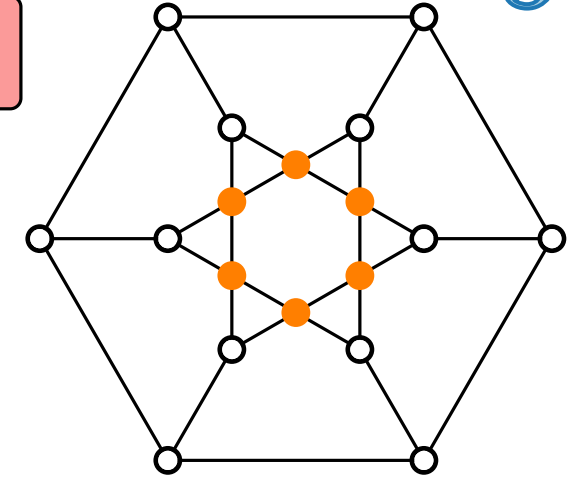
Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit?



Kreuzungen zählen

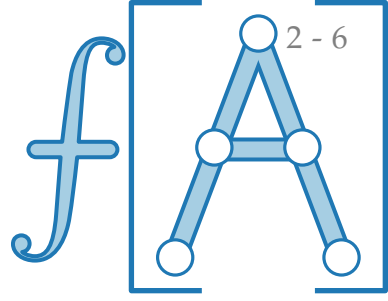
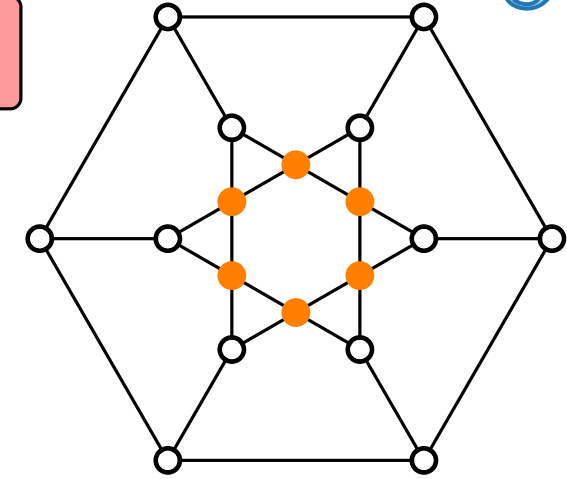
Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

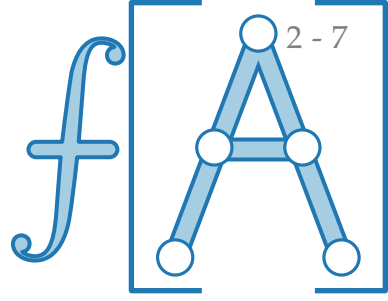
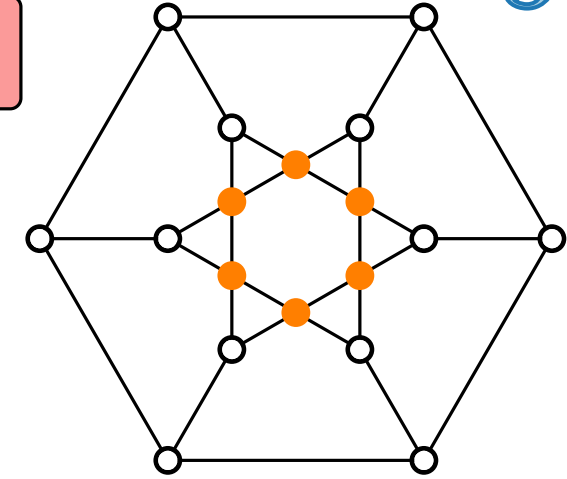
6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$

Optimal?



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

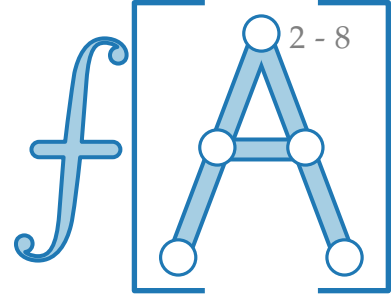
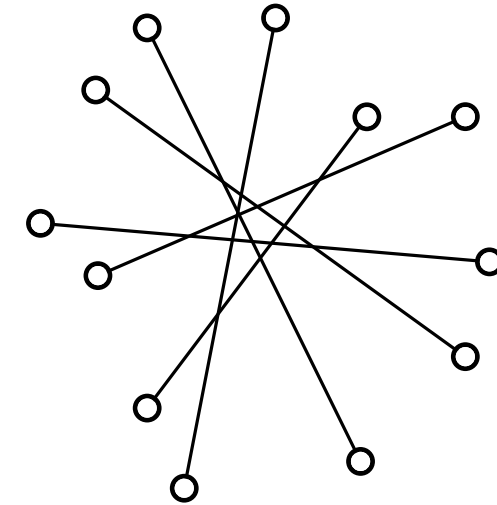
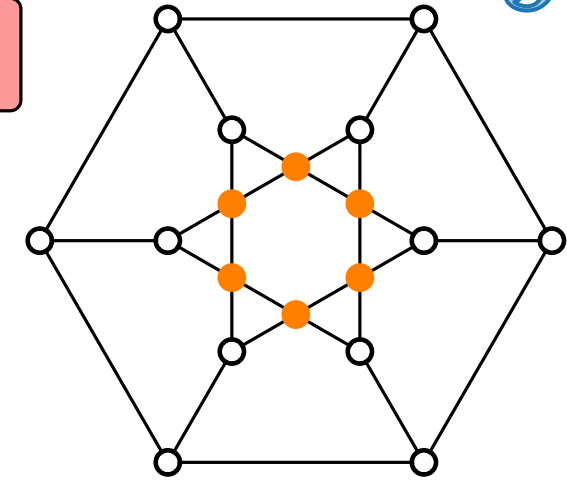
6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$

Optimal?



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

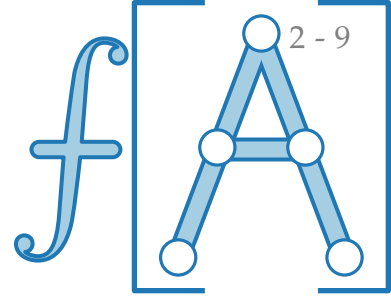
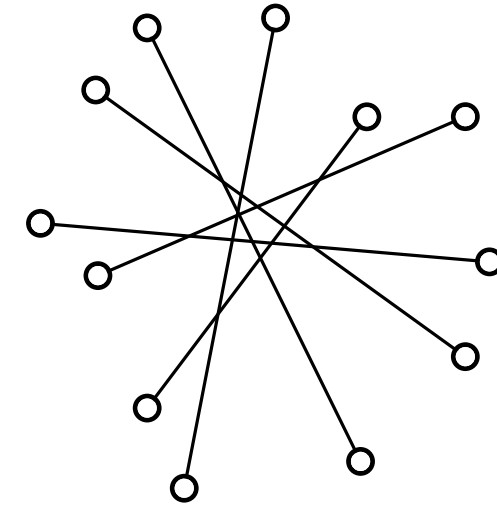
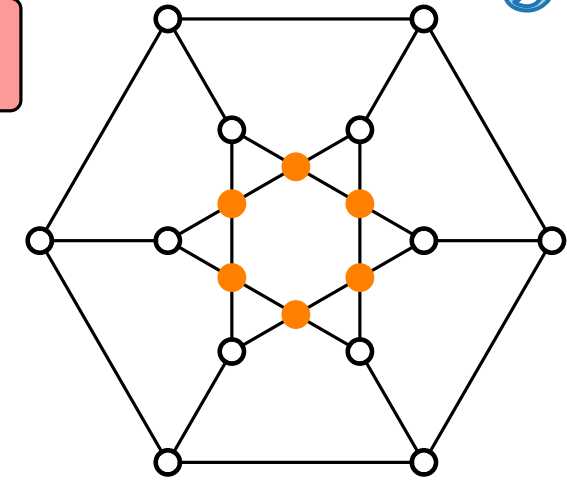
6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$

Optimal? Im schlechtesten Fall und wenn wir alle Kreuzungen ausgeben wollen: ja.



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

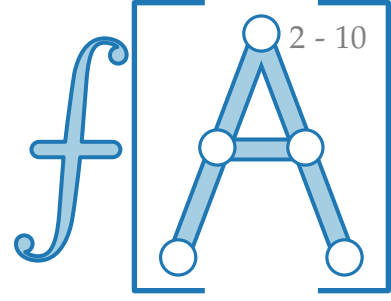
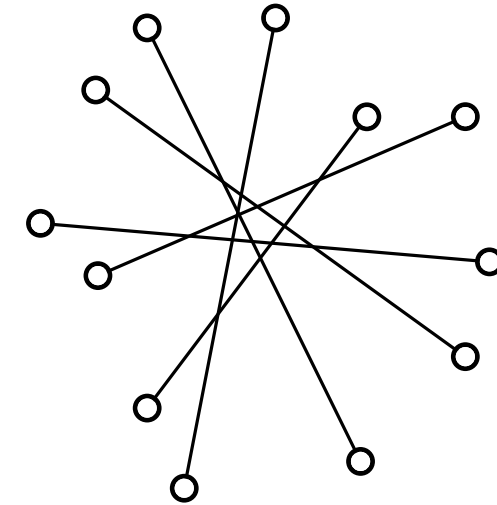
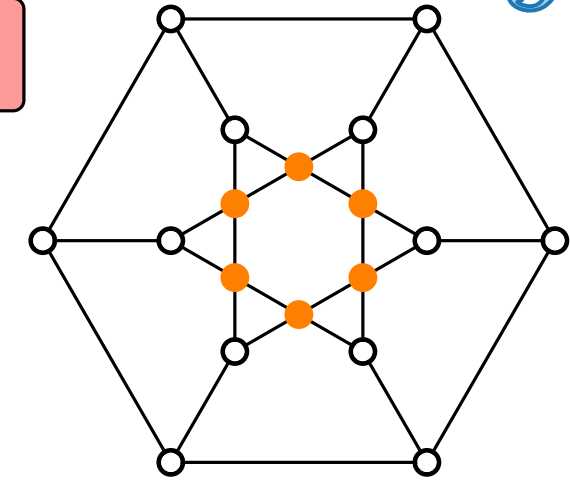
Frage. Wie finden wir alle Kreuzungen?

Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$

Optimal? Im schlechtesten Fall und wenn wir alle Kreuzungen ausgeben wollen: ja.

Frage. Was, wenn es *nicht so viele* Kreuzungen gibt?



Kreuzungen zählen

Wie viel Kreuzungen hat diese geradlinige Graphzeichnung?

6 Kreuzungen

Frage. Wie finden wir alle Kreuzungen?

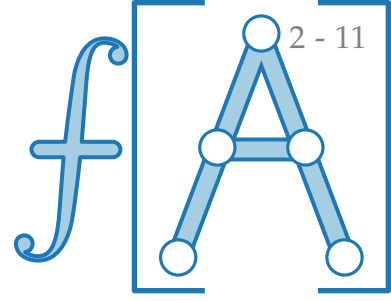
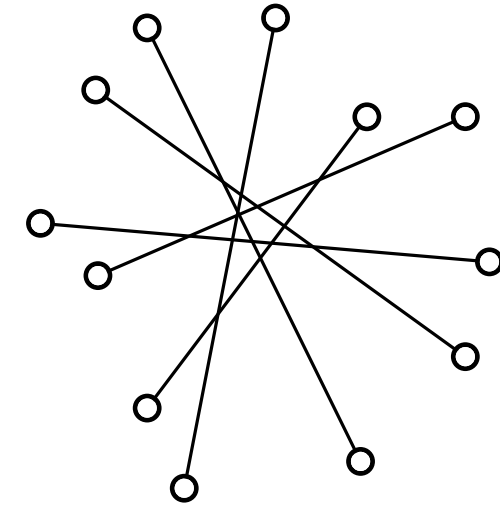
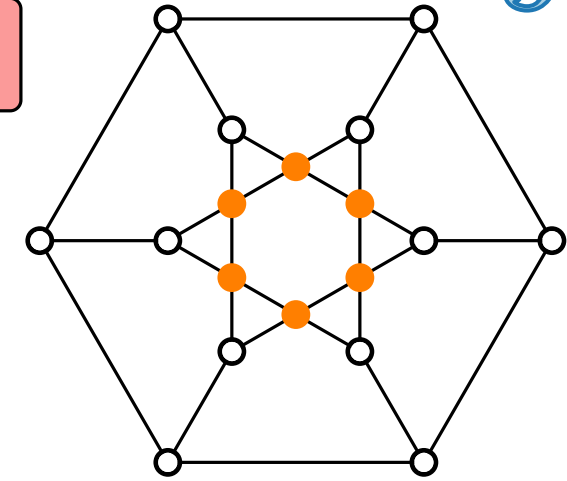
Brute Force. Für jedes Paar von Kanten, finde den Schnittpunkt der Segmente.

Laufzeit? $\binom{m}{2} = \frac{m(m-1)}{2}$

Optimal? Im schlechtesten Fall und wenn wir alle Kreuzungen ausgeben wollen: ja.

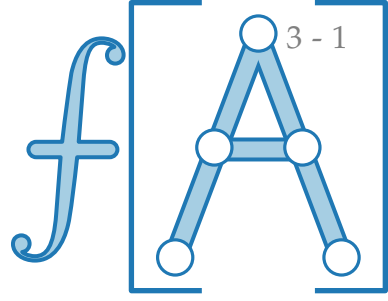
Frage. Was, wenn es *nicht so viele* Kreuzungen gibt?

Idee. Verarbeite Segmente top-to-bottom mit einer **Sweep-Line**.



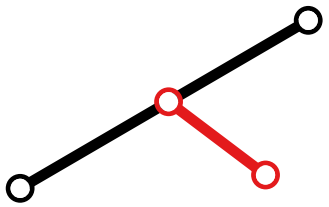
Schnitte von Liniensegmenten

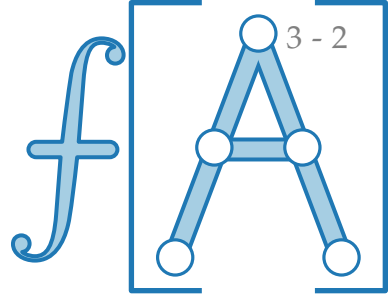
Frage.



Schnitte von Liniensegmenten

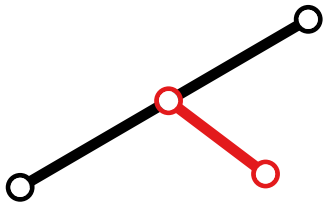
Frage.

Ist  ein Schnittpunkt?



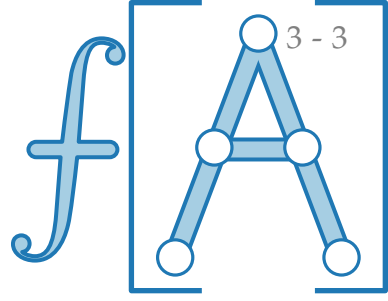
Schnitte von Liniensegmenten

Frage.

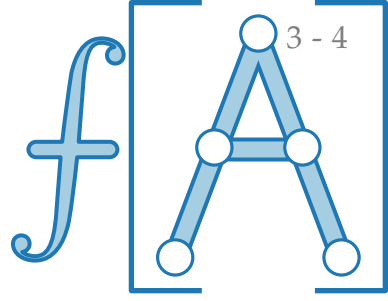
Ist  ein Schnittpunkt?

Antwort.

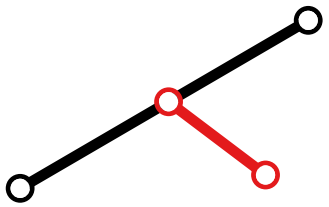
Kommt drauf an...



Schnitte von Liniensegmenten



Frage.

Ist  ein Schnittpunkt?

Antwort.

Kommt drauf an...

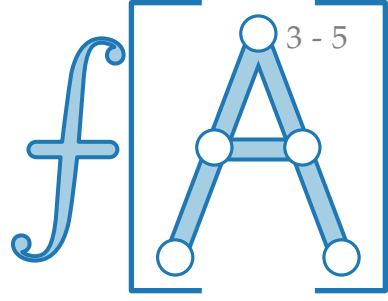
Problem.

LINIENSEGMENTSCHNITTE

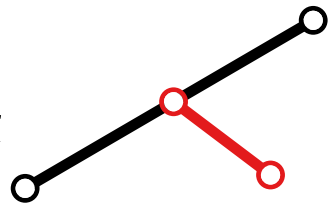
Geg.

Ges.

Schnitte von Liniensegmenten



Frage.

Ist  ein Schnittpunkt?

Antwort.

Kommt drauf an...

Problem.

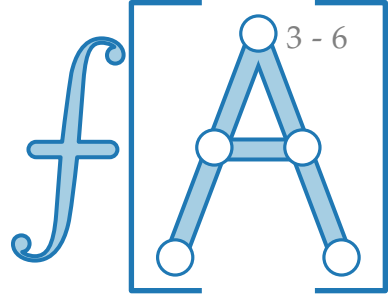
LINIENSEGMENTSCHNITTE

Geg.

Eine Menge S von n **geschlossenen** nicht-überlappenden Liniensegmenten in der Ebene.

Ges.

Schnitte von Liniensegmenten



Frage.



Antwort.

Kommt drauf an...

Problem.

LINIENSEGMENTSCHNITTE

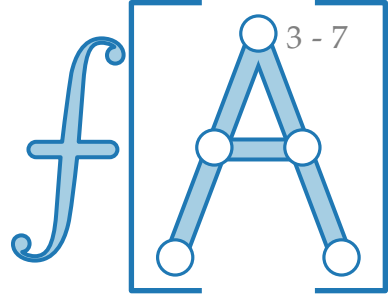
Geg.

Eine Menge S von n **geschlossenen** nicht-überlappenden Liniensegmenten in der Ebene.

Ges.

- Alle Punkte, in denen sich mindestens zwei Segmente schneiden

Schnitte von Liniensegmenten



Frage.



Antwort.

Kommt drauf an...

Problem.

LINIENSEGMENTSCHNITTE

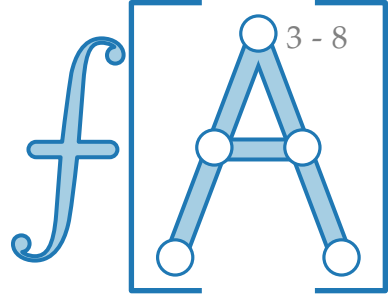
Geg.

Eine Menge S von n **geschlossenen** nicht-überlappenden Liniensegmenten in der Ebene.

Ges.

- Alle Punkte, in denen sich mindestens zwei Segmente schneiden
- Für jeden solchen Punkt alle Segmente, die ihn enthalten

Schnitte von Liniensegmenten



Frage.



Antwort.

Kommt drauf an...

ja!

Problem.

LINIENSEGMENTSCHNITTE

Geg.

Eine Menge S von n **geschlossenen** nicht-überlappenden Liniensegmenten in der Ebene.

Ges.

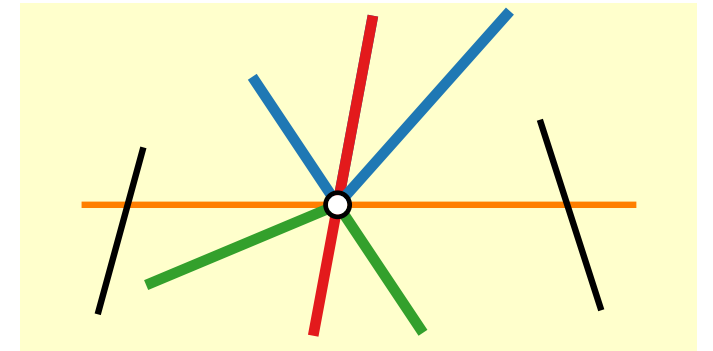
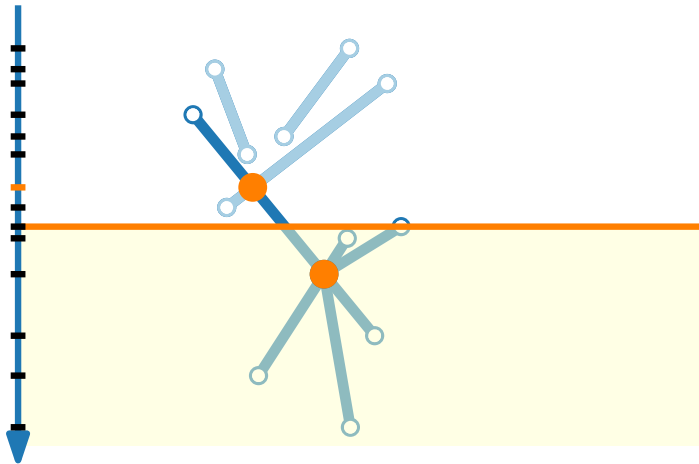
- Alle Punkte, in denen sich mindestens zwei Segmente schneiden
- Für jeden solchen Punkt alle Segmente, die ihn enthalten

Fortgeschrittene Algorithmen

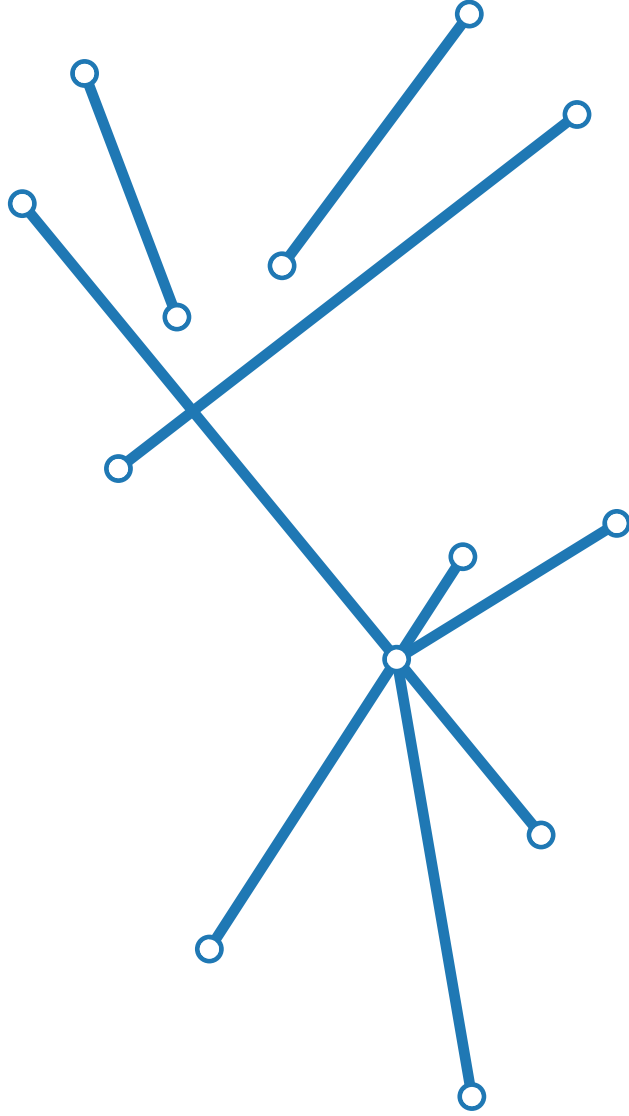
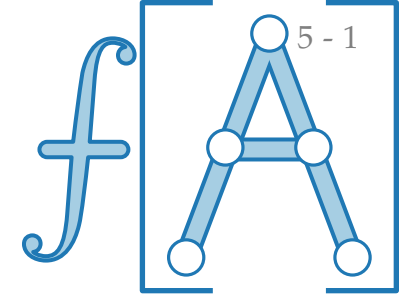
13. Vorlesung

Algorithmische Geometrie II: Sweep-Line Algorithmus

Teil II:
Idee



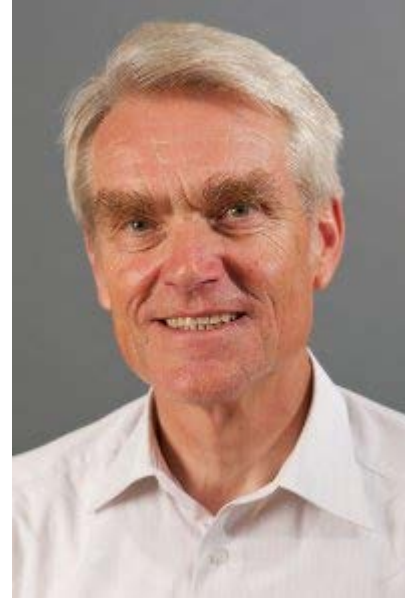
Sweep-Line Algorithmus



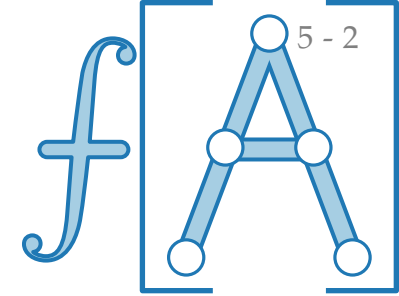
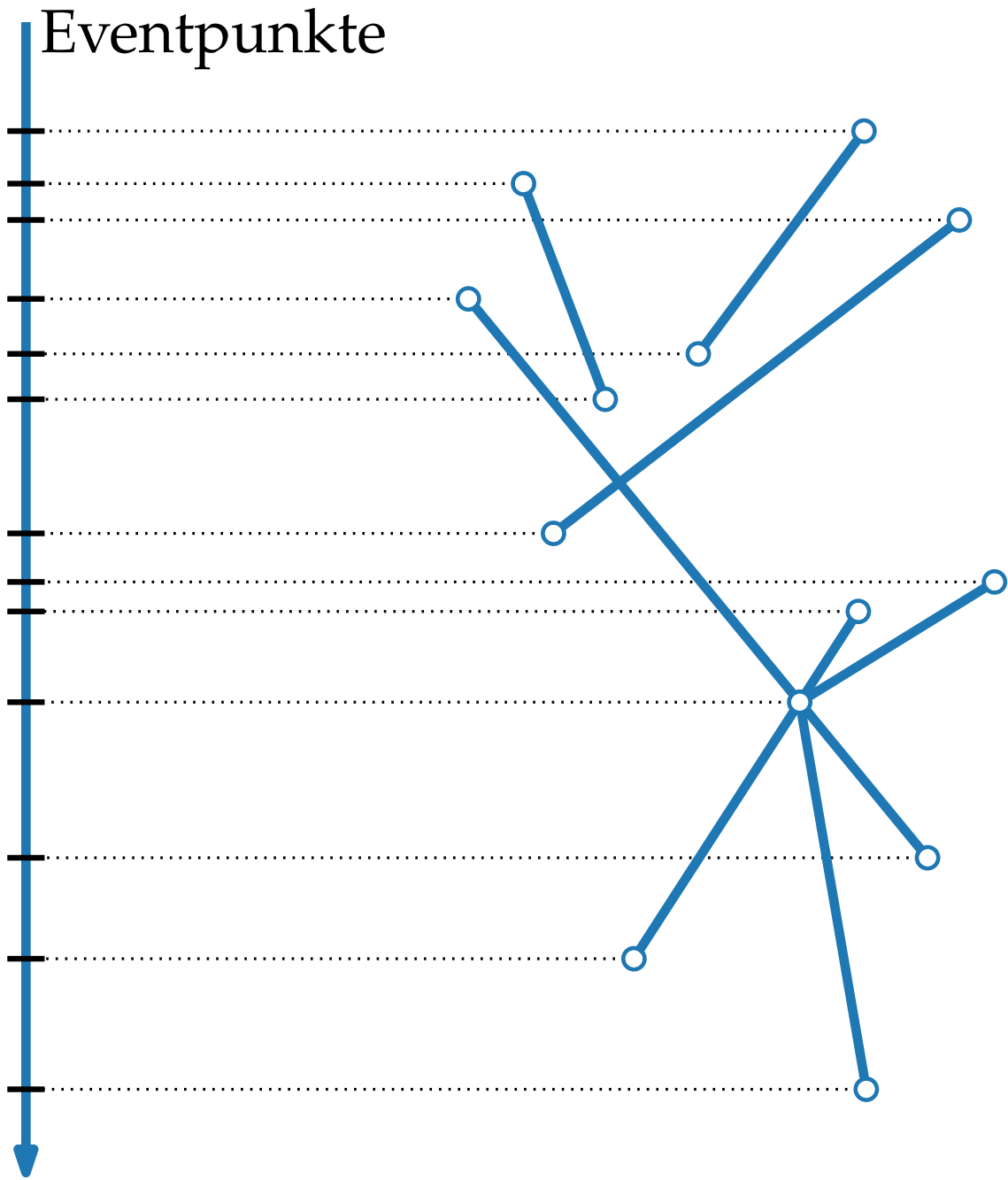
John Bentley
*1953, Long Beach, CA



Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus



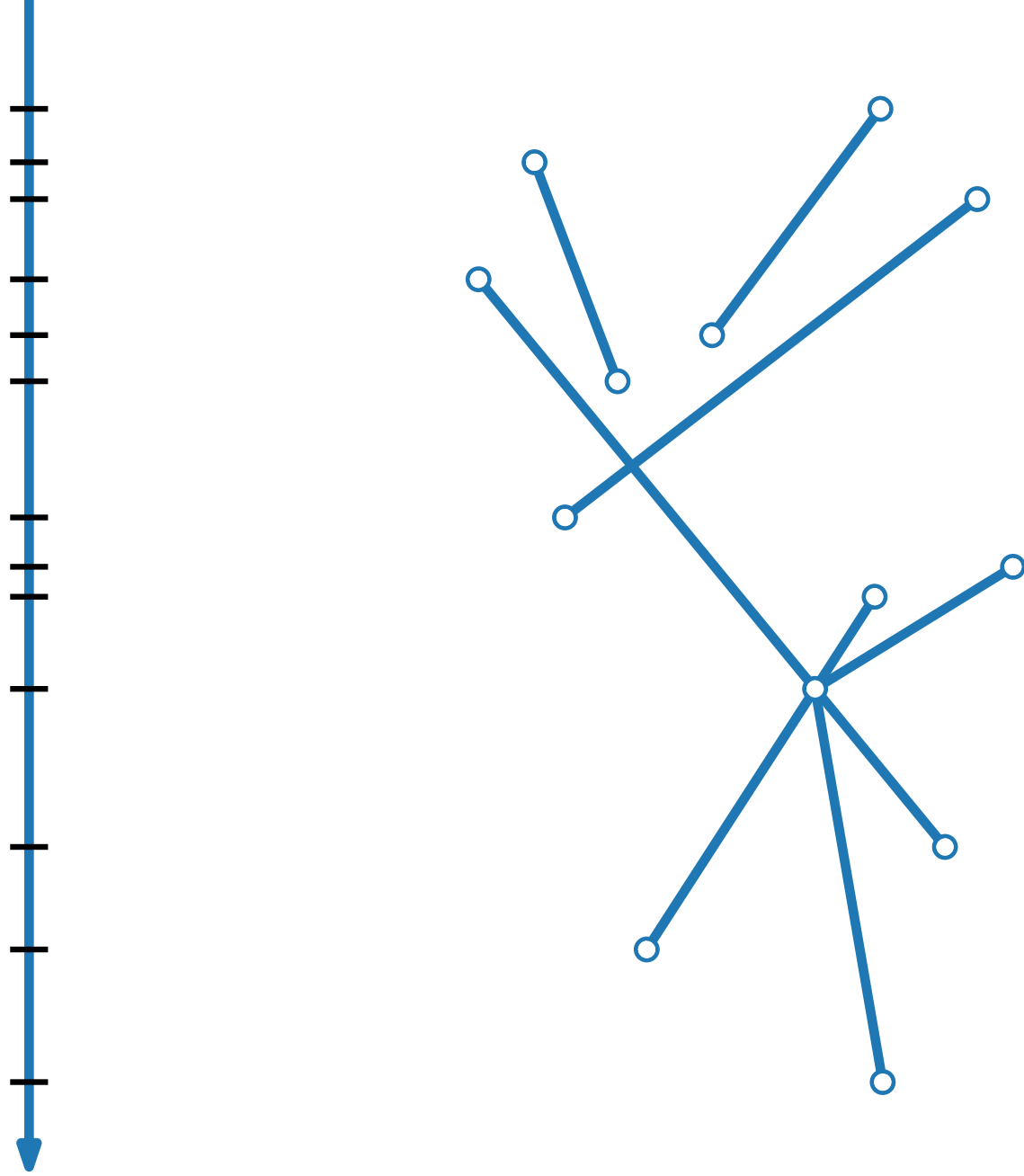
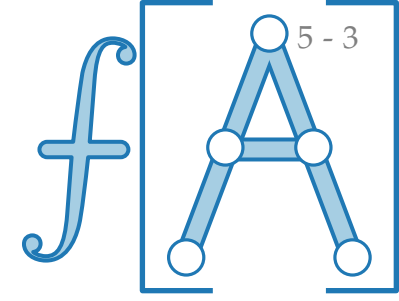
John Bentley
*1953, Long Beach, CA



Thomas Ottmann
*1943, Magdeburg



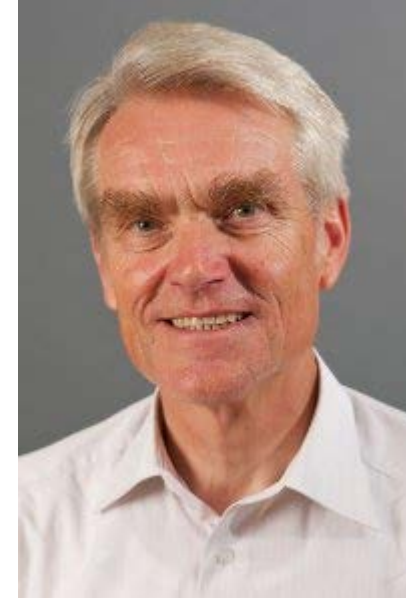
Sweep-Line Algorithmus



John Bentley
*1953, Long Beach, CA

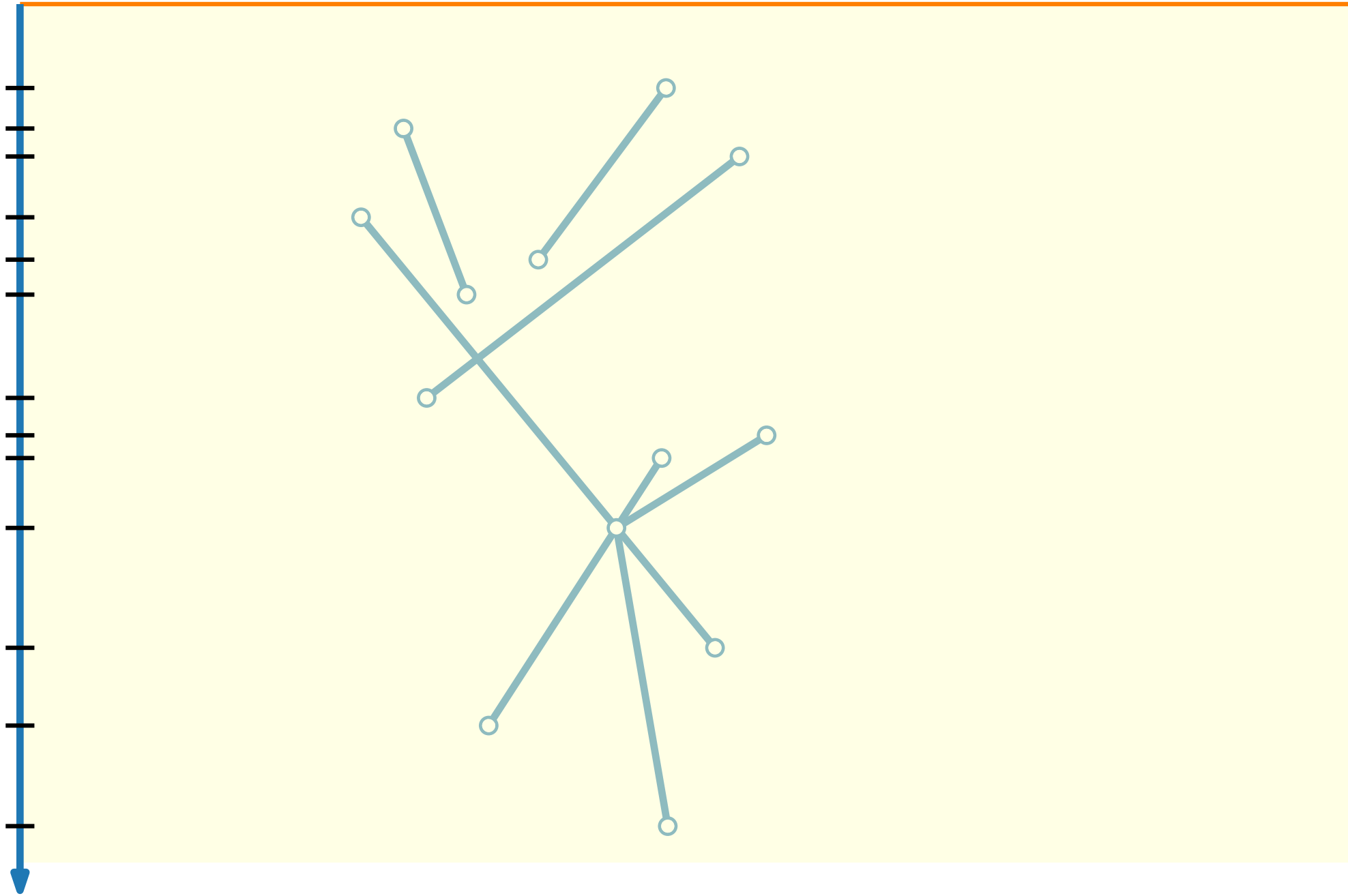
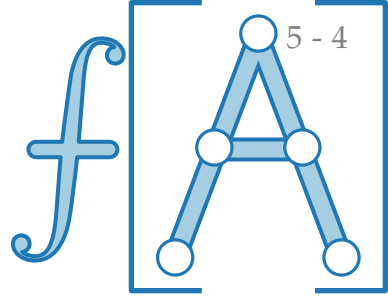


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

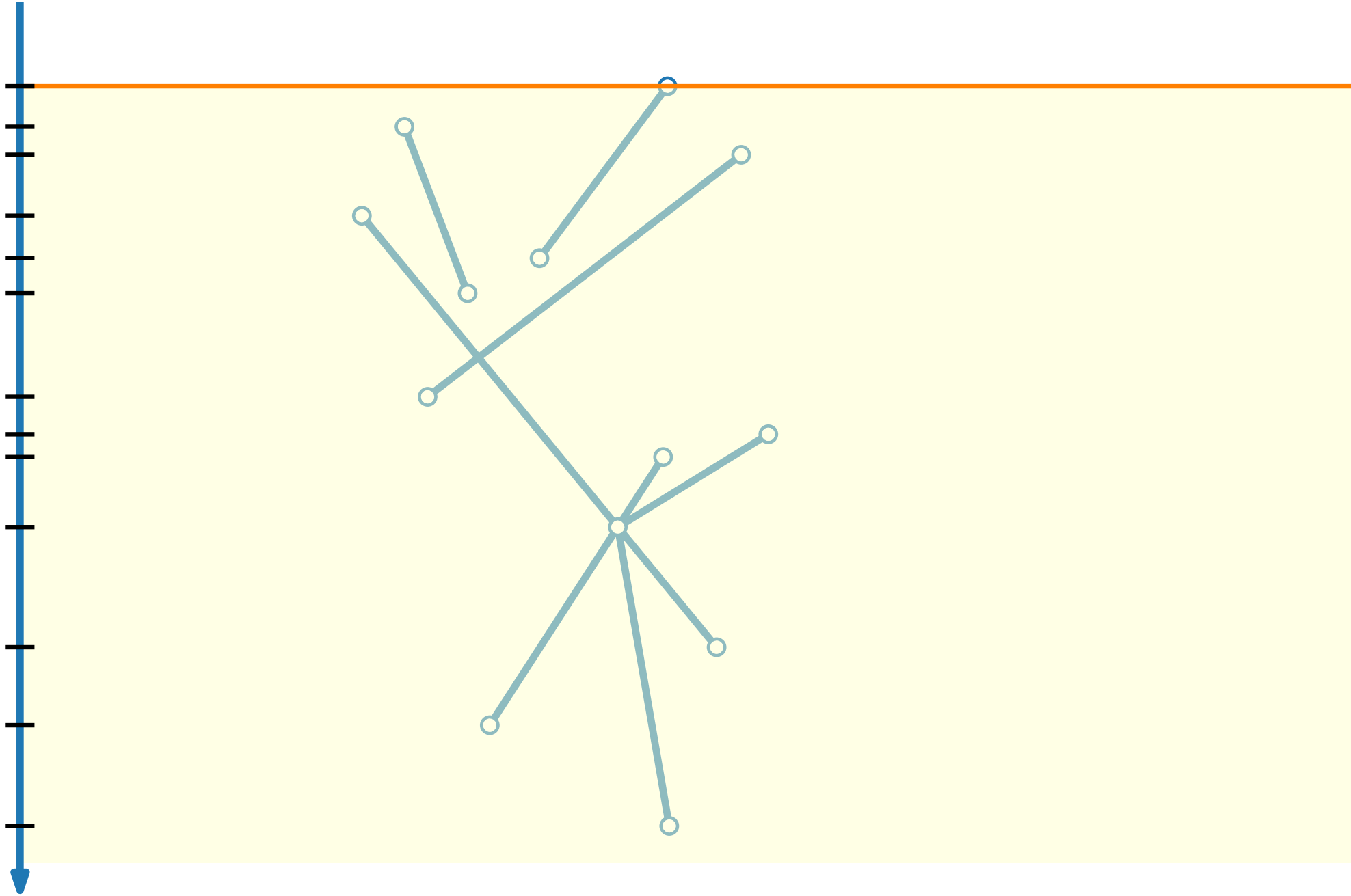
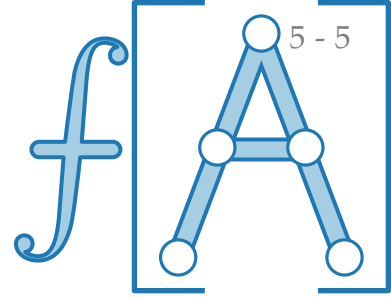


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

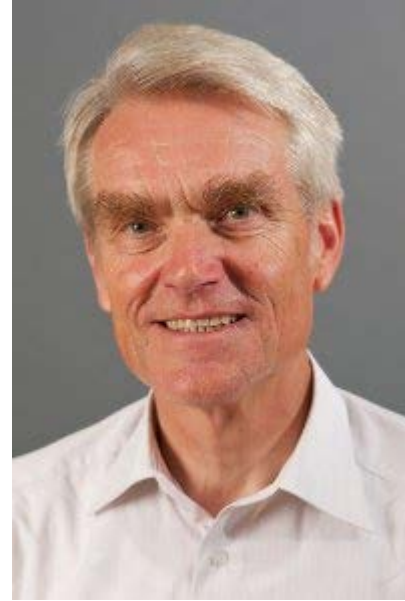
Sweep-Line



John Bentley
*1953, Long Beach, CA

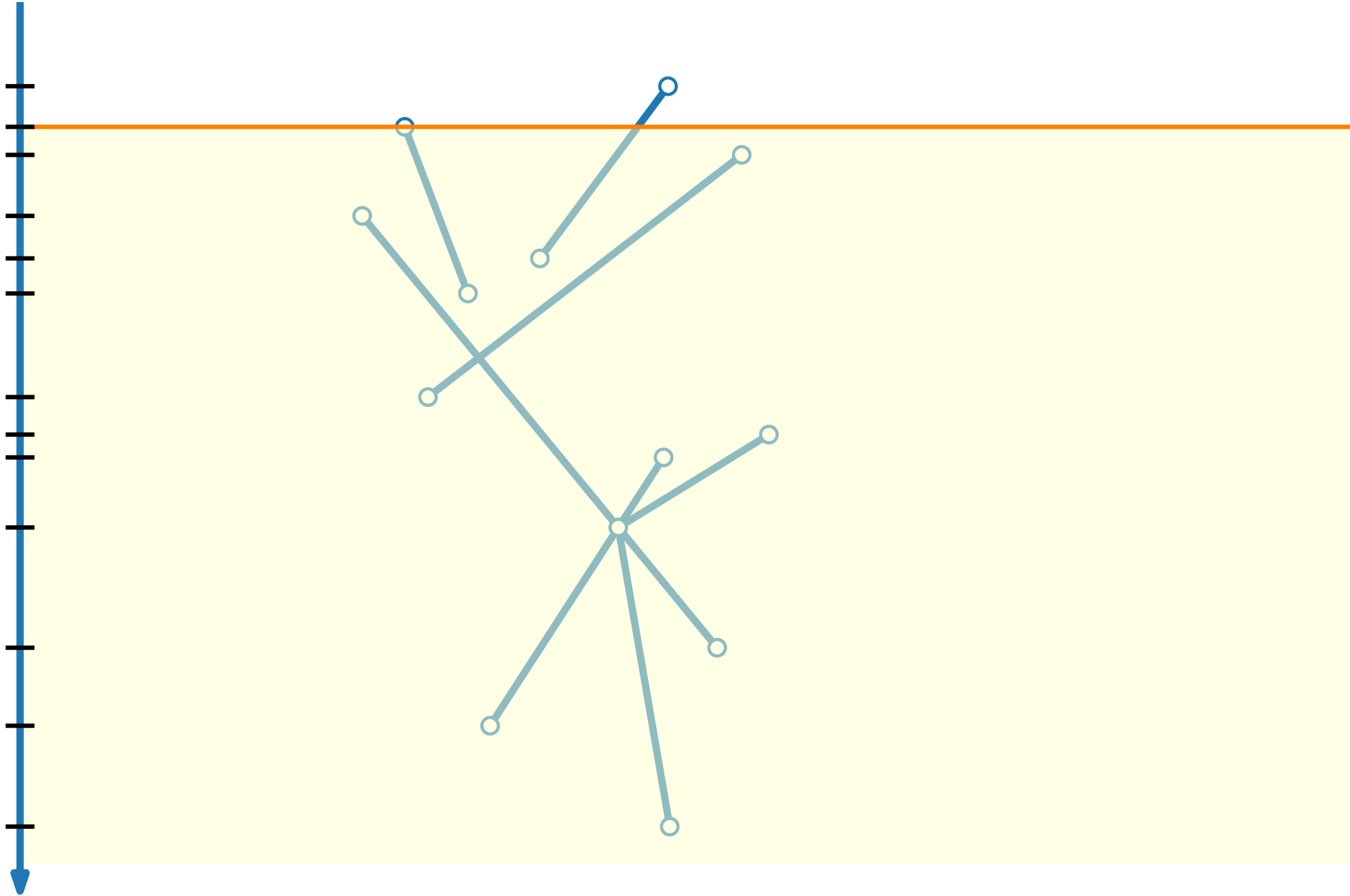
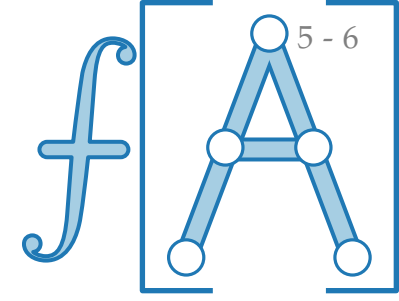


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

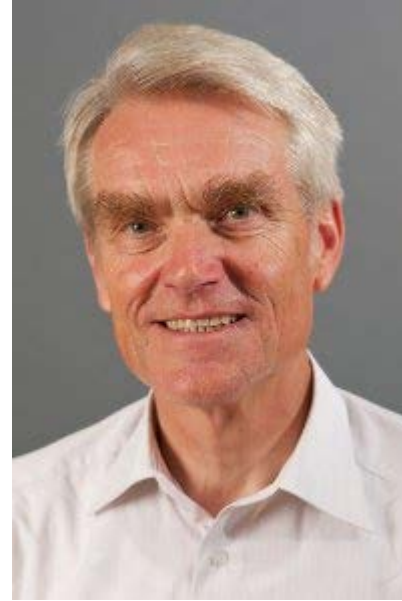
Sweep-Line



John Bentley
*1953, Long Beach, CA

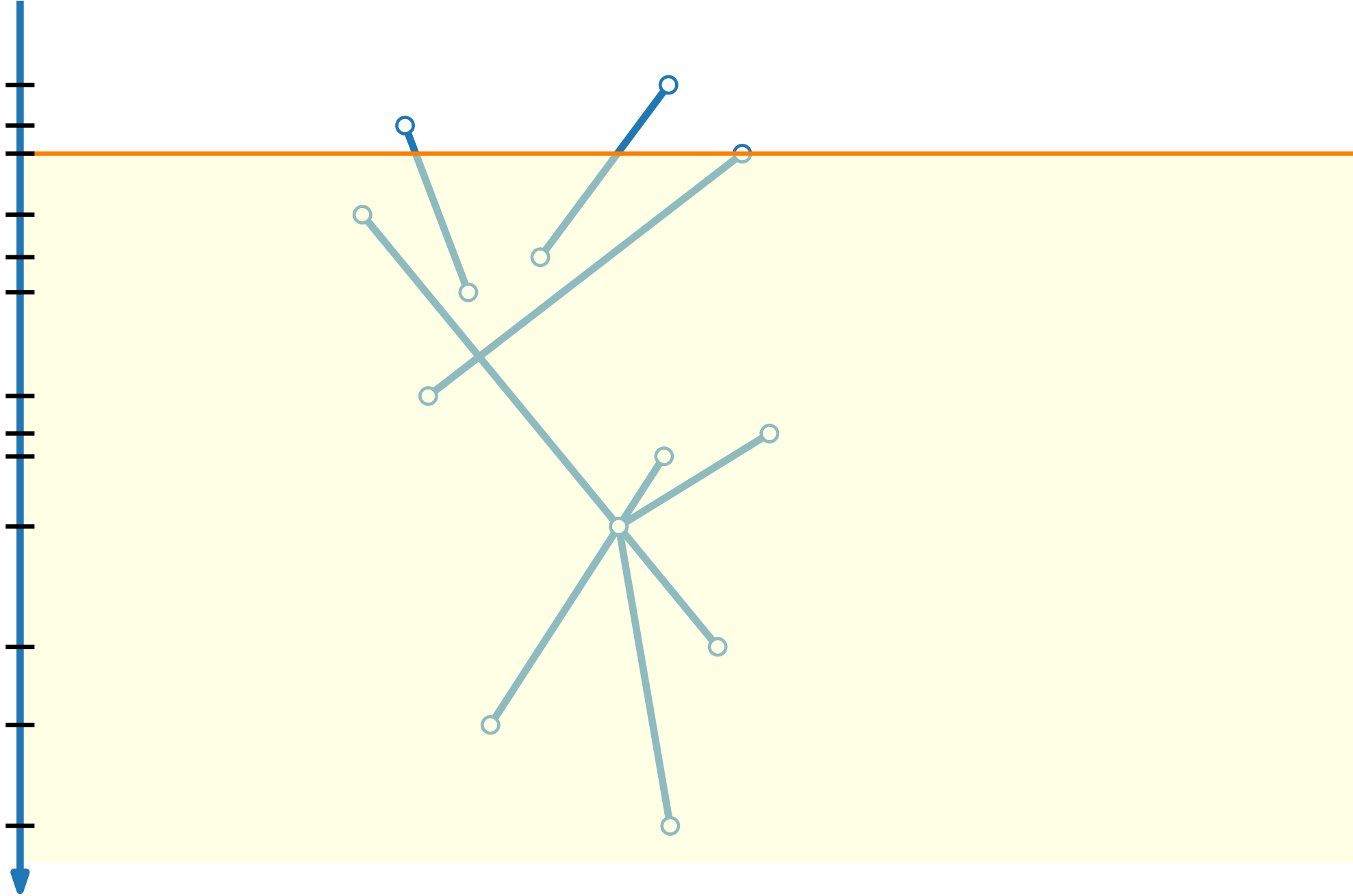
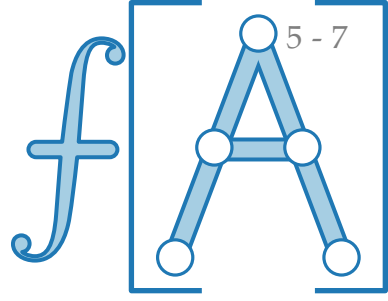


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

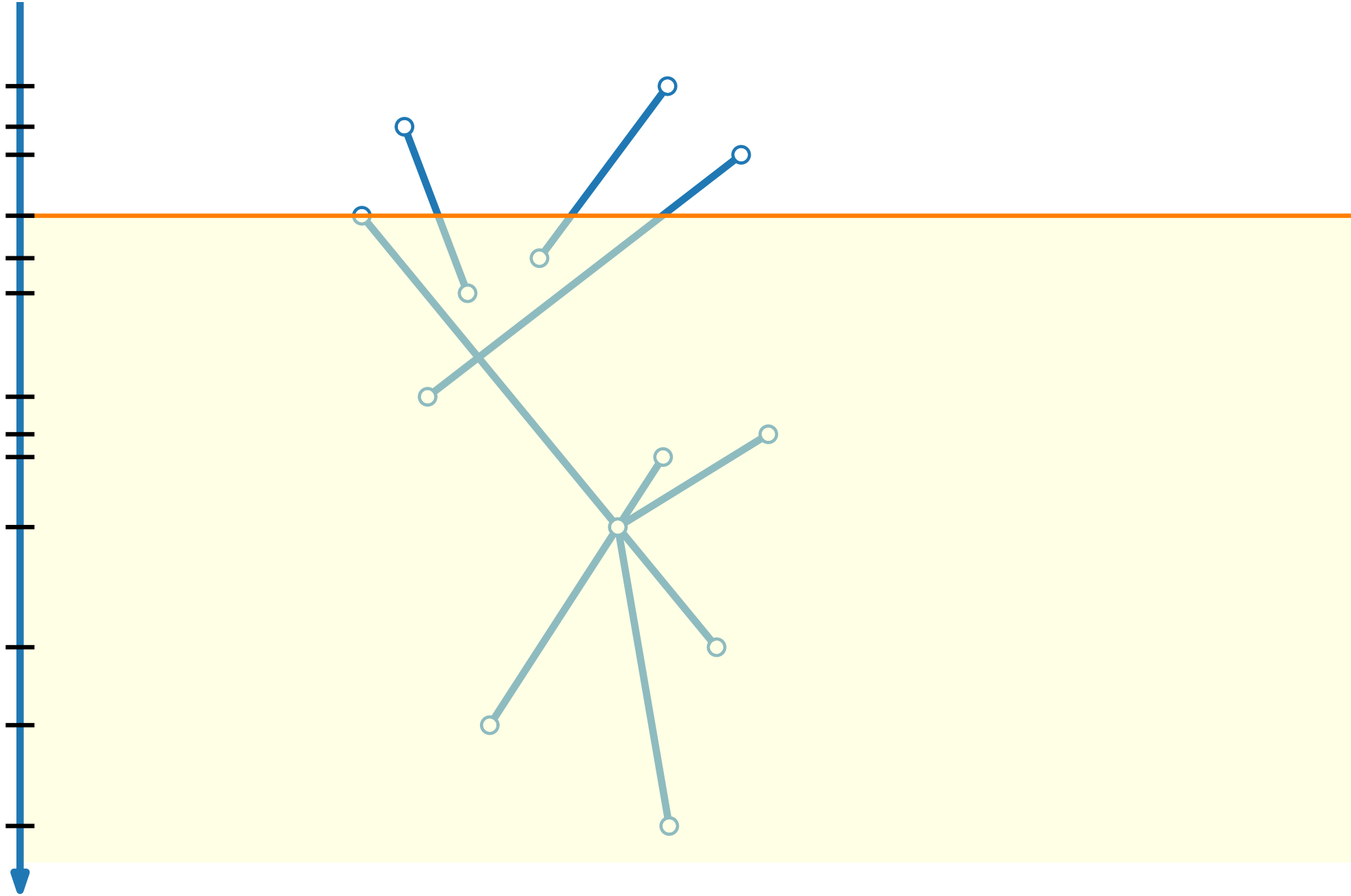
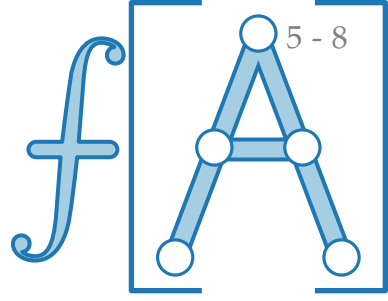


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

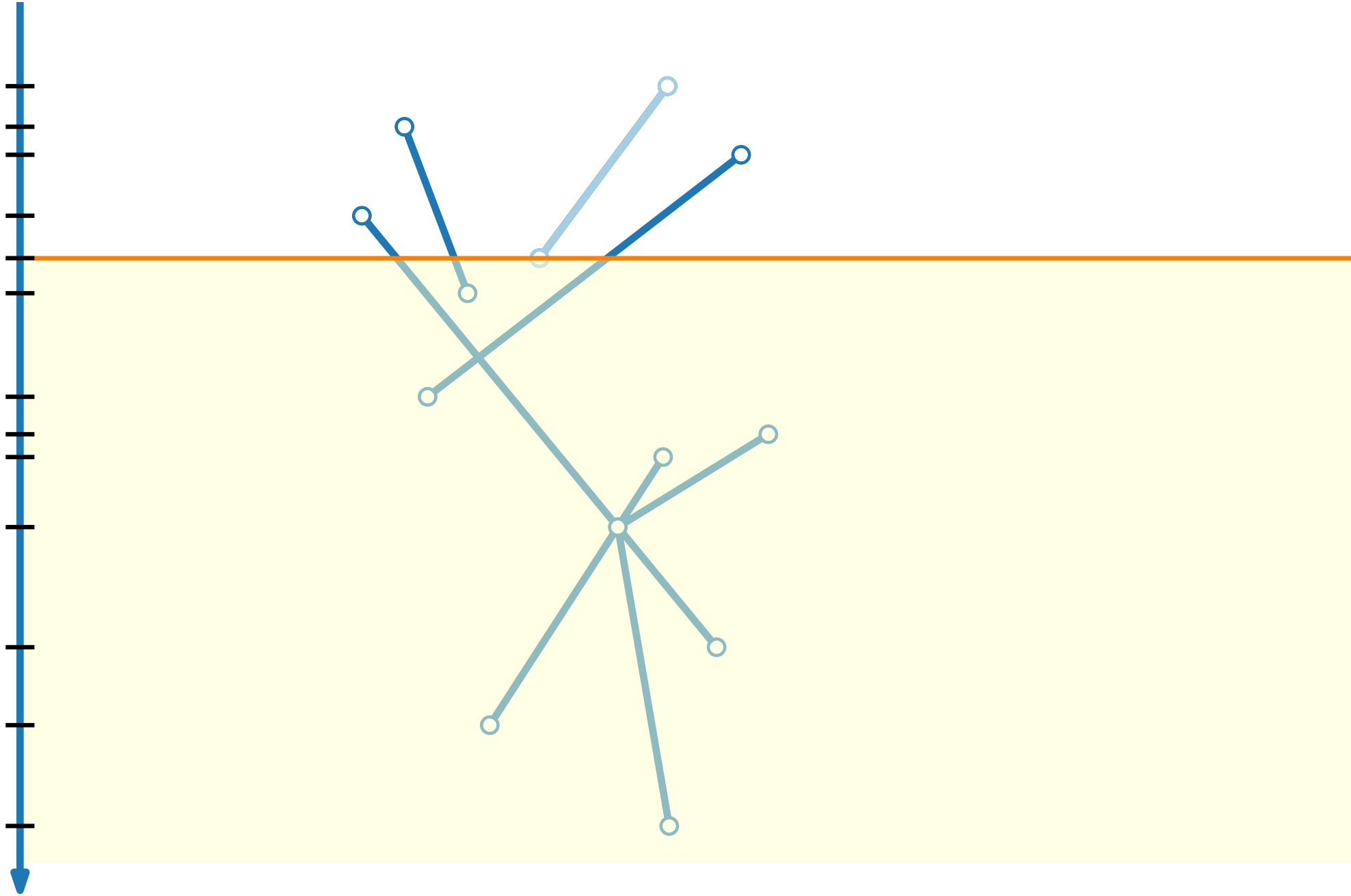
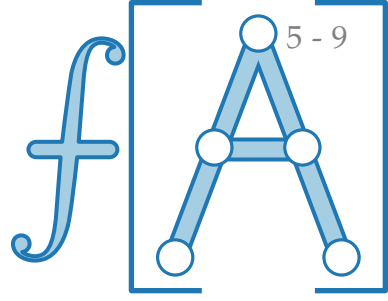


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

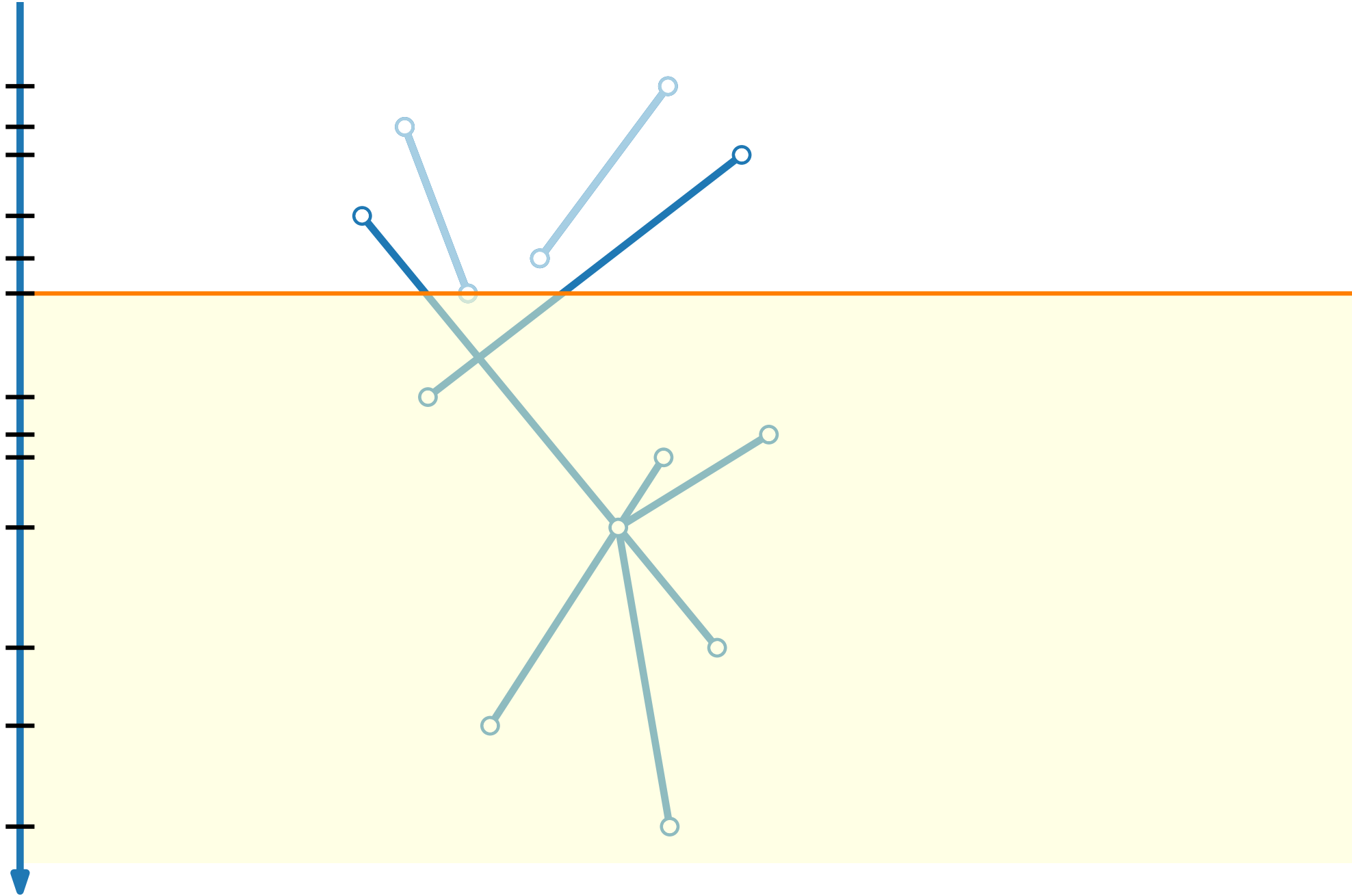
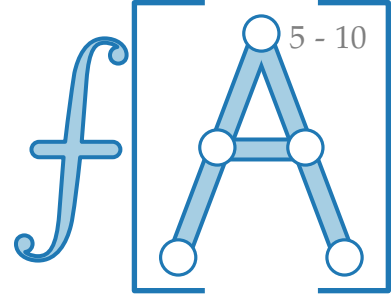


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

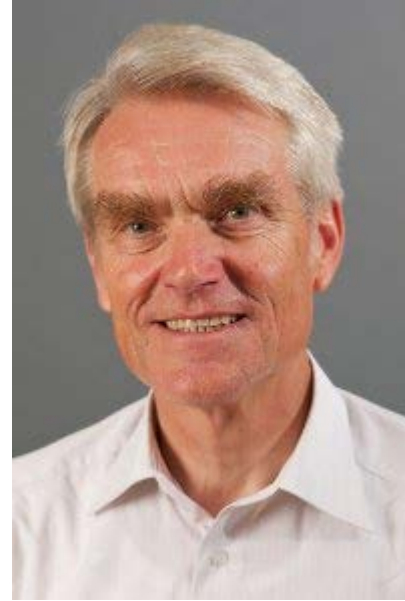
Sweep-Line



John Bentley
*1953, Long Beach, CA

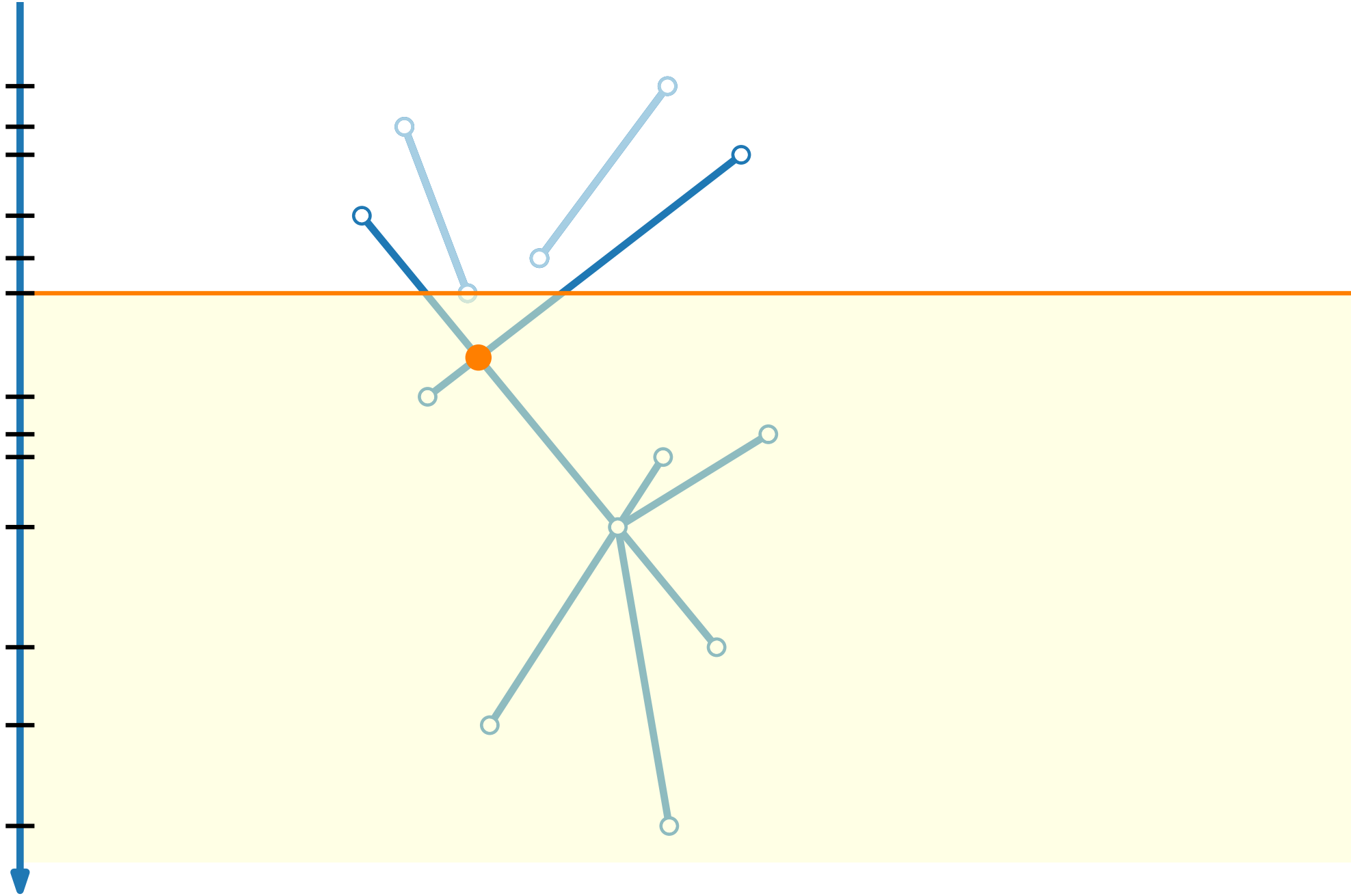
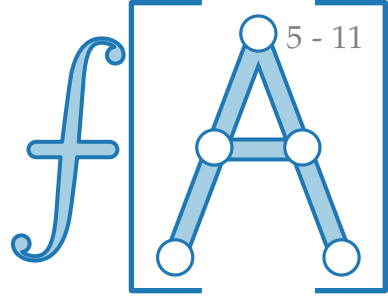


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

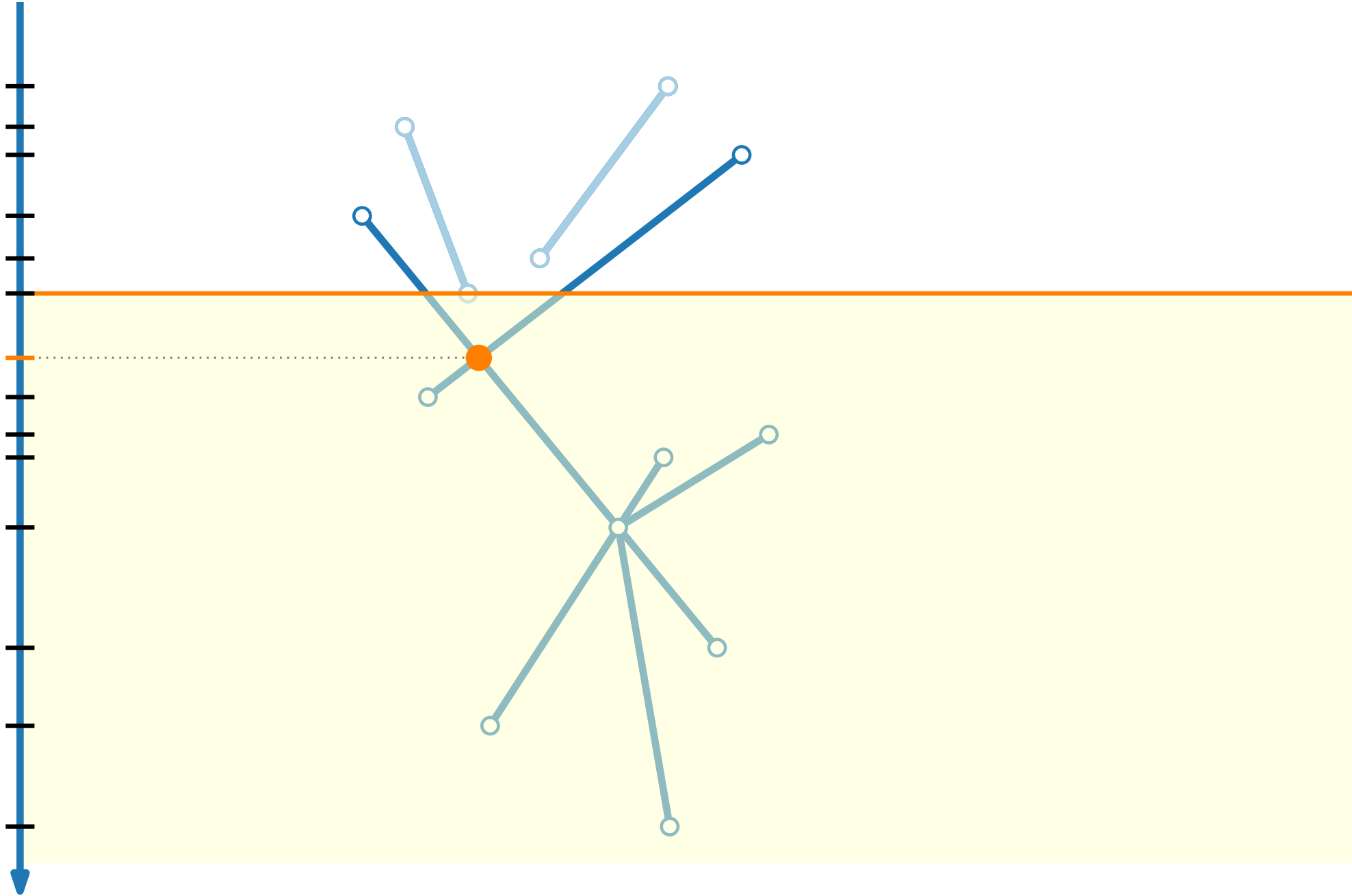
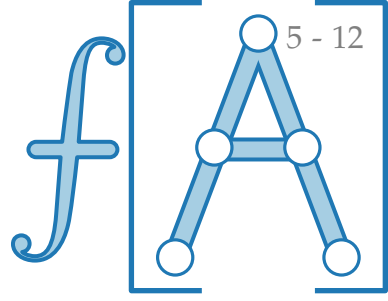


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

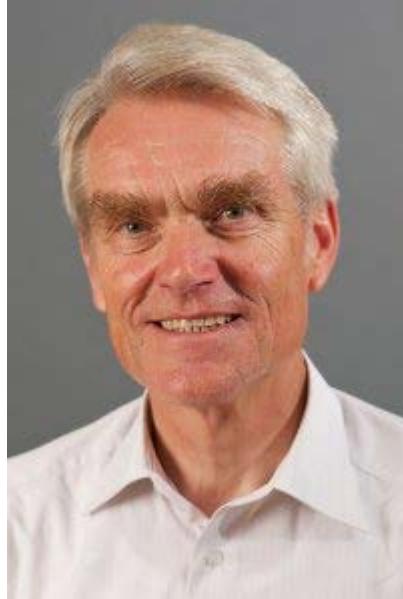
Sweep-Line



John Bentley
*1953, Long Beach, CA

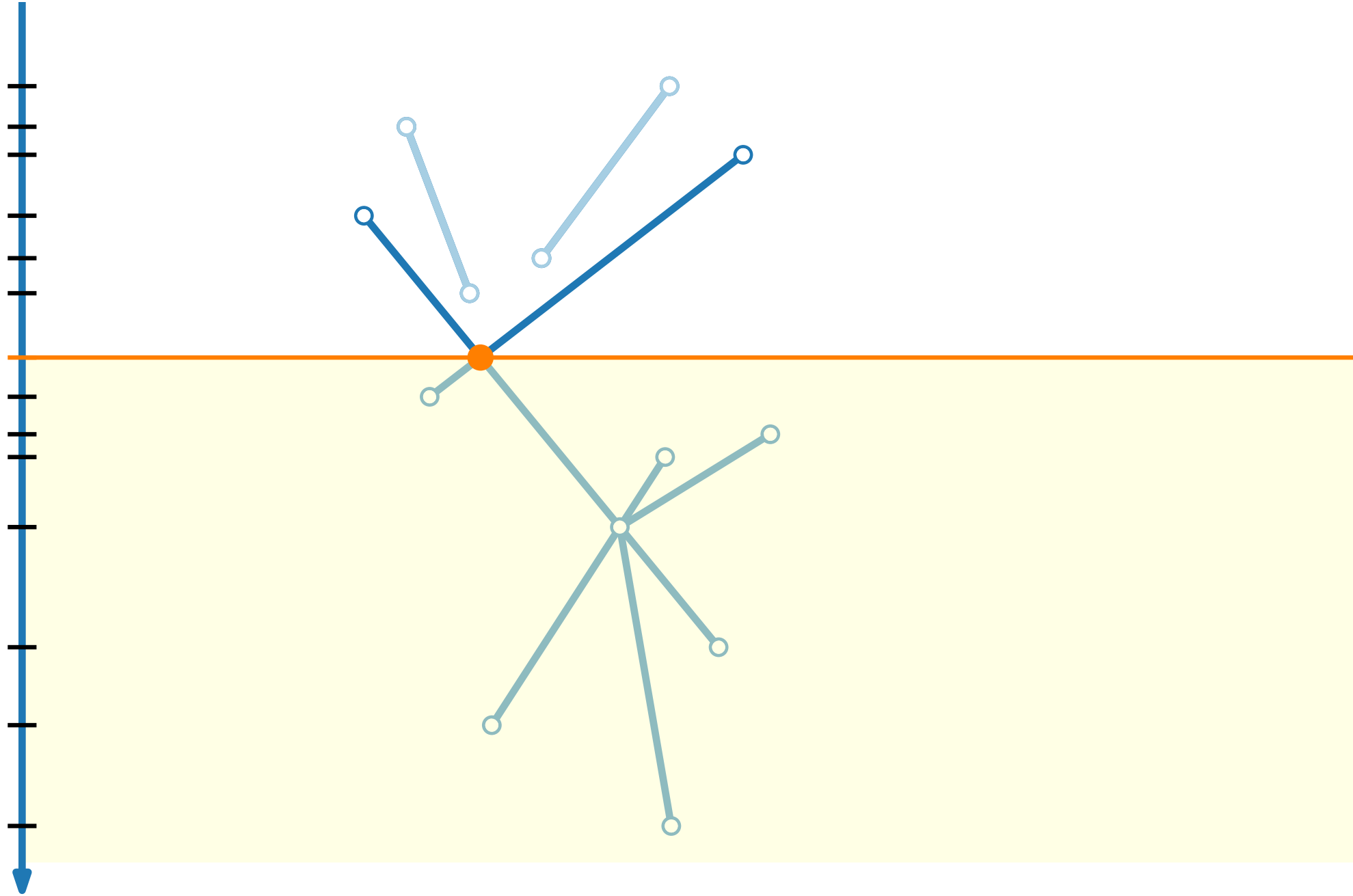
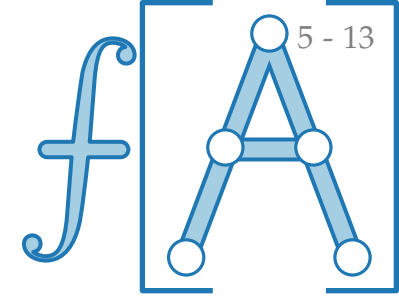


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

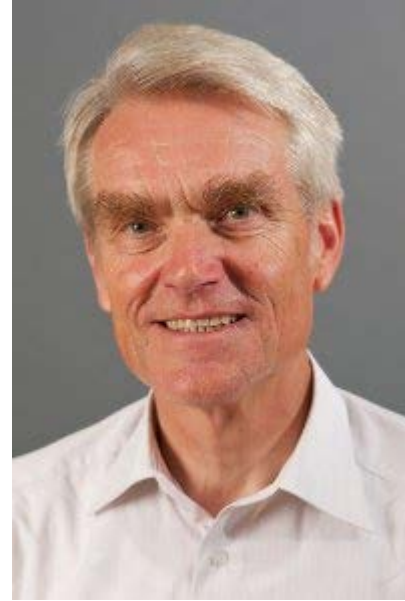
Sweep-Line



John Bentley
*1953, Long Beach, CA

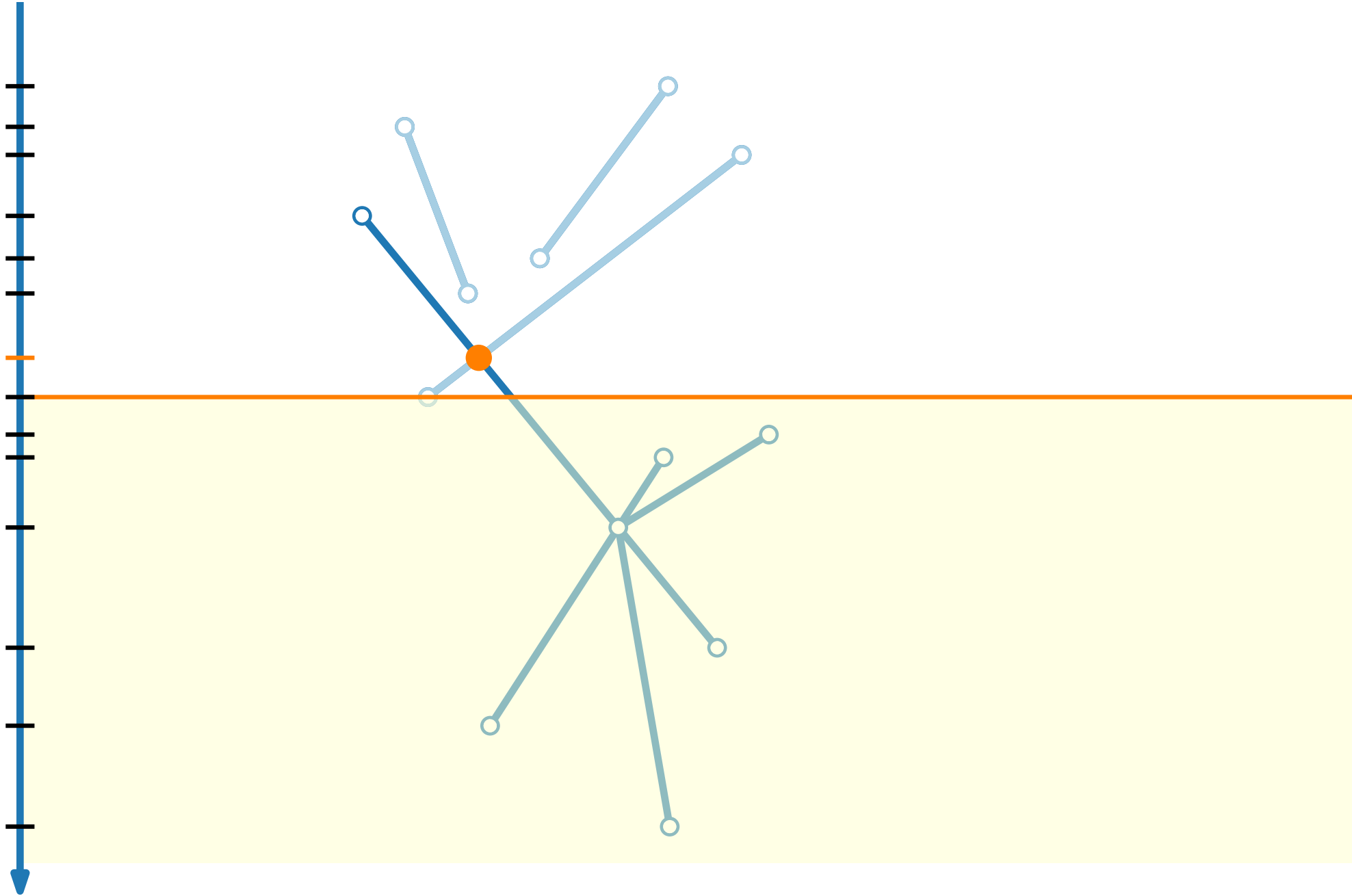
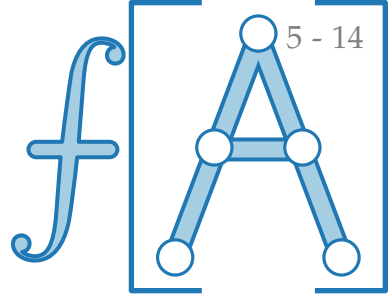


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

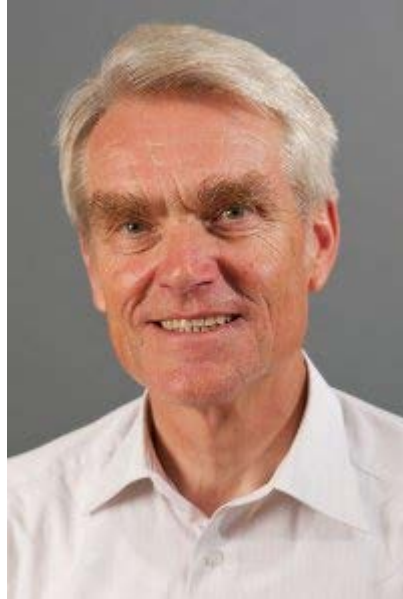
Sweep-Line



John Bentley
*1953, Long Beach, CA

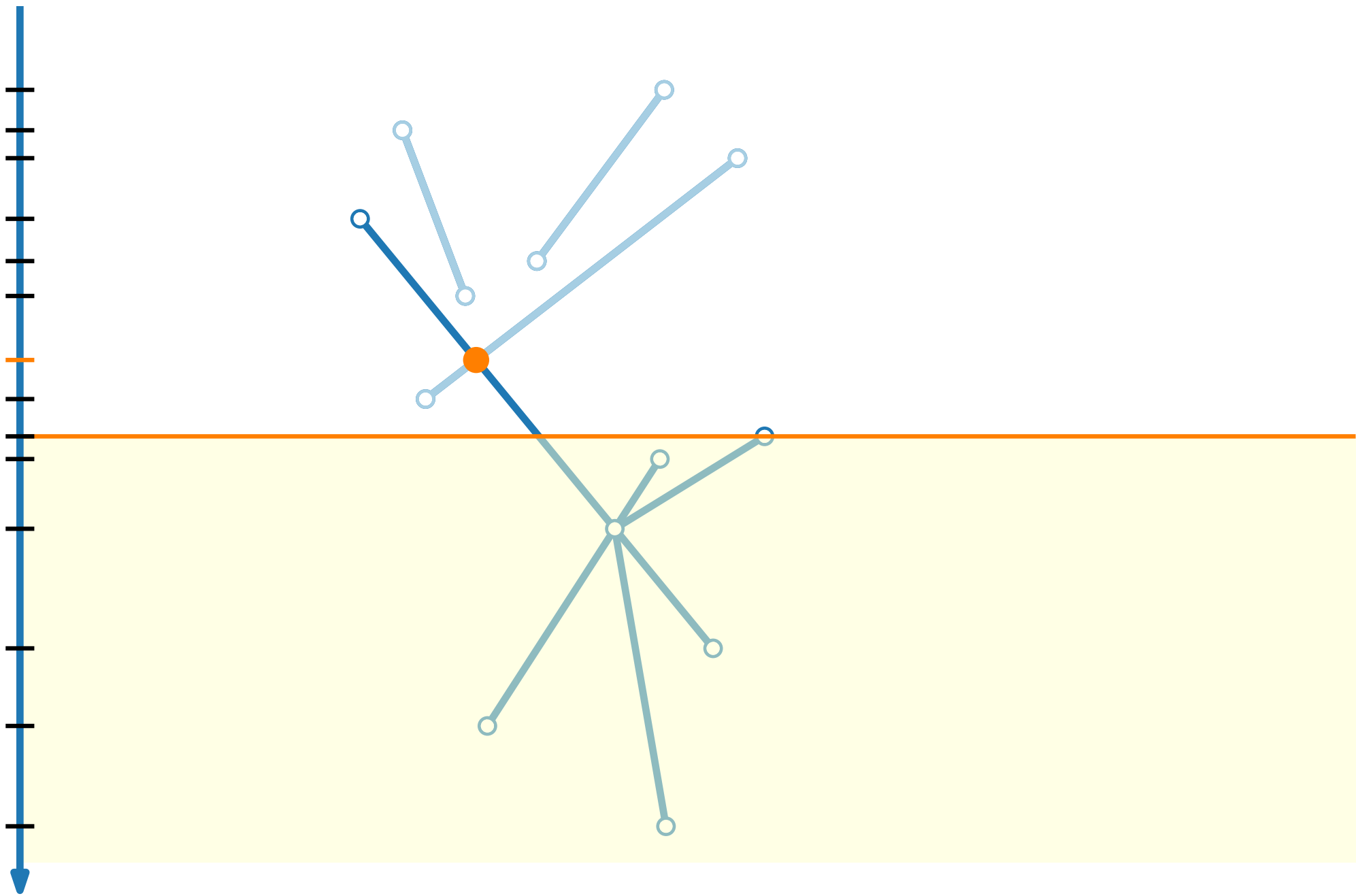
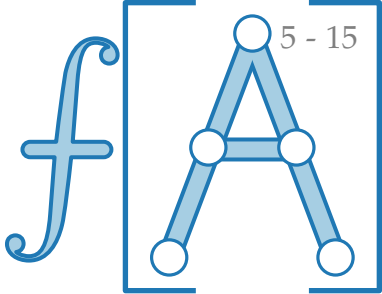


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

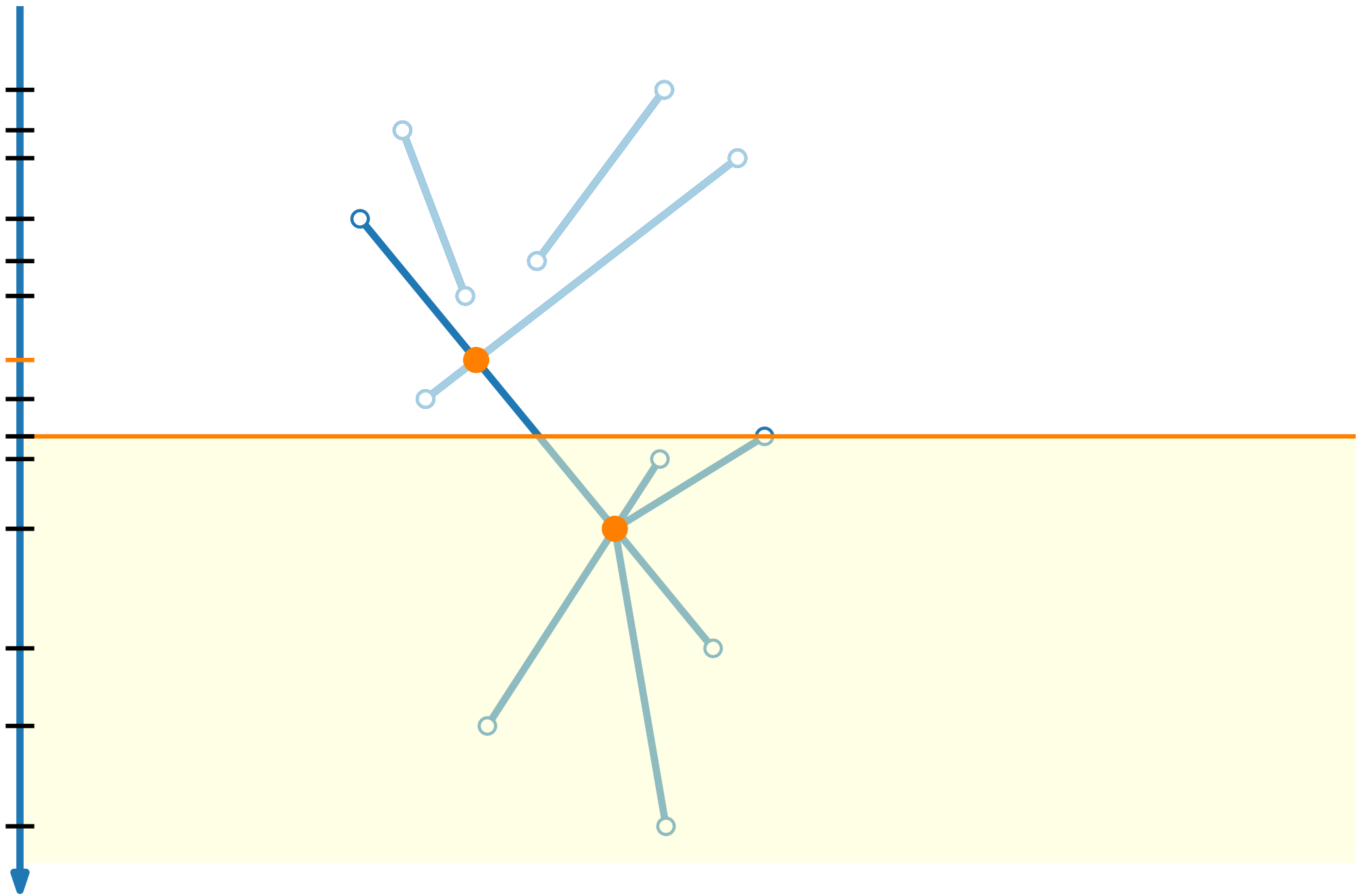
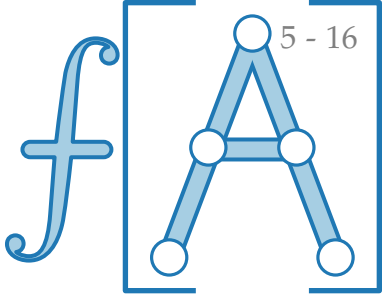


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

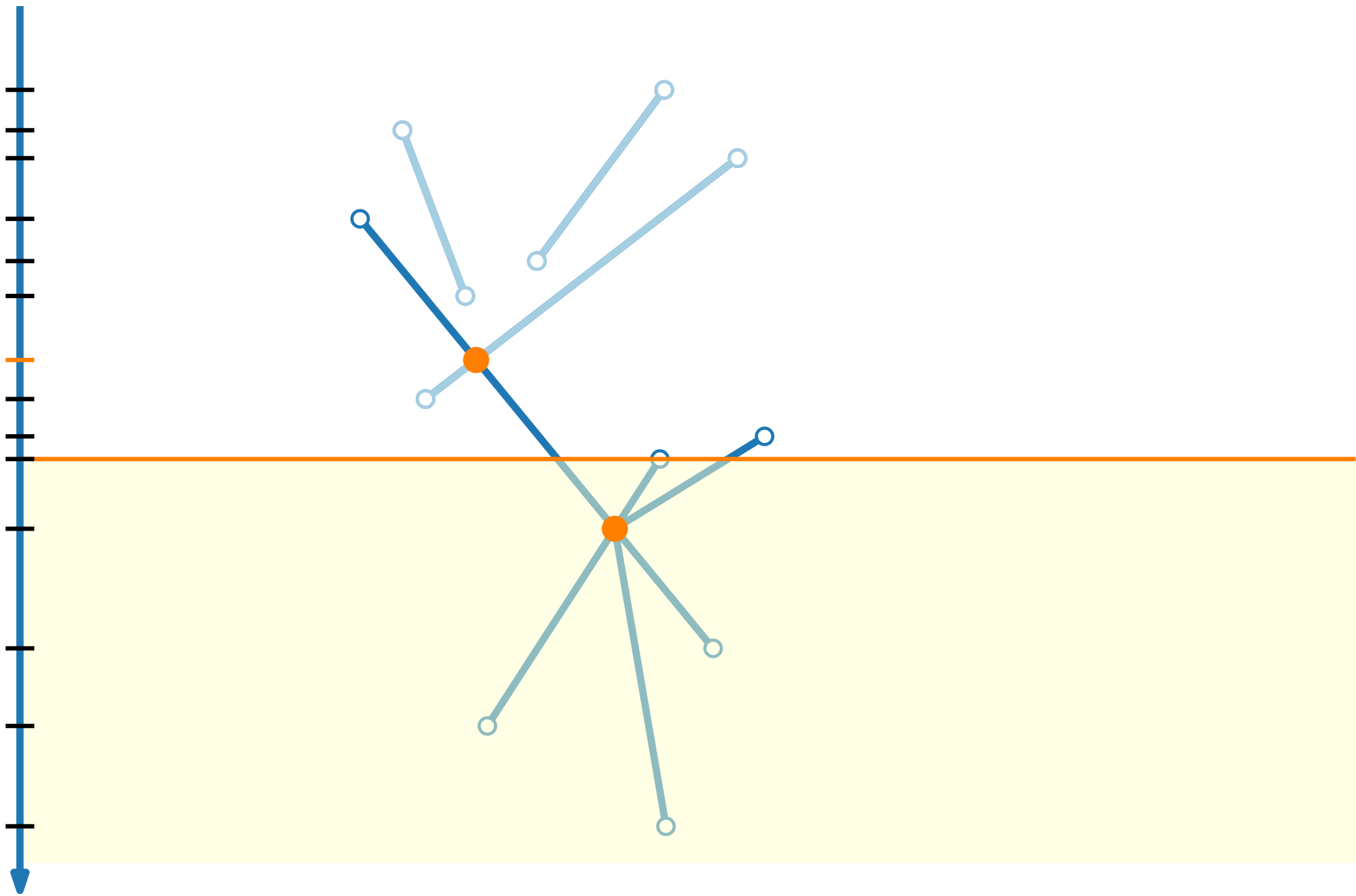
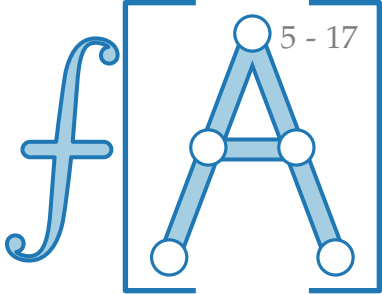


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

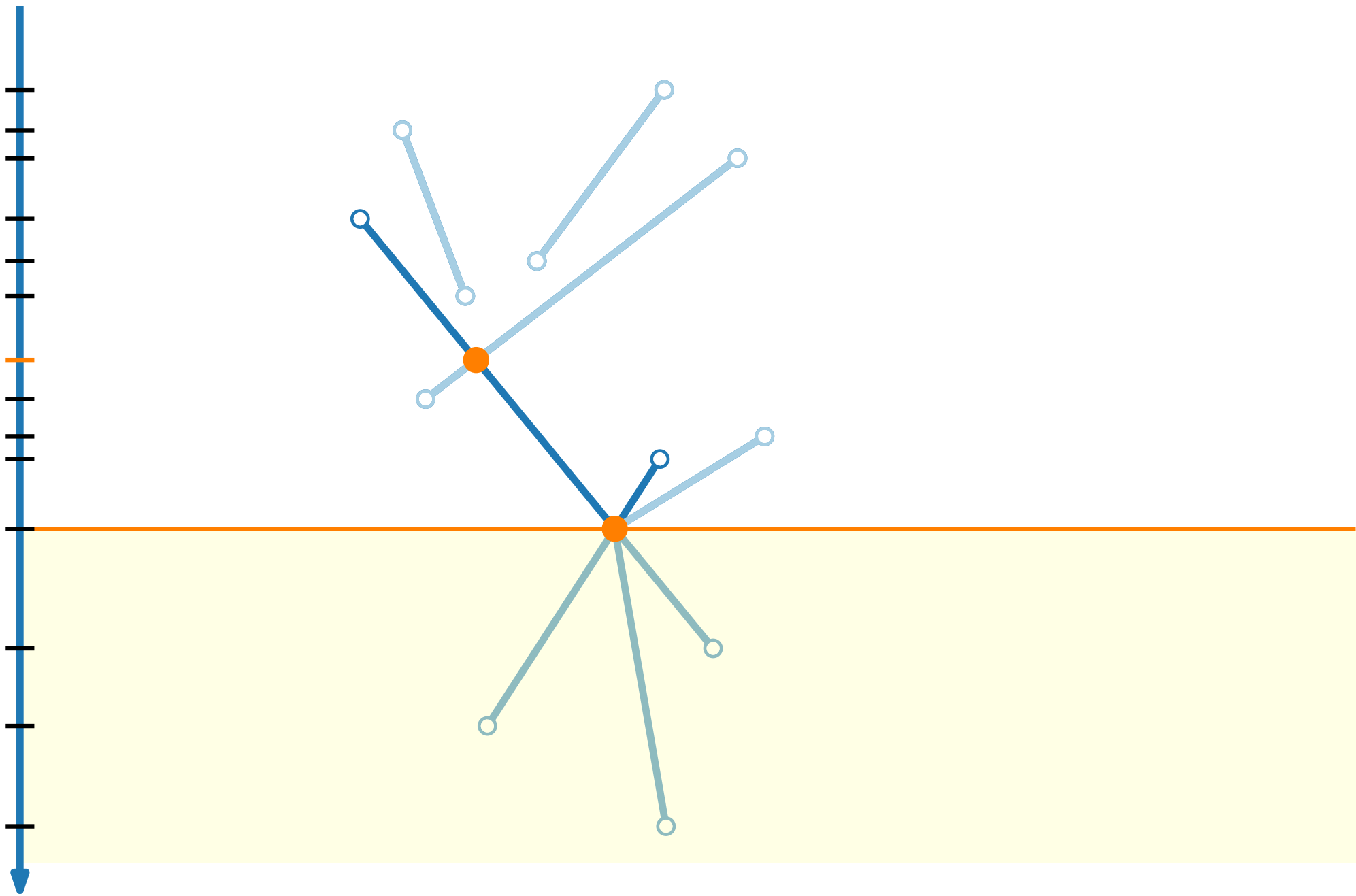
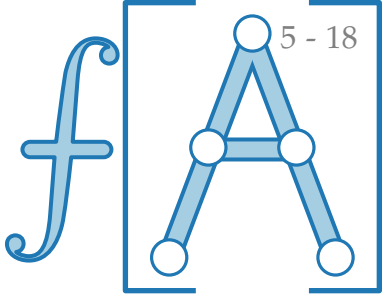


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

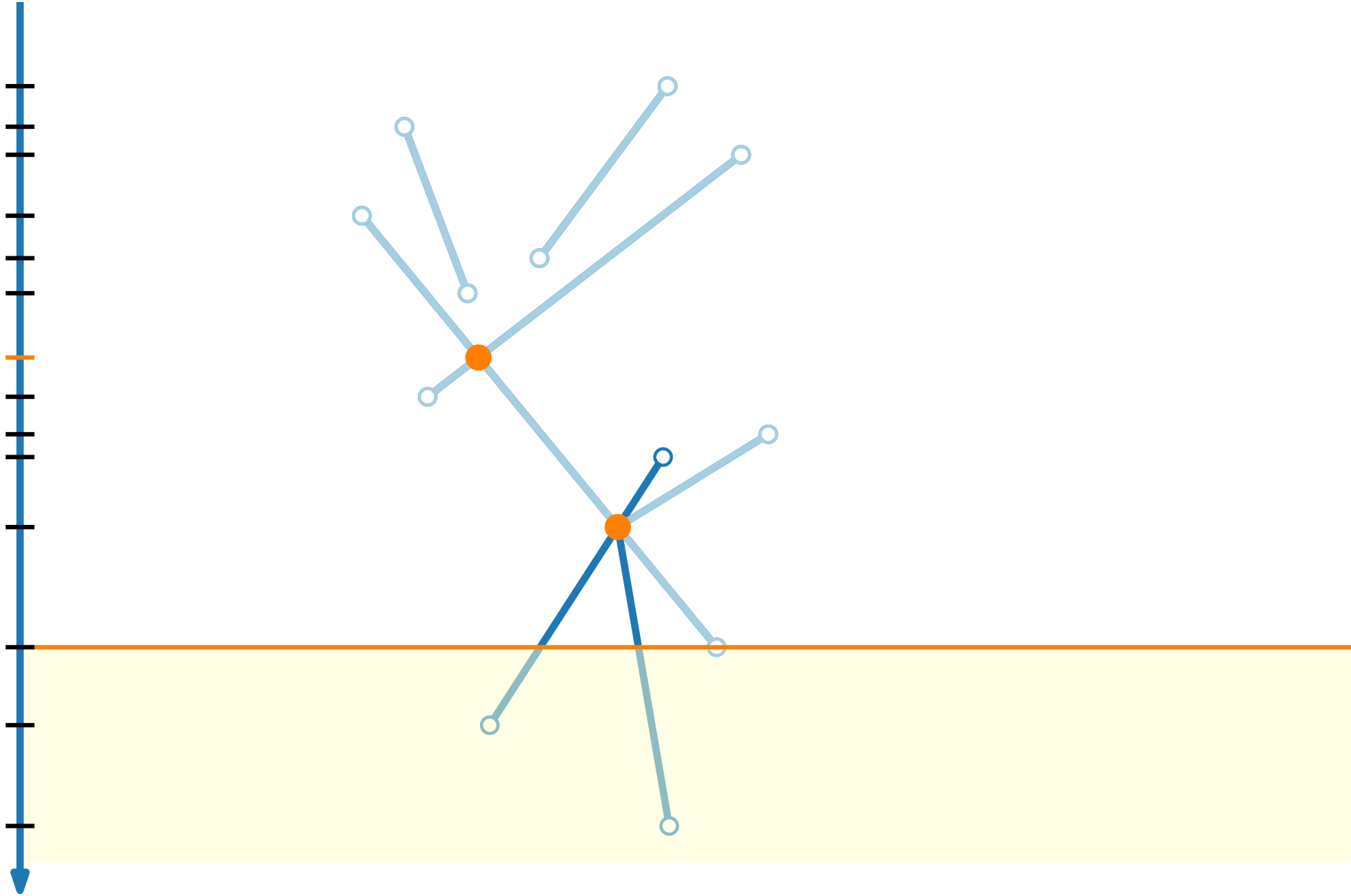
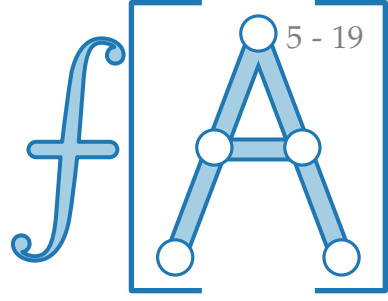


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

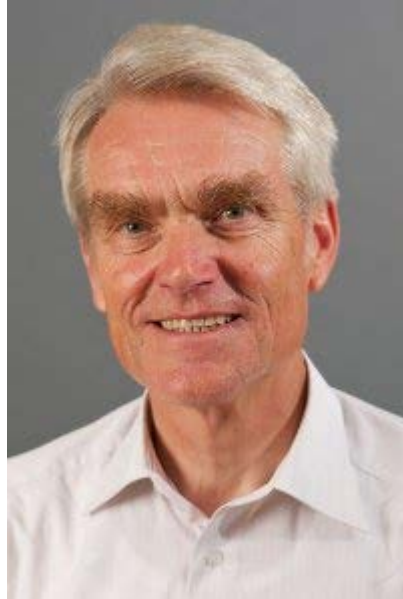
Sweep-Line



John Bentley
*1953, Long Beach, CA

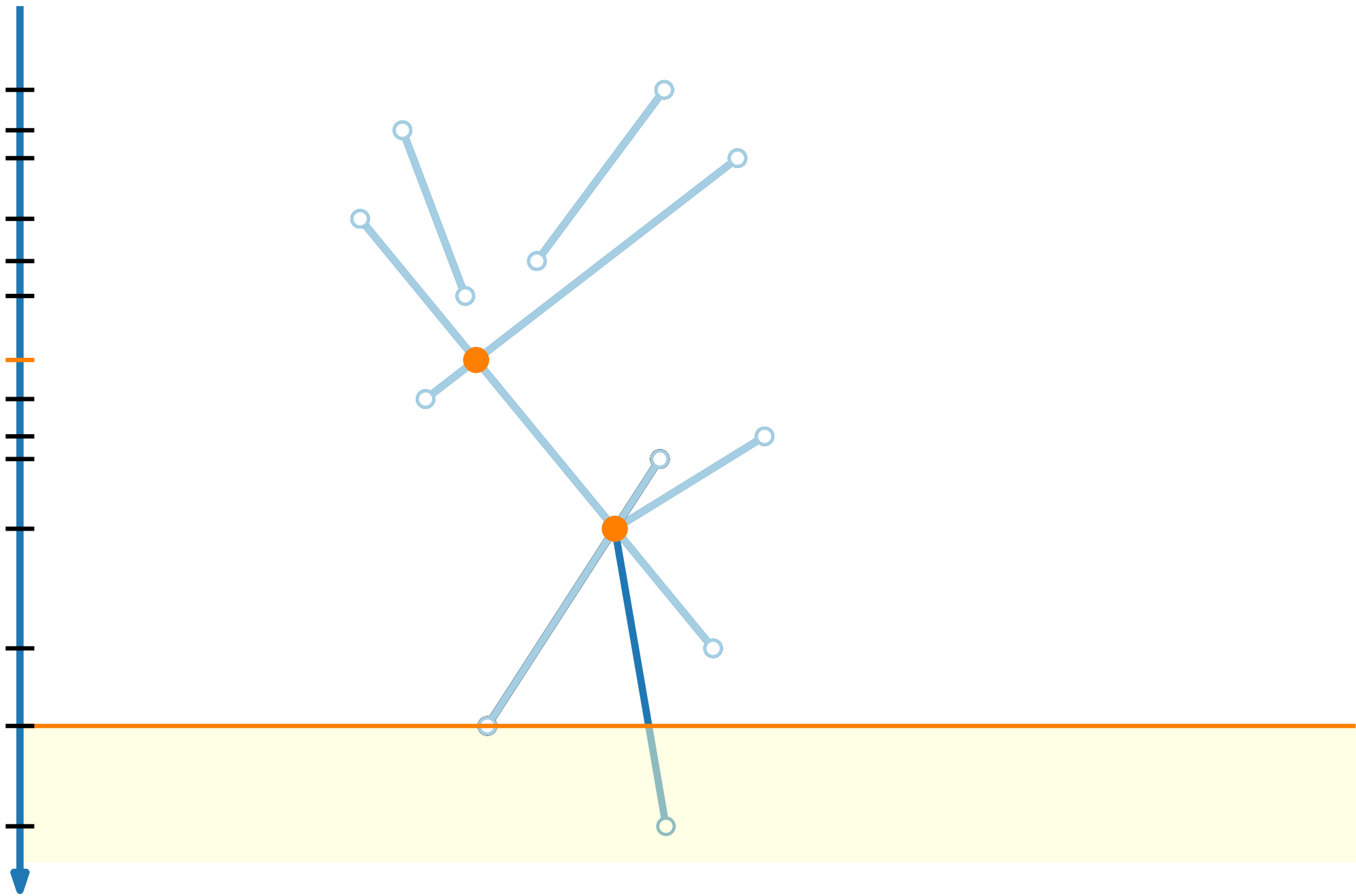
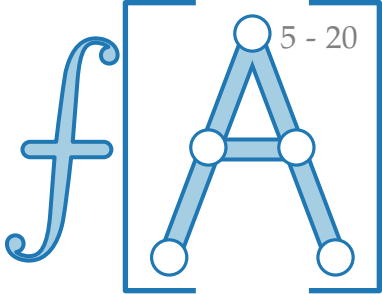


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line



John Bentley
*1953, Long Beach, CA

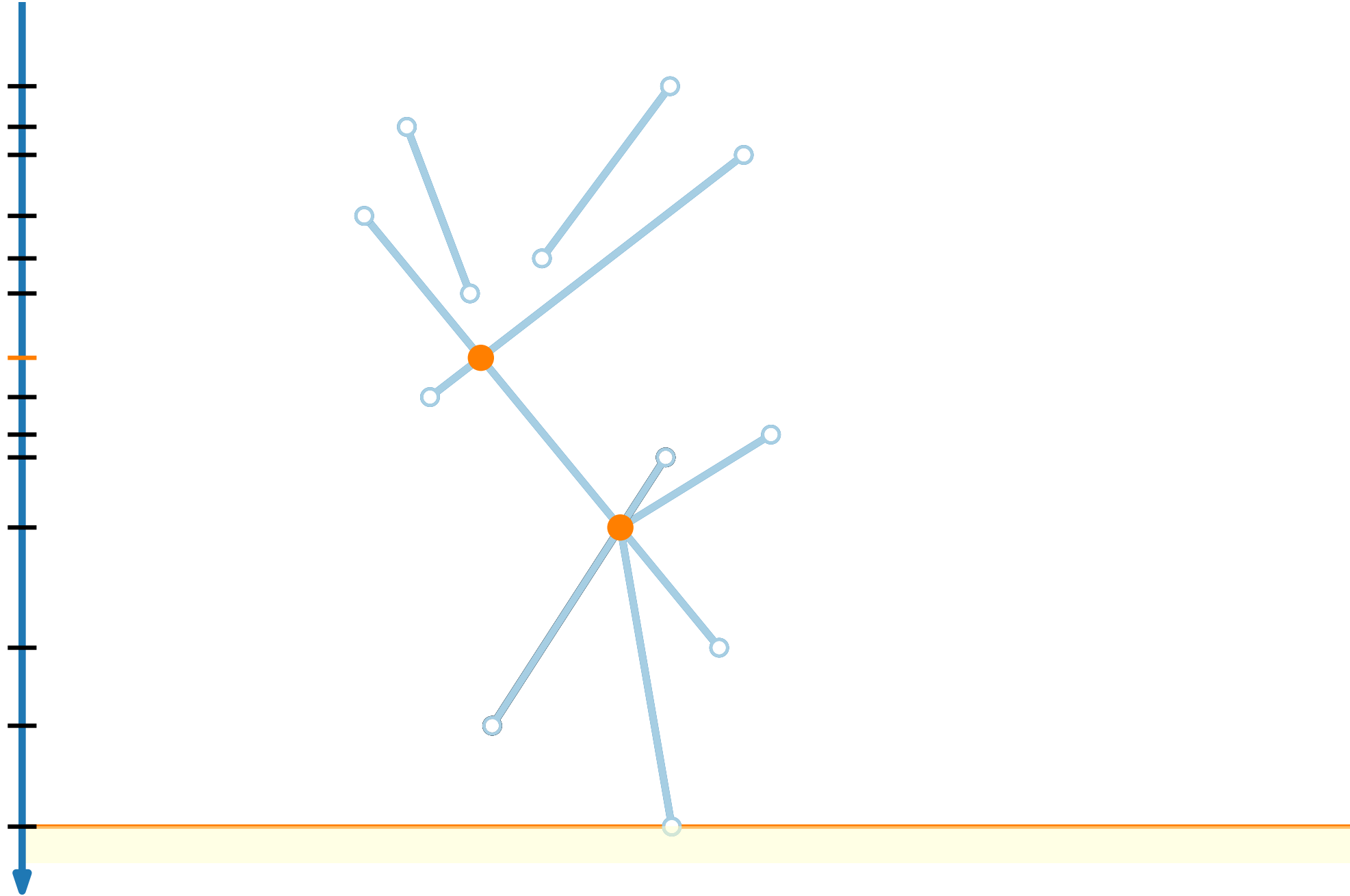
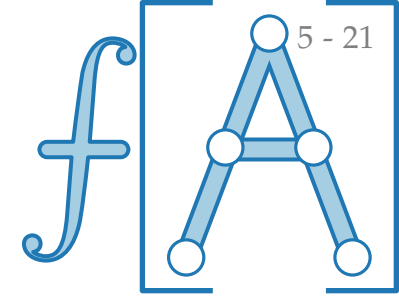


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

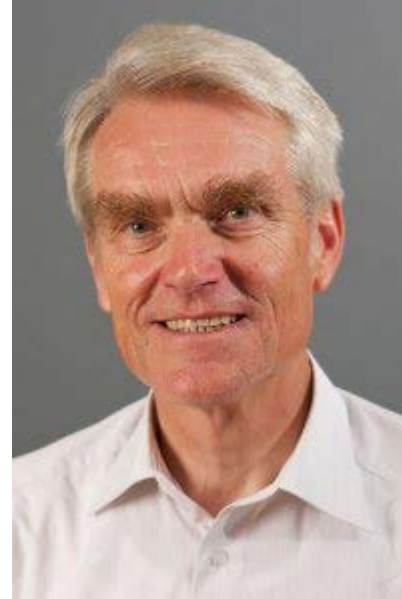
Sweep-Line



John Bentley
*1953, Long Beach, CA

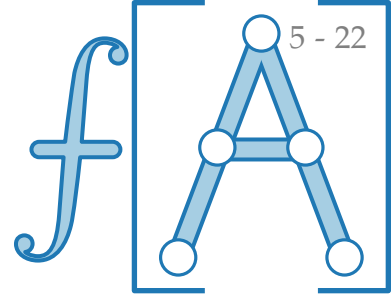


Thomas Ottmann
*1943, Magdeburg



Sweep-Line Algorithmus

Sweep-Line

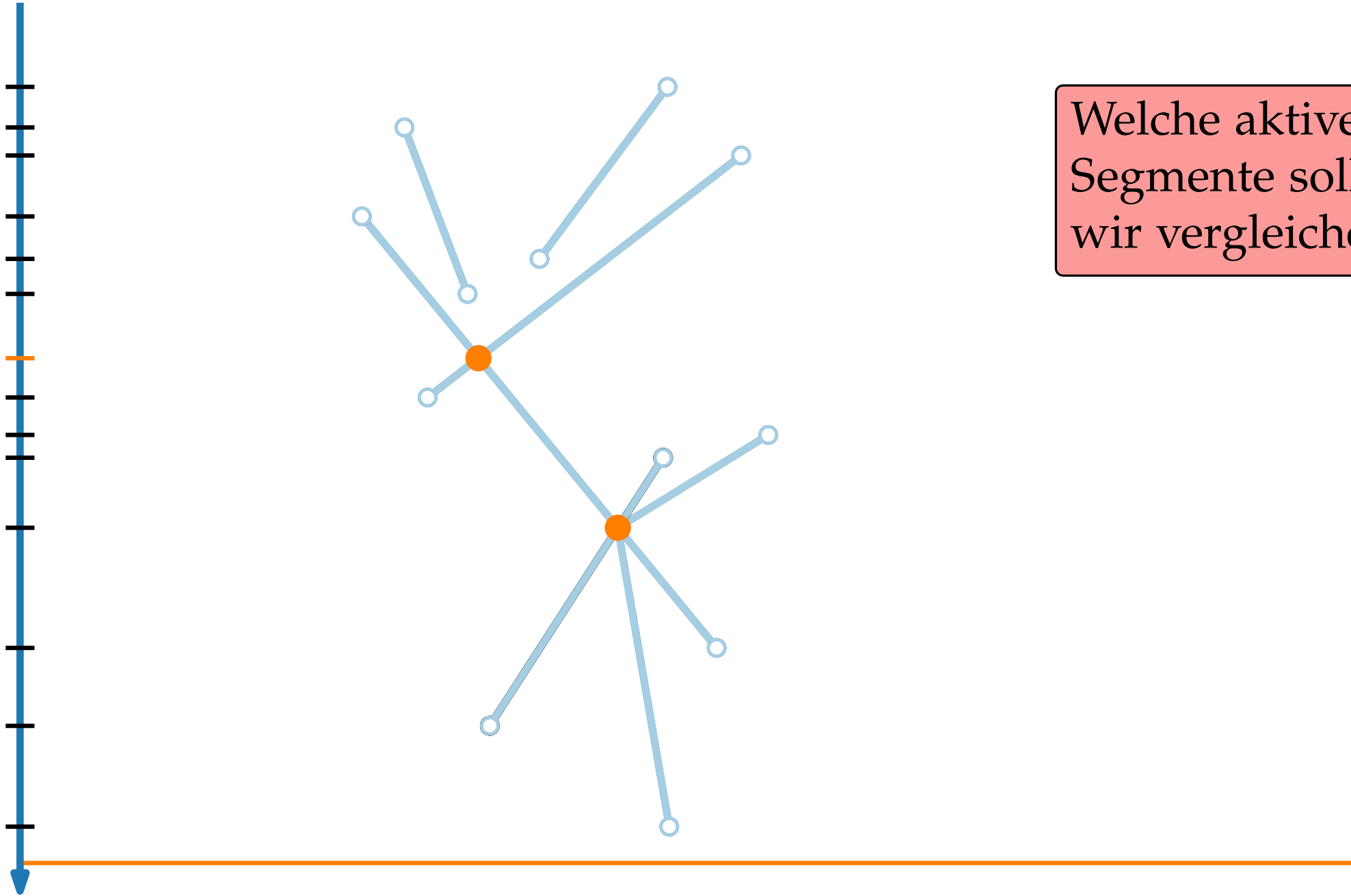
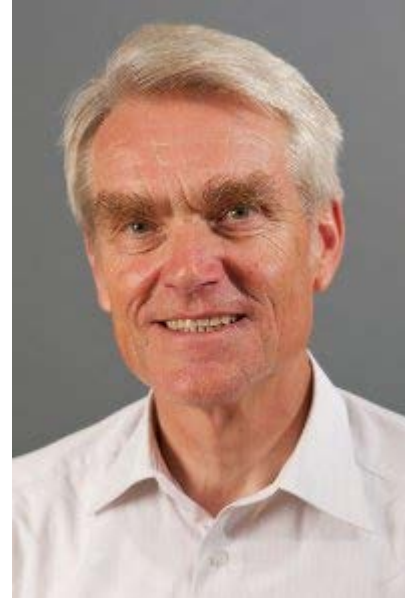


Welche aktiven
Segmente sollten
wir vergleichen?

John Bentley
*1953, Long Beach, CA

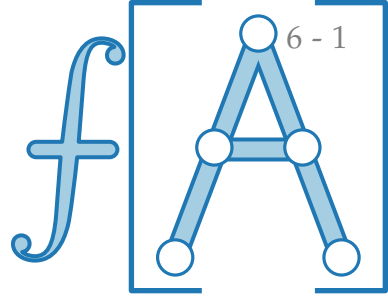


Thomas Ottmann
*1943, Magdeburg



Datenstrukturen

1) Eventschlange $\mathcal{Q}$



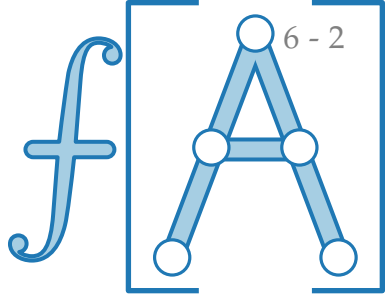
2) (Sweep-Line) Status $\mathcal{T}$

Datenstrukturen

1) Eventschlange $\mathcal{Q}$

$$p \prec q \quad \Leftrightarrow_{\text{def.}}$$

2) (Sweep-Line) Status $\mathcal{T}$

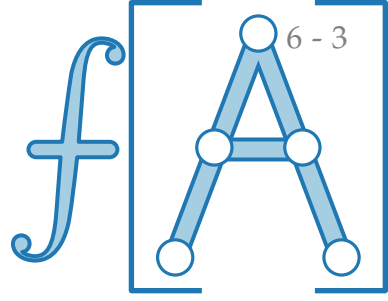


Datenstrukturen

1) Eventschlange $\mathcal{Q}$

$$p \prec q \quad \Leftrightarrow_{\text{def.}} \quad y_p > y_q$$

2) (Sweep-Line) Status $\mathcal{T}$



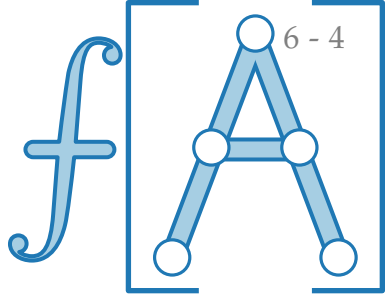
Datenstrukturen

1) Eventschlange $\mathcal{Q}$

$$p \prec q \quad \Leftrightarrow_{\text{def.}} \quad y_p > y_q$$



2) (Sweep-Line) Status $\mathcal{T}$



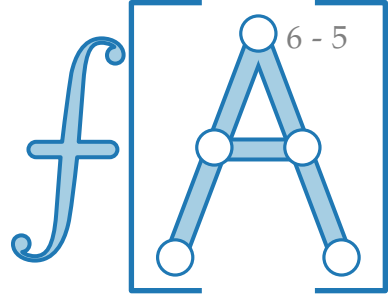
Datenstrukturen

1) Eventschlange $\mathcal{Q}$

$$p \prec q \quad \Leftrightarrow_{\text{def.}} \quad y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$



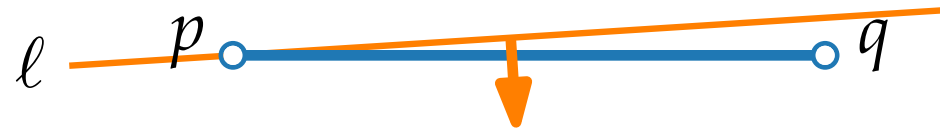
2) (Sweep-Line) Status $\mathcal{T}$



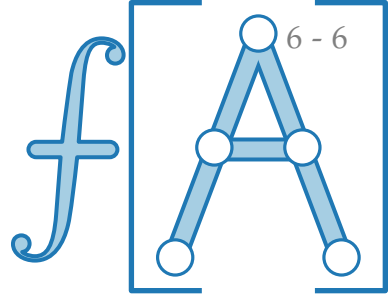
Datenstrukturen

1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$



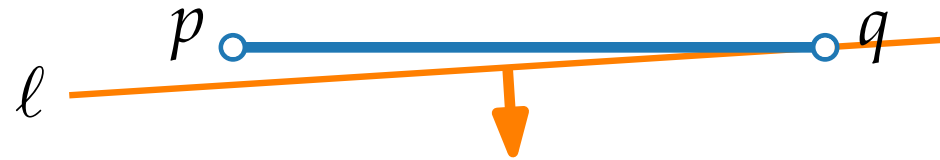
2) (Sweep-Line) Status $\mathcal{T}$



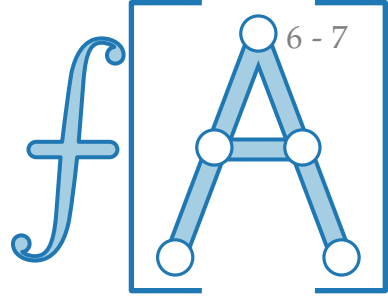
Datenstrukturen

1) Eventschlange $\mathcal{Q}$

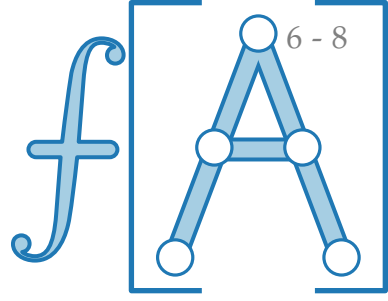
$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$



2) (Sweep-Line) Status $\mathcal{T}$

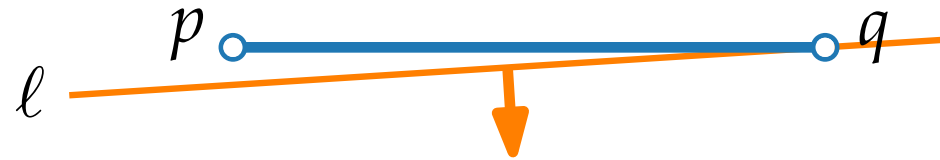


Datenstrukturen



1) Eventschlange $\mathcal{Q}$

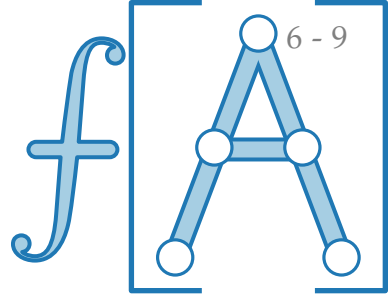
$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$



Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

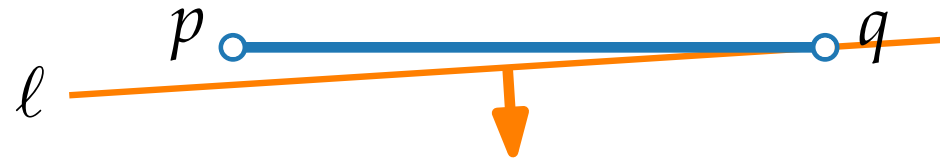
2) (Sweep-Line) Status $\mathcal{T}$

Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$

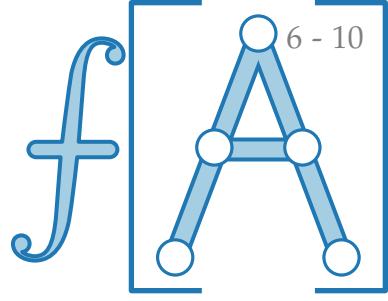


Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

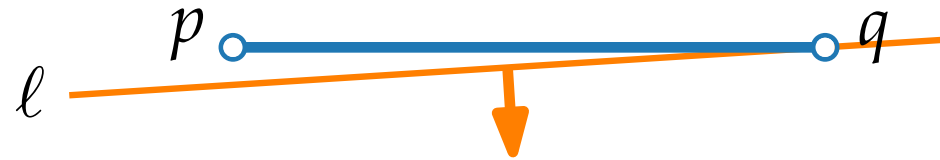
2) (Sweep-Line) Status $\mathcal{T}$

Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$



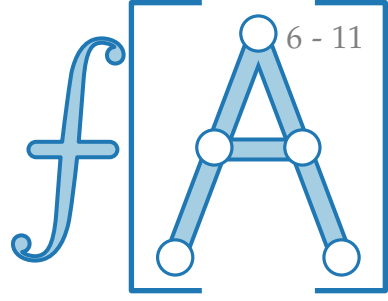
Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

MinHeap reicht auch

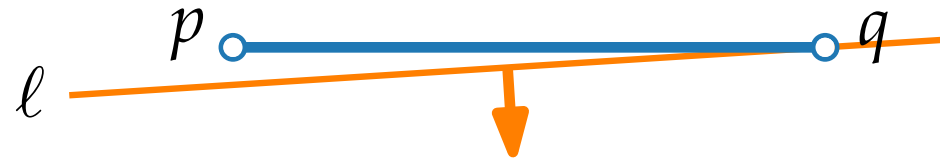
2) (Sweep-Line) Status $\mathcal{T}$

Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$

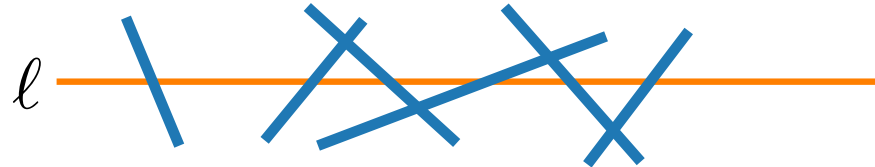


Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

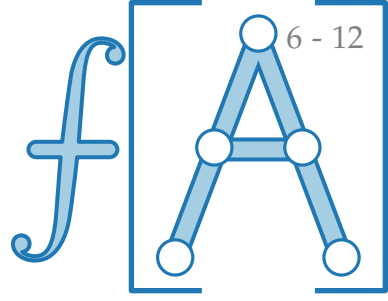
➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

MinHeap reicht auch

2) (Sweep-Line) Status $\mathcal{T}$

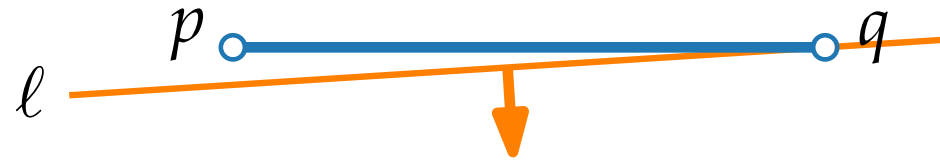


Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$

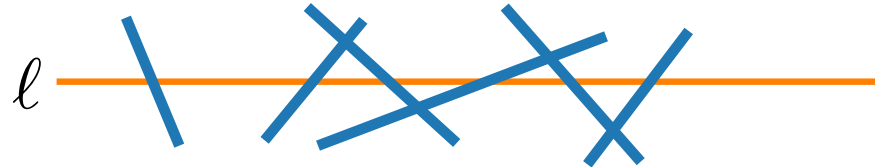


Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

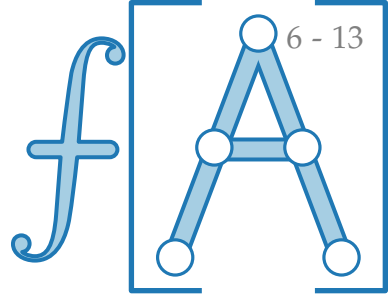
MinHeap reicht auch

2) (Sweep-Line) Status $\mathcal{T}$



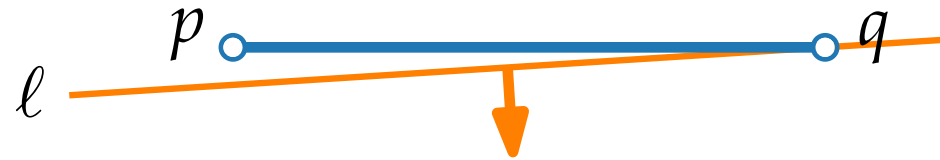
Speichere Segmente, die ℓ schneiden, von links nach rechts

Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$$p \prec q \iff_{\text{def.}} y_p > y_q \quad \text{oder} \quad (y_p = y_q \text{ und } x_p < x_q)$$

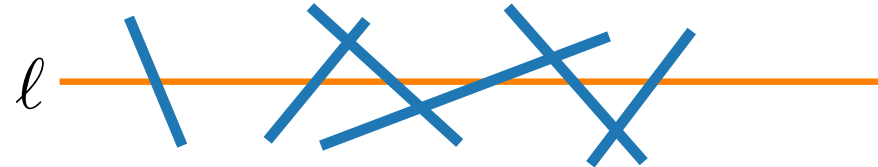


Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

MinHeap reicht auch

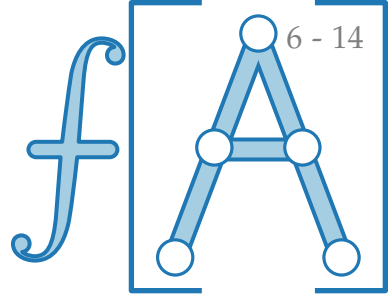
2) (Sweep-Line) Status $\mathcal{T}$



Speichere Segmente, die ℓ schneiden, von links nach rechts

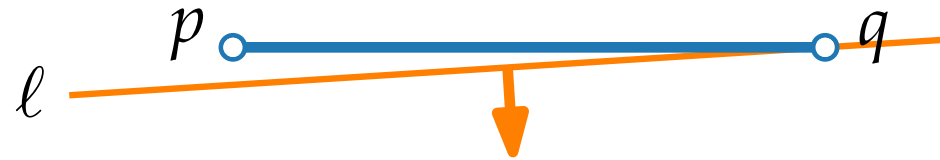
Wie?

Datenstrukturen



1) Eventschlange $\mathcal{Q}$

$p \prec q \iff_{\text{def.}} y_p > y_q$ oder $(y_p = y_q \text{ und } x_p < x_q)$

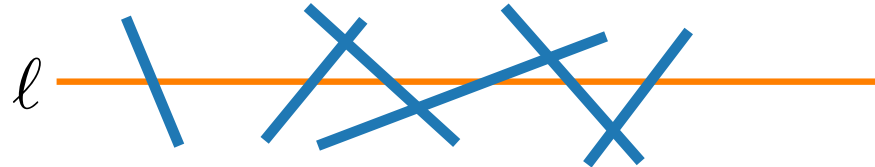


Speichere Eventpunkte in **balanciertem binären Suchbaum** bzgl. $\prec$

➔ NEXTEVENT(), INSERT() und DELETE() brauchen $\mathcal{O}(\log |\mathcal{Q}|)$ Zeit

MinHeap reicht auch

2) (Sweep-Line) Status $\mathcal{T}$



Speichere Segmente, die ℓ schneiden, von links nach rechts

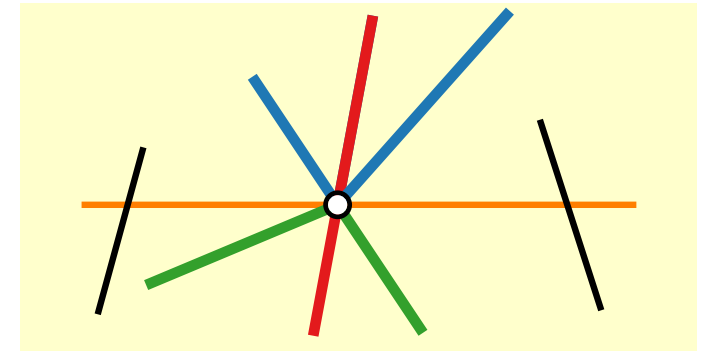
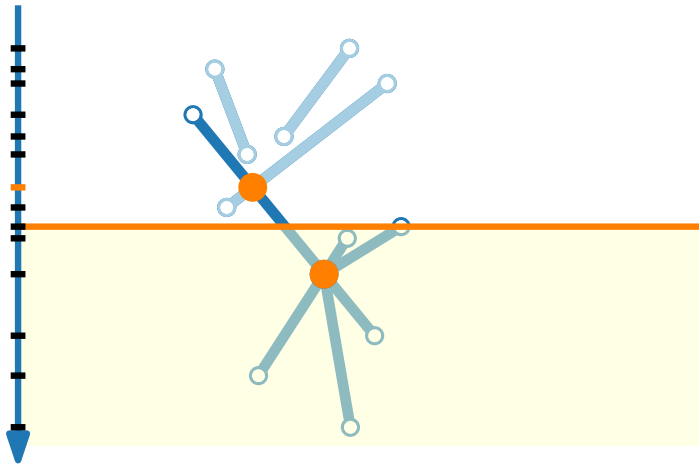
Wie? In einem **balancierten binären Suchbaum**!

Fortgeschrittene Algorithmen

13. Vorlesung

Algorithmische Geometrie II: Sweep-Line Algorithmus

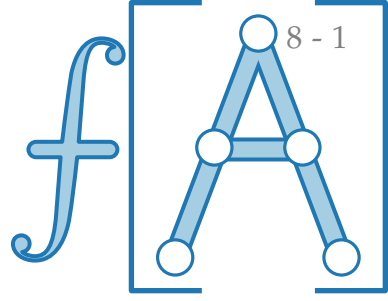
Teil III: Algorithmus



Schnittpunkte finden

FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

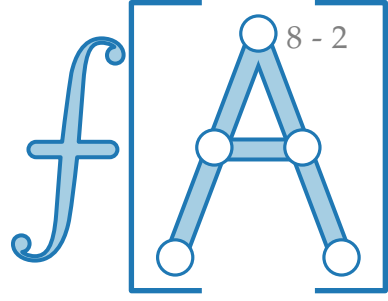


Schnittpunkte finden

FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten



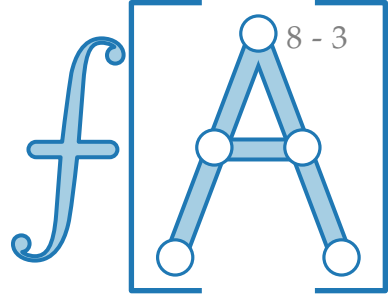
Schnittpunkte finden

FINDINTERSECTIONS(S)

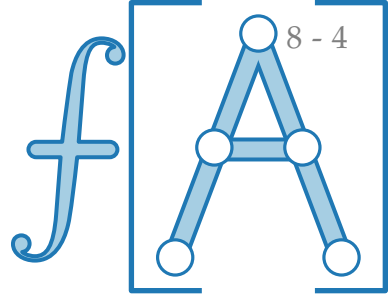
Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$



Schnittpunkte finden



FINDINTERSECTIONS(S)

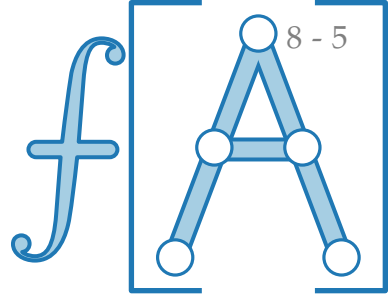
Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

$Q = \emptyset;$

Schnittpunkte finden



FINDINTERSECTIONS(S)

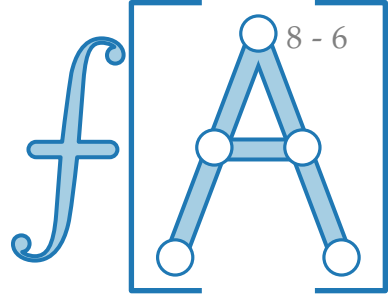
Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

$\mathcal{Q} = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

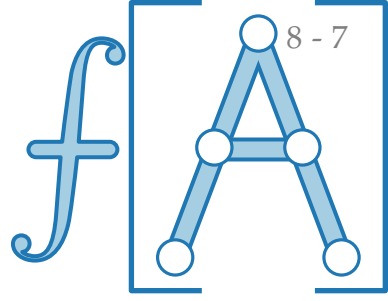
Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

```
 $\mathcal{Q} = \emptyset;$   $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $\mathcal{Q}$ 
```



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

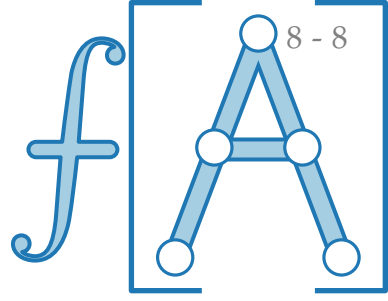
- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

```
 $\mathcal{Q} = \emptyset;$   $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $\mathcal{Q}$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do
```



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

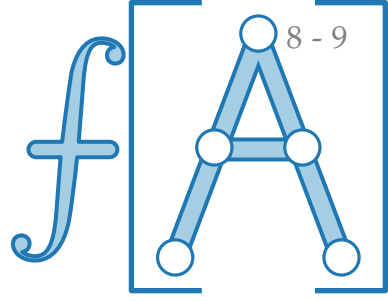
Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

```
 $Q = \emptyset$ ;  $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do  
        if  $p \notin Q$  then  $Q.\text{INSERT}(p)$ ;
```

Schnittpunkte finden



FINDINTERSECTIONS(S)

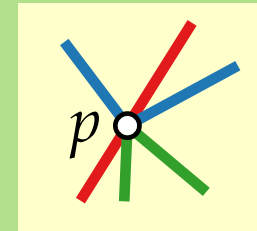
Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

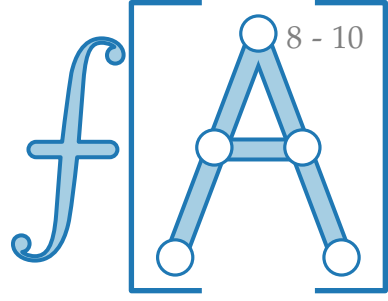
- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

```
 $Q = \emptyset;$   $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do  
        if  $p \notin Q$  then  $Q.\text{INSERT}(p);$ 
```



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

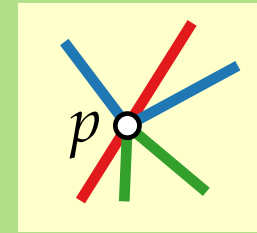
Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

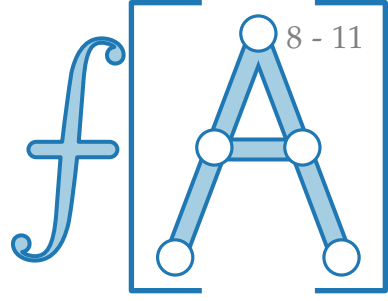
```
 $Q = \emptyset;$   $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do
```

```
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
```



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten

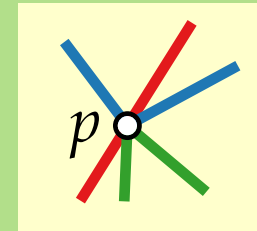
■ für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

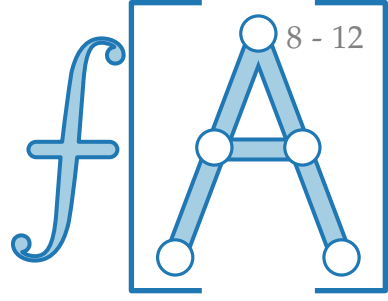
foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten

■ für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

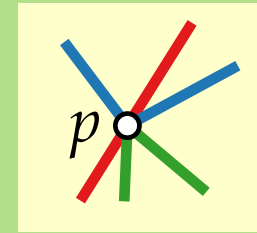
$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

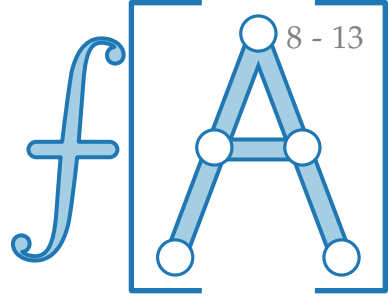
if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$



Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten

■ für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

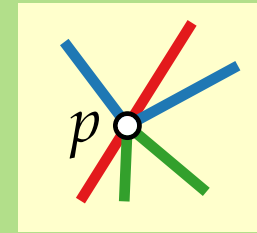
```
 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do
```

```
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
```

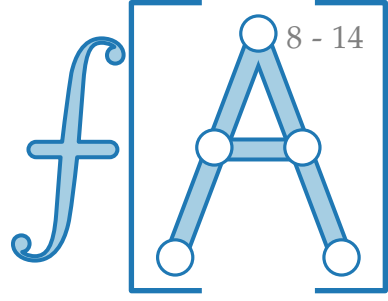
```
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
```

```
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 
```



```
while  $Q \neq \emptyset$  do
```

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

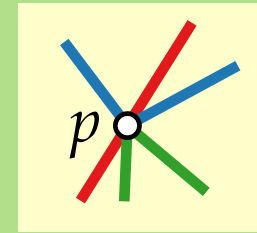
$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

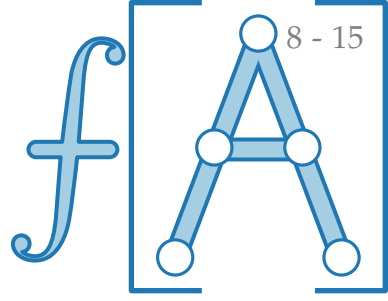
if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$



while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten

■ für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

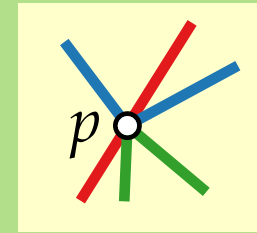
```
 $Q = \emptyset$ ;  $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
```

```
    foreach Endpunkt  $p$  von  $s$  do
```

```
        if  $p \notin Q$  then  $Q.\text{INSERT}(p)$ ;  $L(p) = U(p) = C(p) = \emptyset$ 
```

```
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
```

```
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 
```

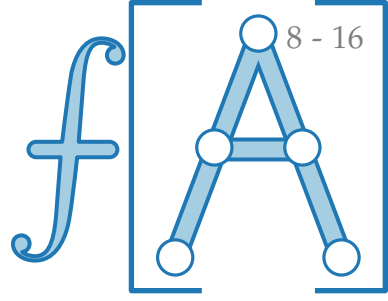


```
while  $Q \neq \emptyset$  do
```

```
     $p = Q.\text{NEXTEVENT}()$ 
```

```
     $Q.\text{DELETE}(p)$ 
```

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

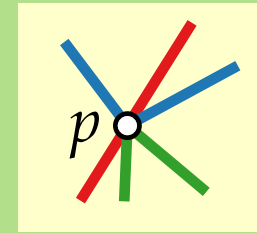
$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$



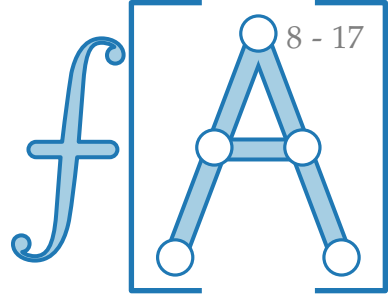
while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges.

- Menge I von k Schnittpunkten
- für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

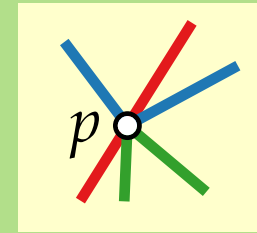
$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$



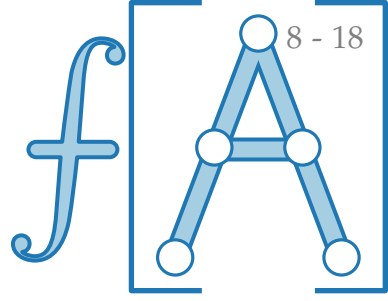
while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

 HANDLEEVENT(p)

Schnittpunkte finden



FINDINTERSECTIONS(S)

Geg. Menge S von n nicht-überlappenden Liniensegmenten

Ges. ■ Menge I von k Schnittpunkten

■ für jeden Punkt $p \in I$ alle Segmente $s \in S$ mit $p \in s$

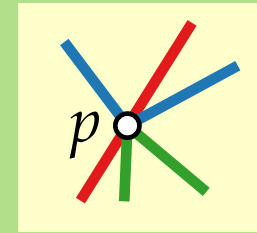
$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$



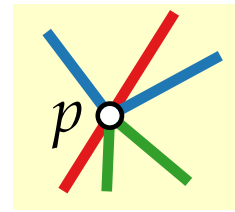
while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}$ Diese Subroutine macht die Hauptarbeit.

$Q.\text{DELETE}(p)$ Wie implementieren wir sie?

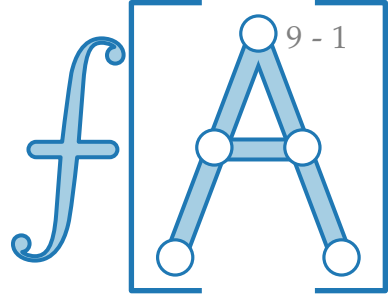
 HANDLEEVENT(p)

Events verarbeiten

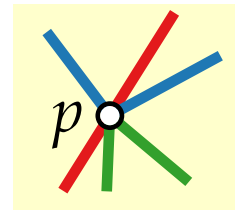


$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

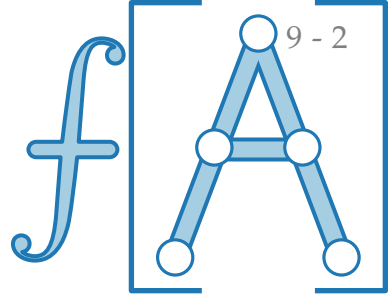


Events verarbeiten

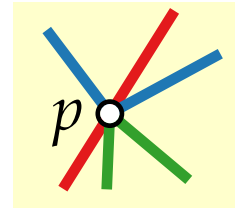


$C(p), L(p), U(p)$

```
HANDLEEVENT(Event  $p$ )  
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then  
  |
```



Events verarbeiten

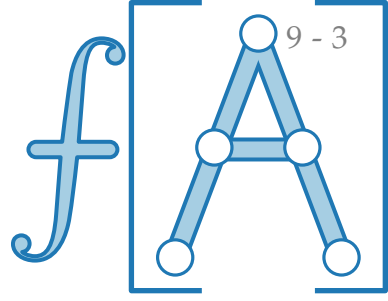


$C(p), L(p), U(p)$

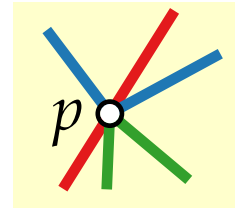
HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus



Events verarbeiten

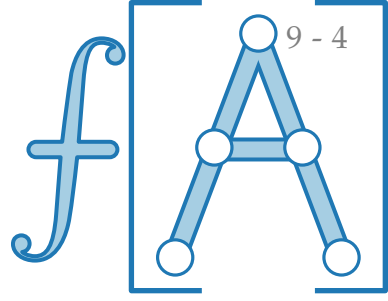


$C(p), L(p), U(p)$

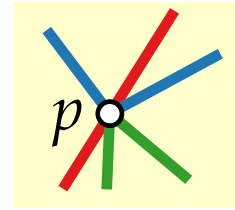
HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!



Events verarbeiten



$C(p), L(p), U(p)$

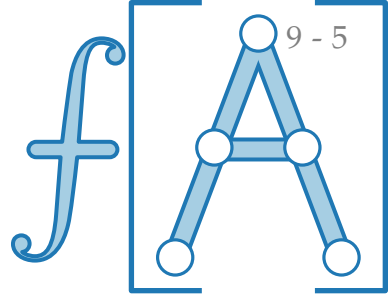
HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

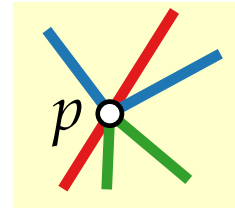
 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus

 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

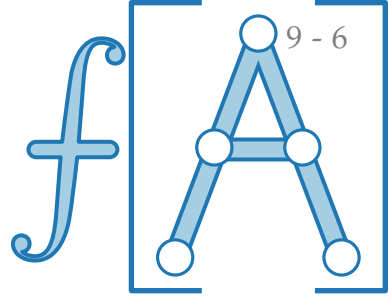
 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein



Events verarbeiten



$C(p), L(p), U(p)$



HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 |_gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

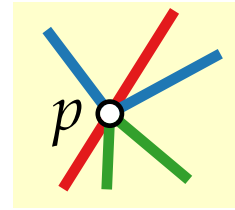
if $U(p) \cup C(p) = \emptyset$ **then**

|_

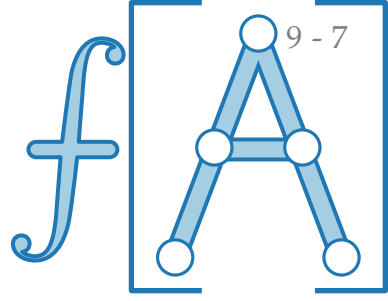
else

|_

Events verarbeiten



$C(p), L(p), U(p)$



HANDLEEVENT(Event p)

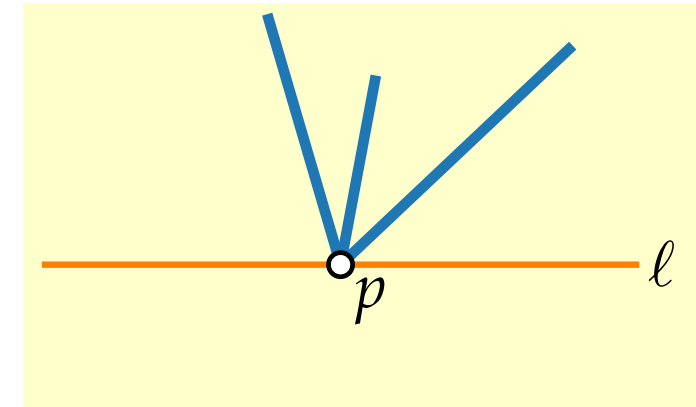
if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

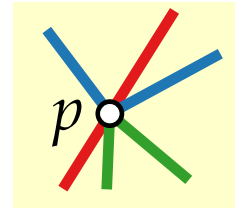
 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

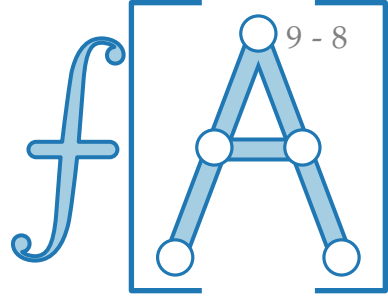
else



Events verarbeiten



$C(p), L(p), U(p)$



HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

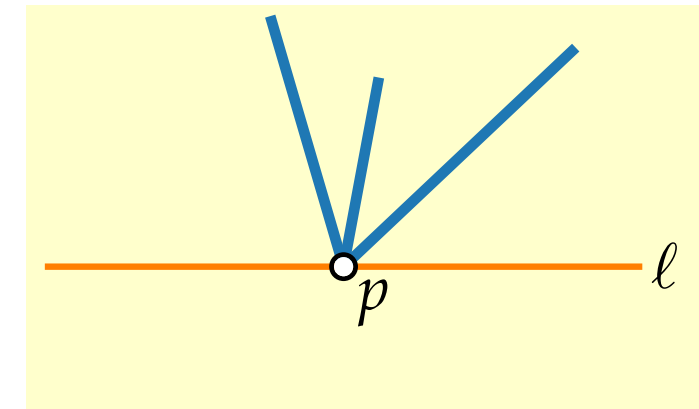
 |_gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

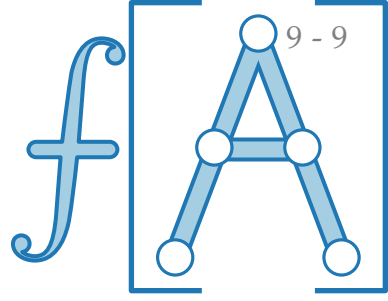
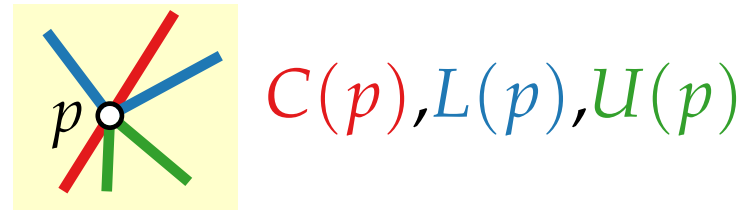
if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

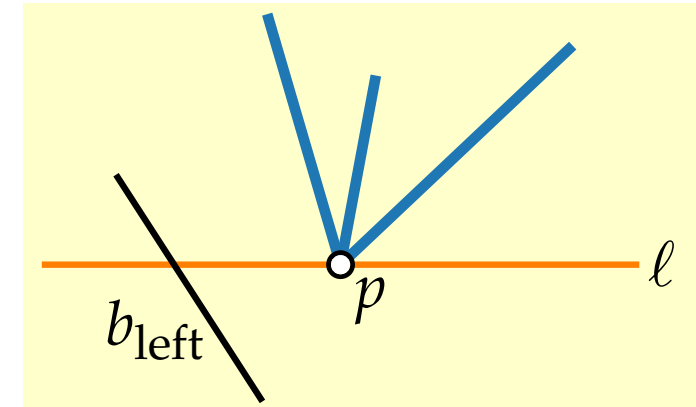
else



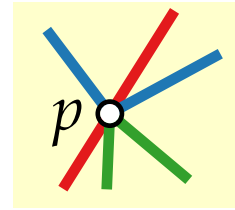
Events verarbeiten



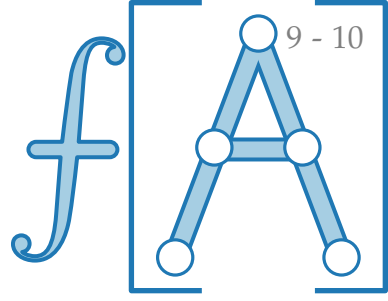
```
HANDLEEVENT(Event  $p$ )  
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then  
  _gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus  
  lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !  
  füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein  
  if  $U(p) \cup C(p) = \emptyset$  then  
     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$   
  else
```



Events verarbeiten



$C(p), L(p), U(p)$



HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

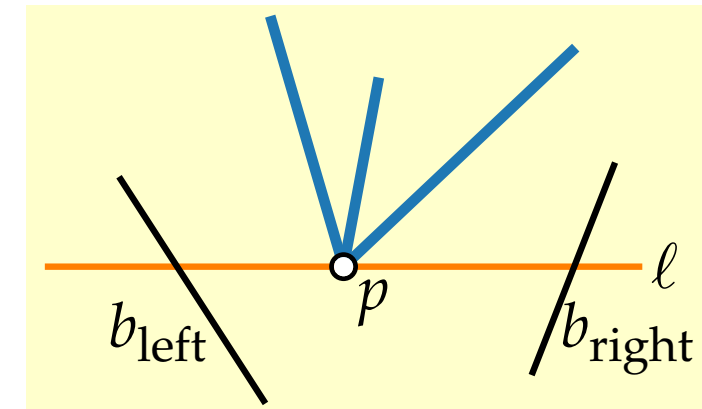
 |_gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

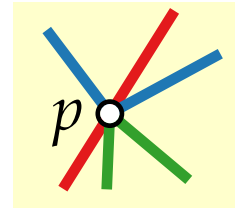
if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

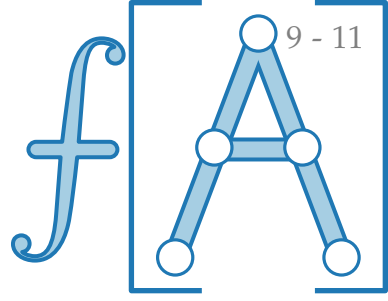
else



Events verarbeiten



$C(p), L(p), U(p)$



HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

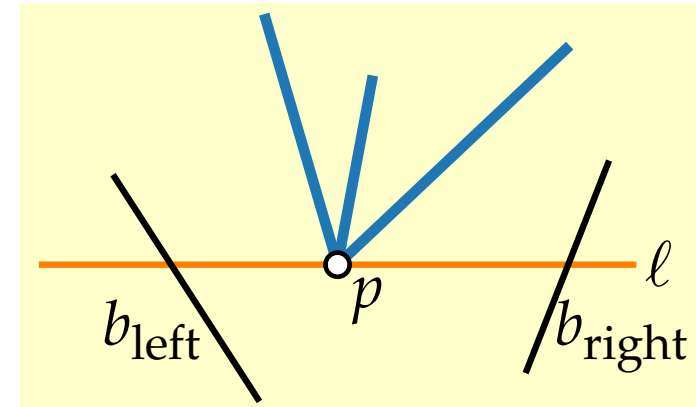
 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

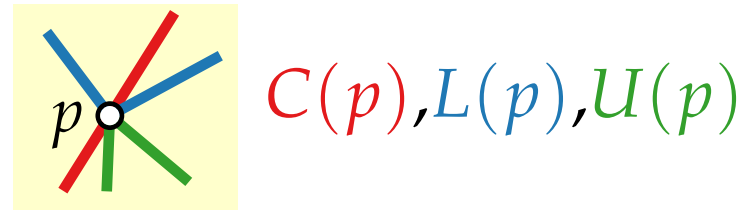
$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else



Events verarbeiten



FINDNEWEVENT(s, s', p)

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

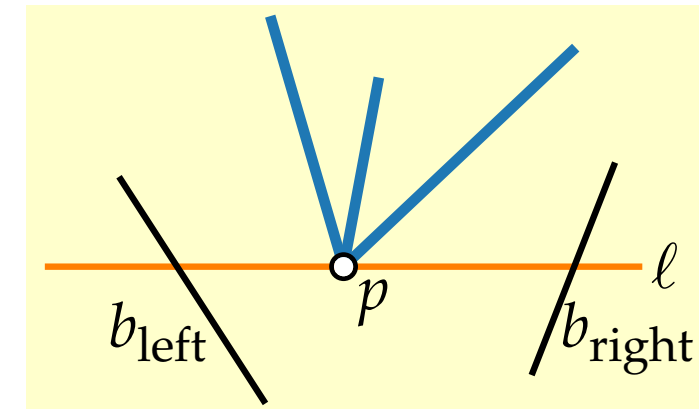
 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

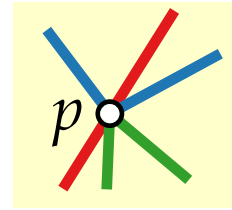
$b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else



Events verarbeiten



$C(p), L(p), U(p)$

```
FINDNEWEVENT( $s, s', p$ )  
if  $s \cap s' = \emptyset$  then
```

```
HANDLEEVENT(Event  $p$ )
```

```
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
```

```
  _gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus  
  lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
```

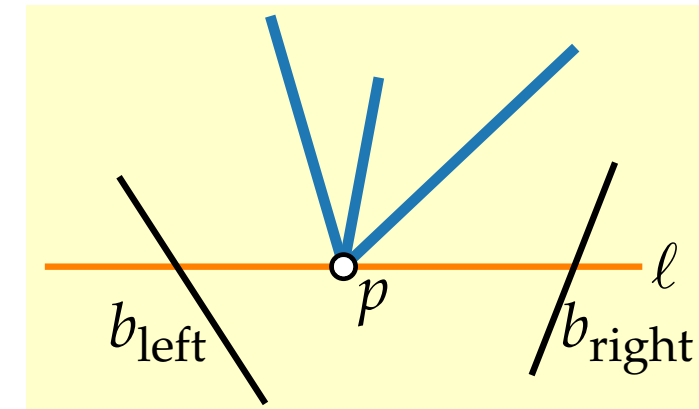
```
  füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
```

```
  if  $U(p) \cup C(p) = \emptyset$  then
```

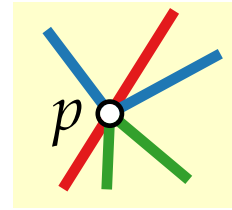
```
     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
```

```
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
```

```
  else
```



Events verarbeiten



$C(p), L(p), U(p)$

```
FINDNEWEVENT( $s, s', p$ )  
if  $s \cap s' = \emptyset$  then return
```

```
HANDLEEVENT(Event  $p$ )
```

```
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
```

```
  _gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus  
  lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
```

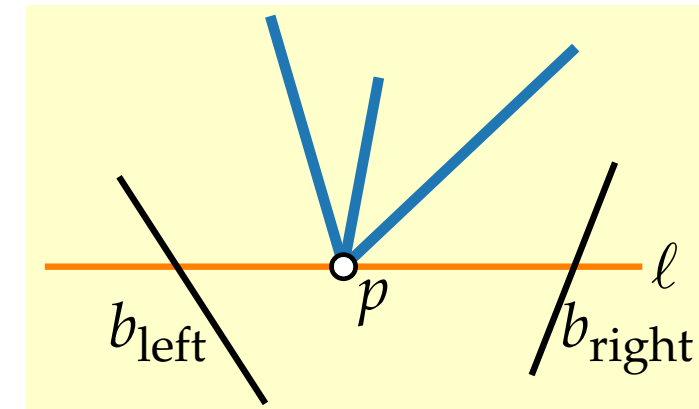
```
  füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
```

```
  if  $U(p) \cup C(p) = \emptyset$  then
```

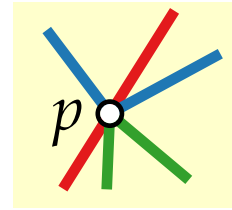
```
     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
```

```
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
```

```
  else
```



Events verarbeiten



$C(p), L(p), U(p)$

```
FINDNEWEVENT( $s, s', p$ )  
if  $s \cap s' = \emptyset$  then return  
 $\{x\} = s \cap s'$ 
```

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ then

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

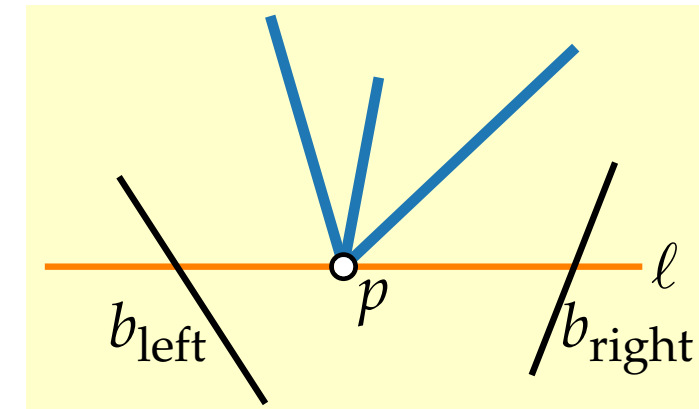
 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ then

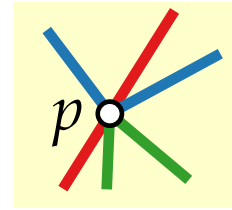
$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else



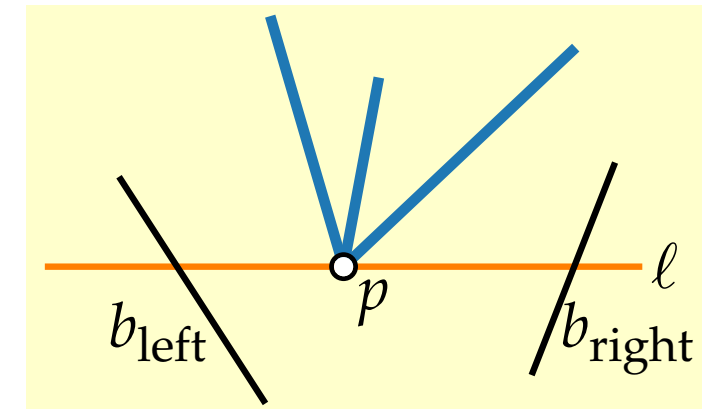
Events verarbeiten



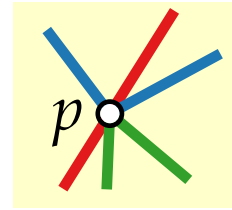
$C(p), L(p), U(p)$

```
FINDNEWEVENT( $s, s', p$ )  
if  $s \cap s' = \emptyset$  then return  
 $\{x\} = s \cap s'$   
if  $x$  unter  $\ell$  oder auf  $\ell$  rechts von  $p$  then
```

```
HANDLEEVENT(Event  $p$ )  
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then  
  _gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus  
  lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !  
  füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein  
  if  $U(p) \cup C(p) = \emptyset$  then  
     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$   
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  
  else
```



Events verarbeiten



$C(p), L(p), U(p)$

```
FINDNEWEVENT( $s, s', p$ )  
if  $s \cap s' = \emptyset$  then return  
 $\{x\} = s \cap s'$   
if  $x$  unter  $\ell$  oder auf  $\ell$  rechts von  $p$  then  
    if  $x \notin Q$  then  $Q.add(x)$ 
```

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ then

_gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

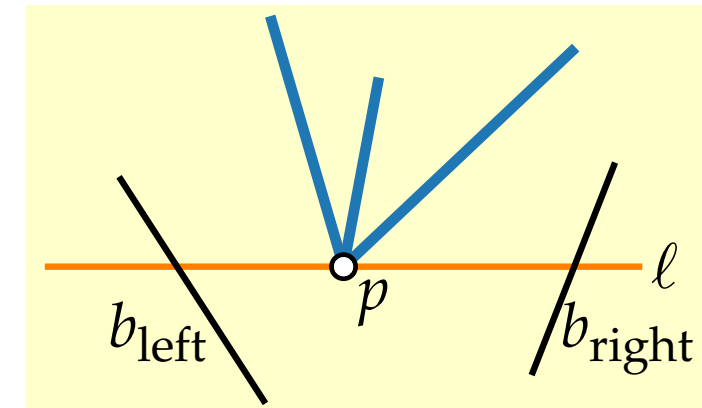
füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ then

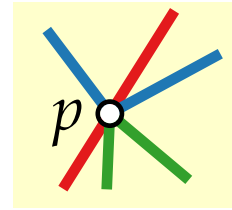
$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

FINDNEWEVENT(s, s', p)

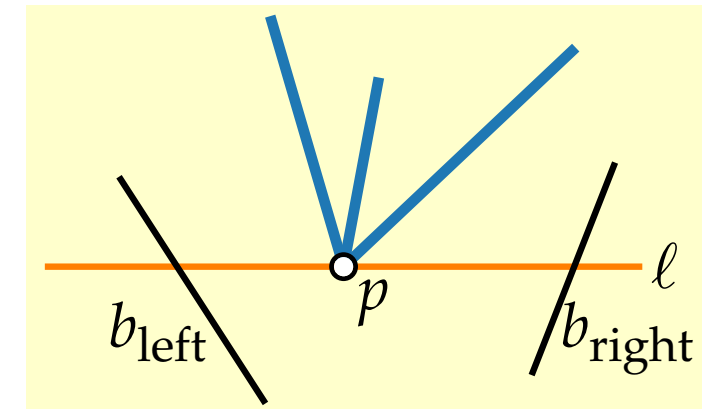
if $s \cap s' = \emptyset$ **then return**

$\{x\} = s \cap s'$

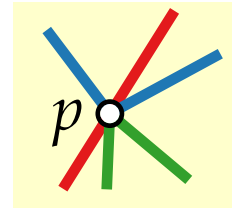
if x unter ℓ oder auf ℓ rechts von p **then**

if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

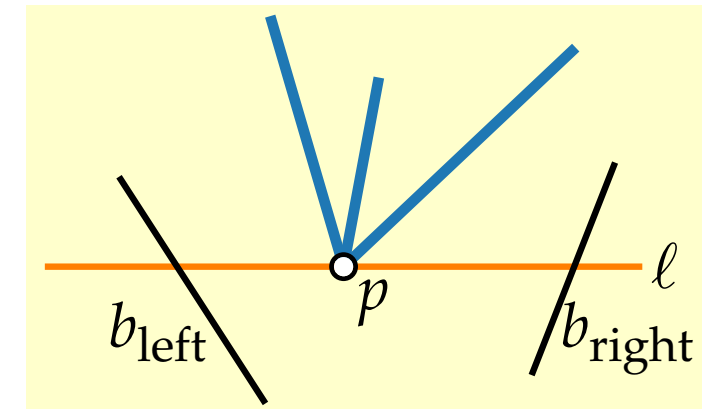
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

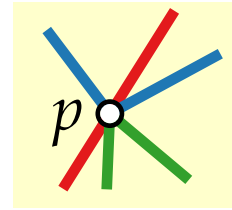
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

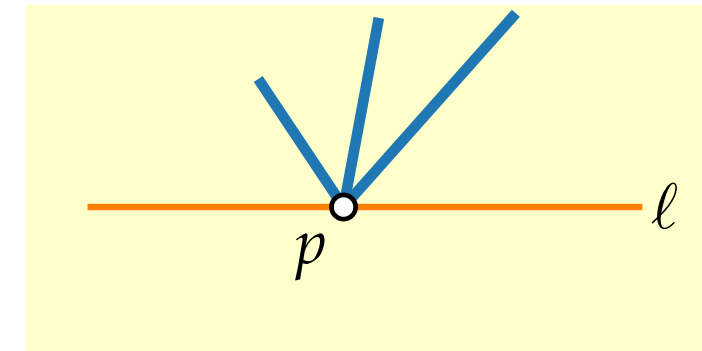
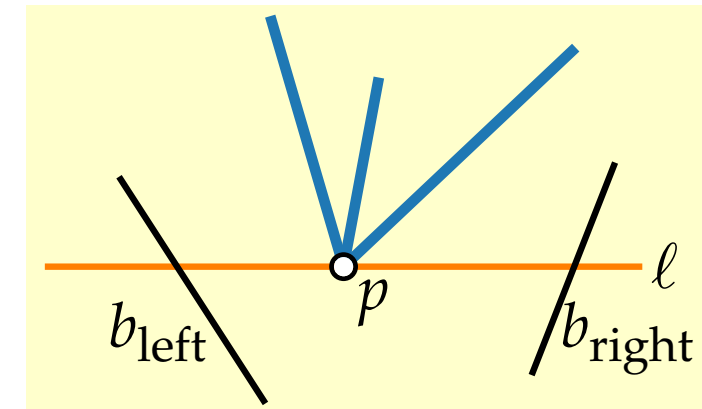
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

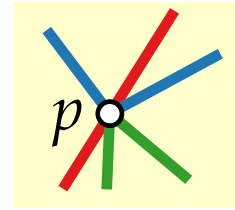
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

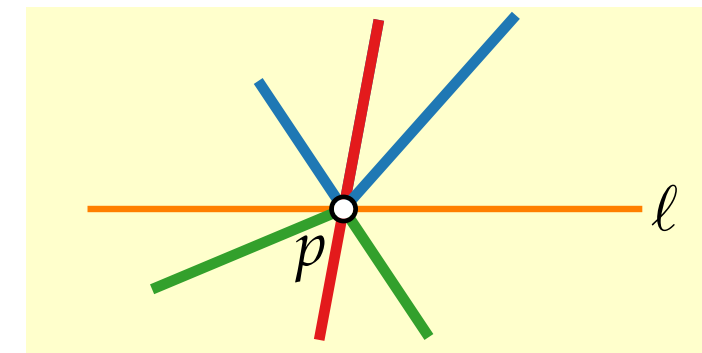
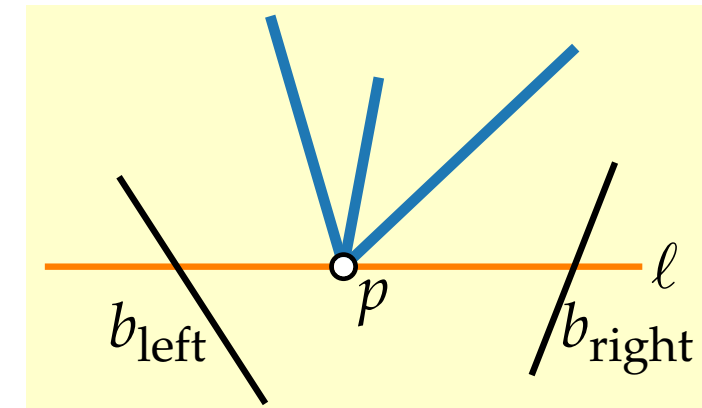
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

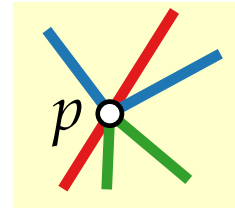
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

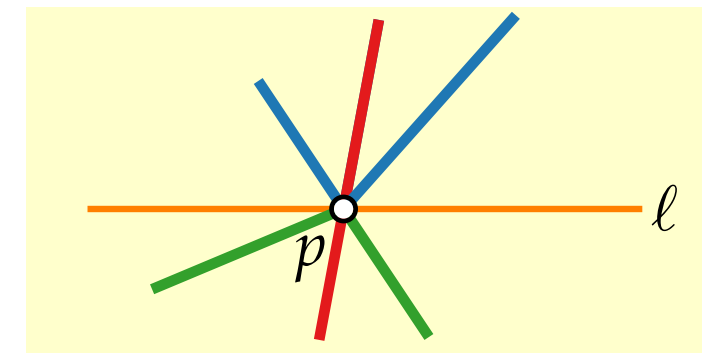
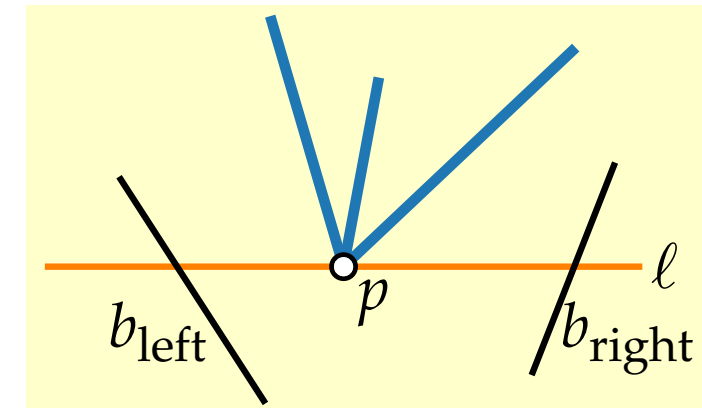
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

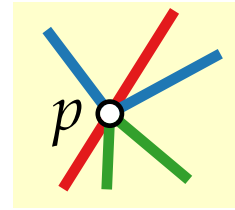
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 |_gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$
 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

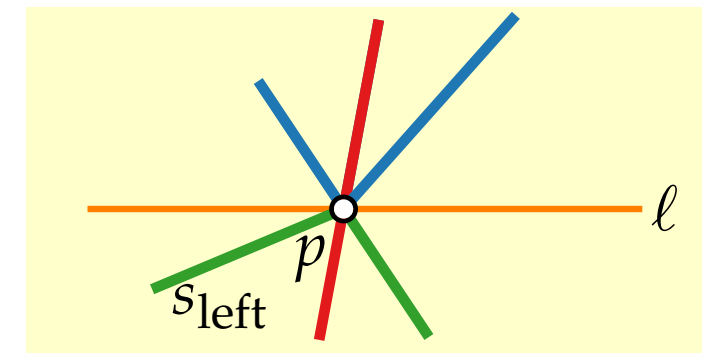
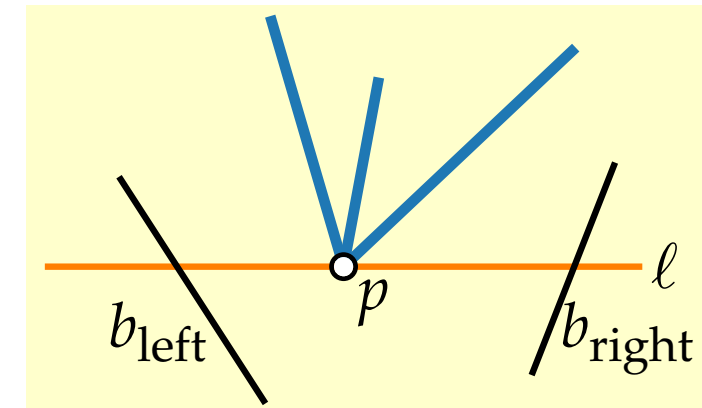
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

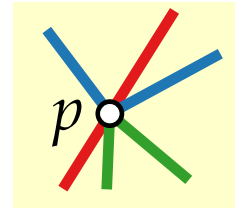
if $x \notin Q$ **then** $Q.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

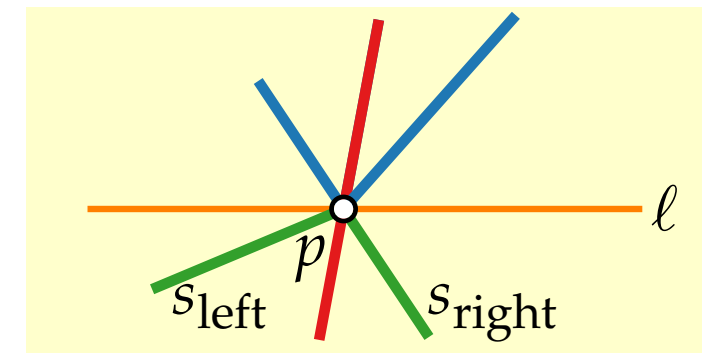
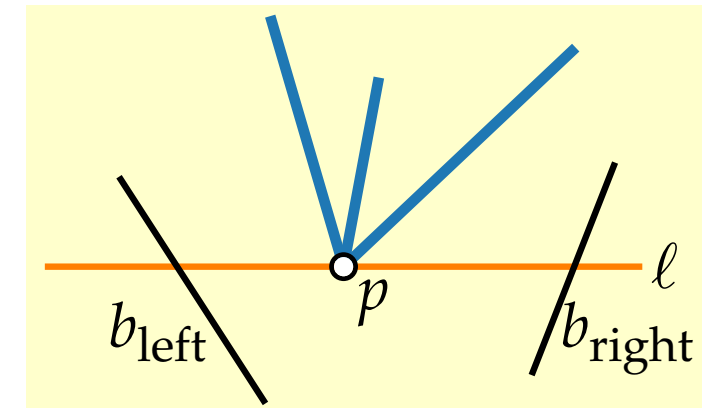
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

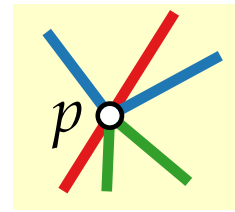
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

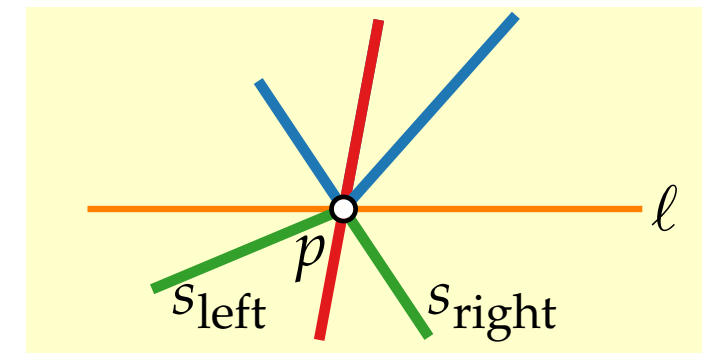
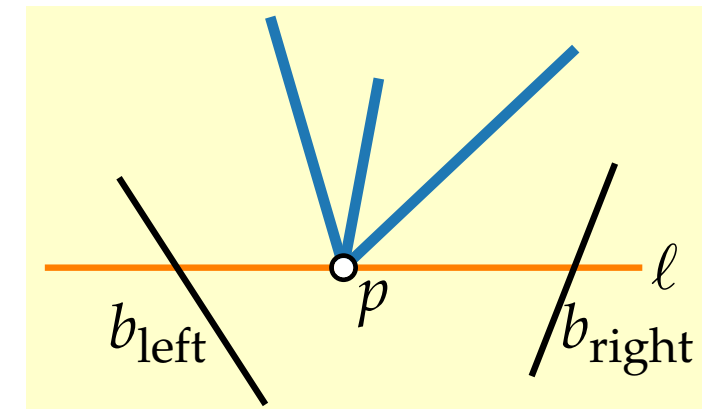
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

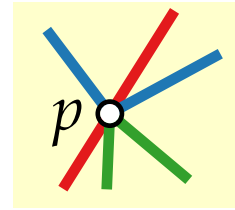
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

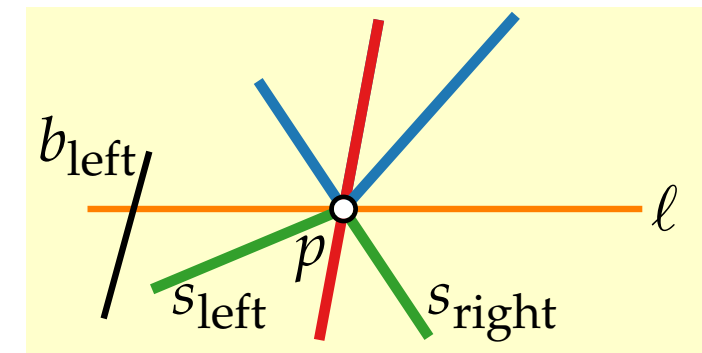
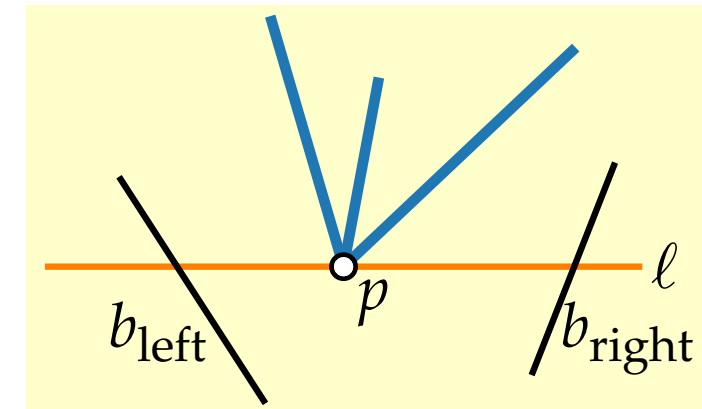
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

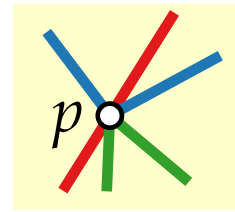
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

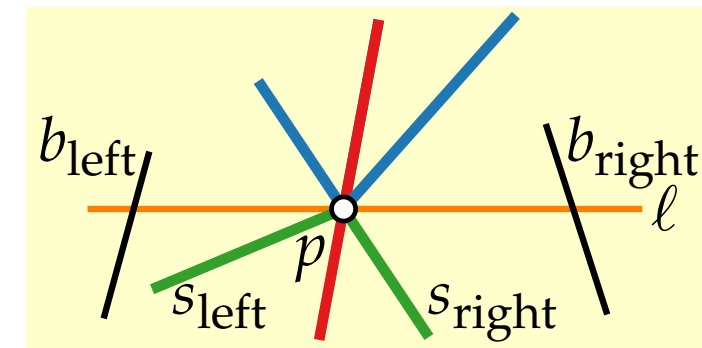
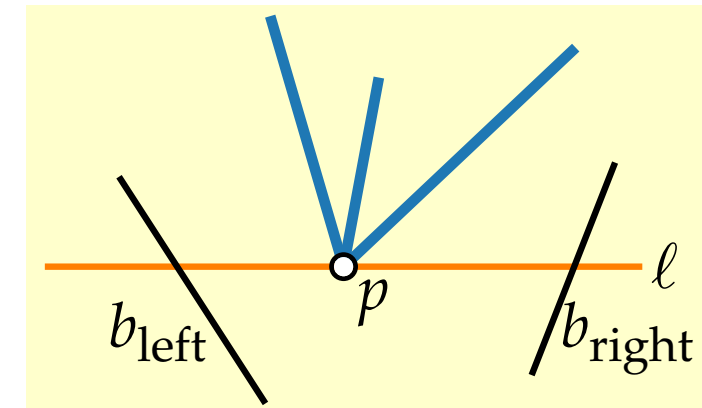
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

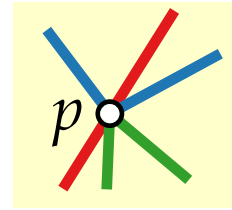
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

 FINDNEWEVENT($b_{\text{left}}, s_{\text{left}}, p$)

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

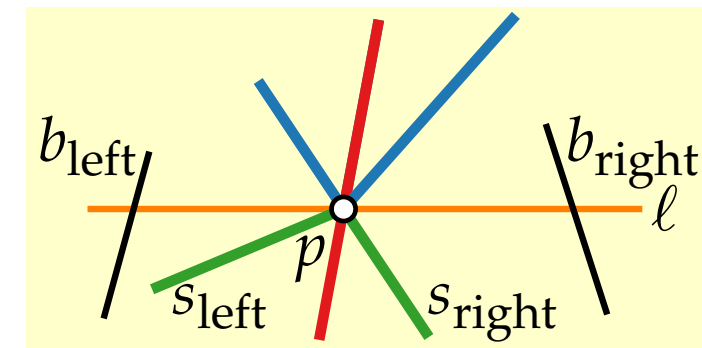
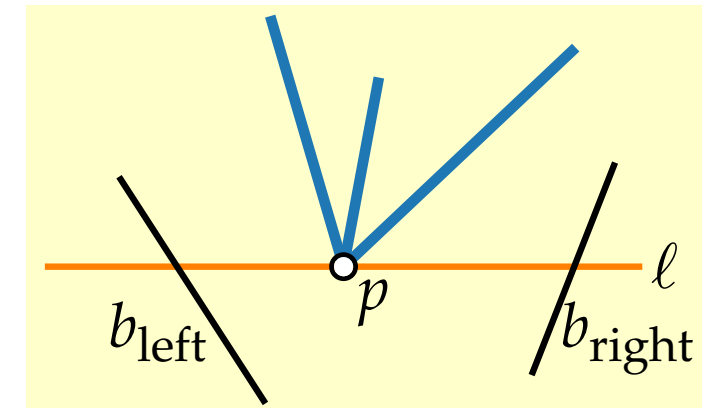
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

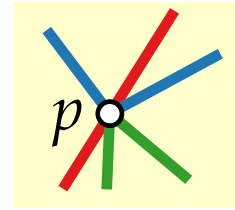
if $x \notin \mathcal{Q}$ **then** $\mathcal{Q}.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$



Events verarbeiten



$C(p), L(p), U(p)$

HANDLEEVENT(Event p)

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$
 FINDNEWEVENT($b_{\text{left}}, b_{\text{right}}, p$)

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$
 b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$
 b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$
 FINDNEWEVENT($b_{\text{left}}, s_{\text{left}}, p$)
 FINDNEWEVENT($b_{\text{right}}, s_{\text{right}}, p$)

FINDNEWEVENT(s, s', p)

if $s \cap s' = \emptyset$ **then return**

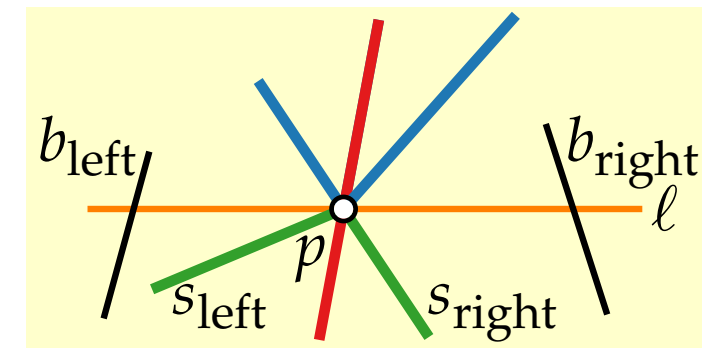
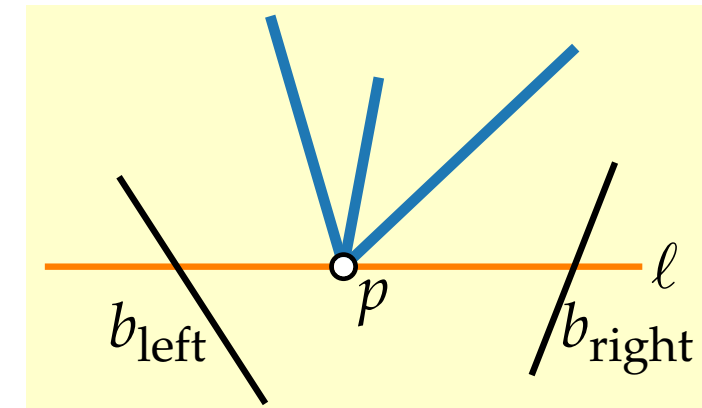
$\{x\} = s \cap s'$

if x unter ℓ oder auf ℓ rechts von p **then**

if $x \notin Q$ **then** $Q.\text{add}(x)$

if $x \in \text{rel-int}(s)$ **then** $C(x) = C(x) \cup \{s\}$

if $x \in \text{rel-int}(s')$ **then** $C(x) = C(x) \cup \{s'\}$

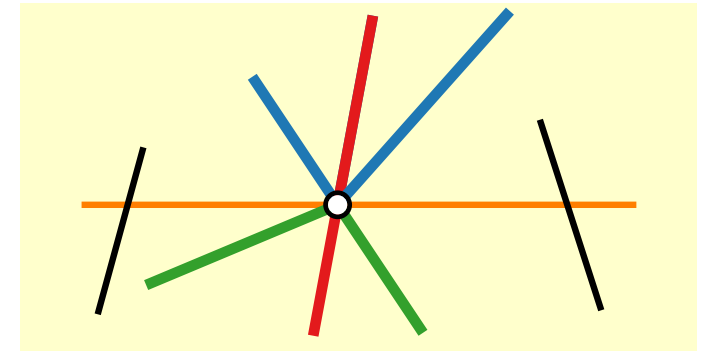
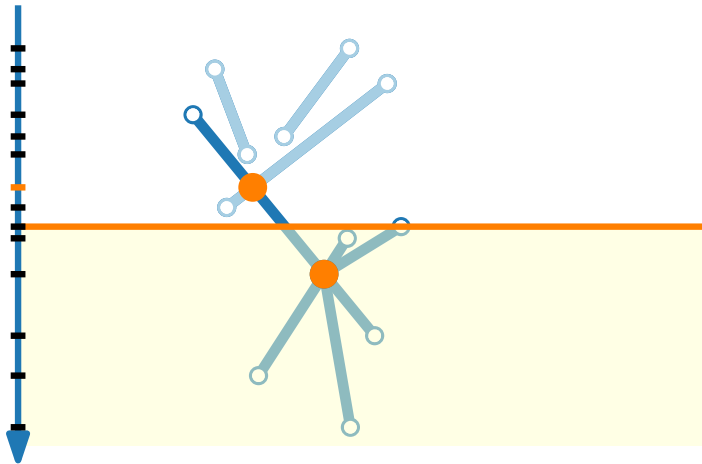


Fortgeschrittene Algorithmen

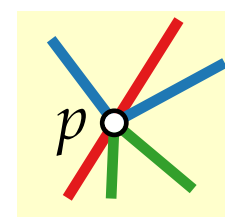
13. Vorlesung

Algorithmische Geometrie II: Sweep-Line Algorithmus

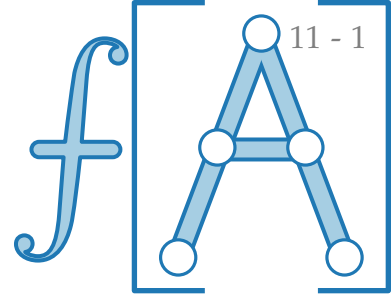
Teil IV: Korrektheit



Korrektheit

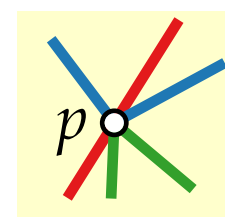


$C(p), L(p), U(p)$

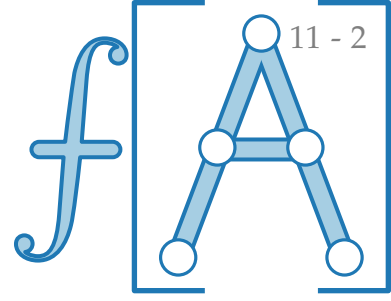


Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Korrektheit



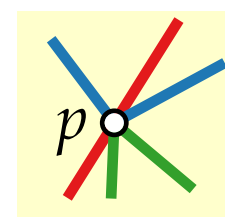
$C(p), L(p), U(p)$



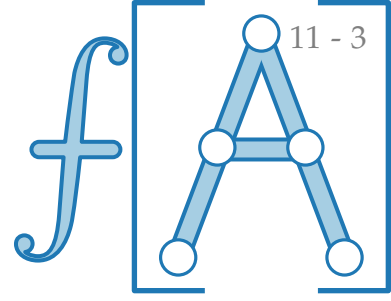
Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt.

Korrektheit



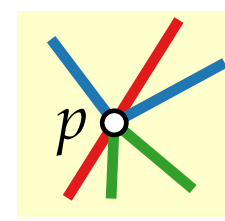
$C(p), L(p), U(p)$



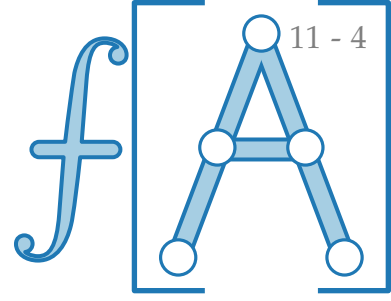
Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt. Angenommen:

Korrektheit



$C(p), L(p), U(p)$

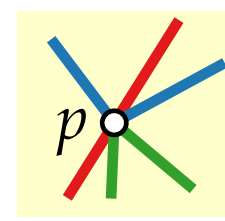


Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

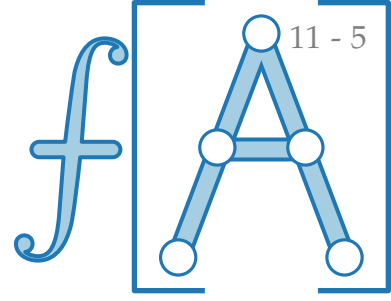
Beweis. Sei p ein Schnittpunkt. Angenommen:

- Jeder Schnittpunkt $q \prec p$ wurde gefunden

Korrektheit



$C(p), L(p), U(p)$

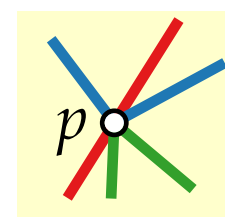


Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

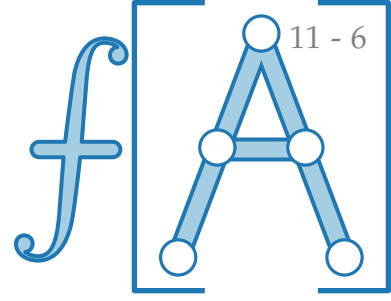
Beweis. Sei p ein Schnittpunkt. Angenommen:

- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

Korrektheit



$C(p), L(p), U(p)$

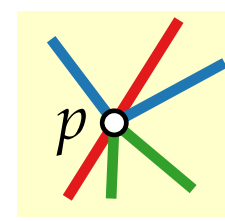


Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

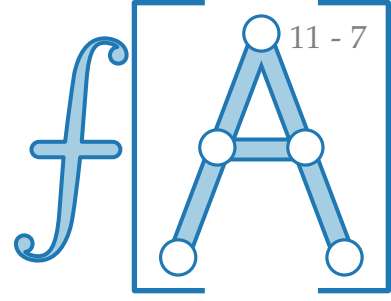
Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

Korrektheit



$C(p), L(p), U(p)$



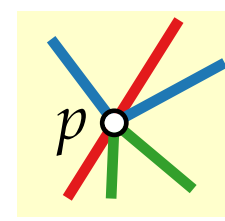
Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

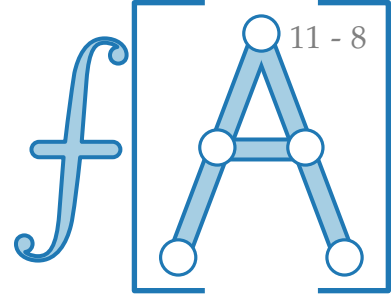
- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

Fall 1. p ist kein innerer Punkt eines Segments.

Korrektheit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

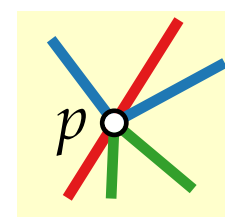
Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

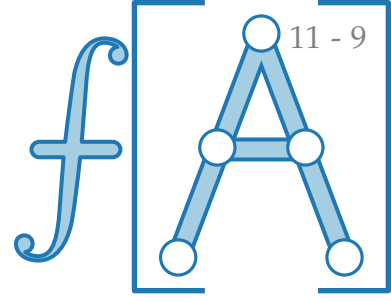
Fall 1. p ist kein innerer Punkt eines Segments.

➔ p wurde zu Beginn in $\mathcal{Q}$ eingefügt.

Korrektheit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

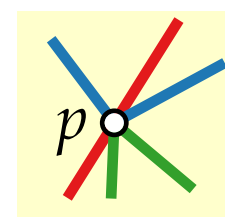
- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

Fall 1. p ist kein innerer Punkt eines Segments.

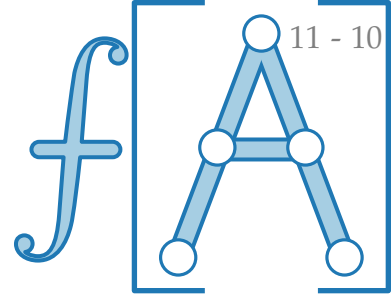
➔ p wurde zu Beginn in $\mathcal{Q}$ eingefügt.

Segmente in $U(p)$ und $L(p)$ sind von Beginn an mit p gespeichert.

Korrektheit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

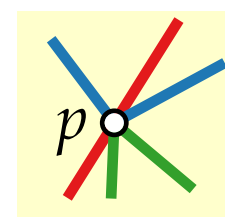
Fall 1. p ist kein innerer Punkt eines Segments.

➔ p wurde zu Beginn in $\mathcal{Q}$ eingefügt.

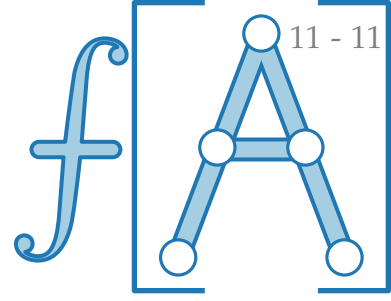
Segmente in $U(p)$ und $L(p)$ sind von Beginn an mit p gespeichert.

Wenn p verarbeitet wird, geben wir alle Segmente in $U(p) \cup L(p)$ aus.

Korrektheit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS(S) findet alle Schnittpunkte und die Segmente, die sie enthalten.

Beweis. Sei p ein Schnittpunkt. Angenommen (induktiv):

- Jeder Schnittpunkt $q \prec p$ wurde gefunden
- $\mathcal{T}$ enthält alle Segmente, die ℓ schneiden, in Reihenfolge von links nach rechts

Fall 1. p ist kein innerer Punkt eines Segments.

➔ p wurde zu Beginn in $\mathcal{Q}$ eingefügt.

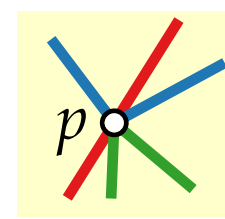
Segmente in $U(p)$ und $L(p)$ sind von Beginn an mit p gespeichert.

Wenn p verarbeitet wird, geben wir alle Segmente in $U(p) \cup L(p)$ aus.

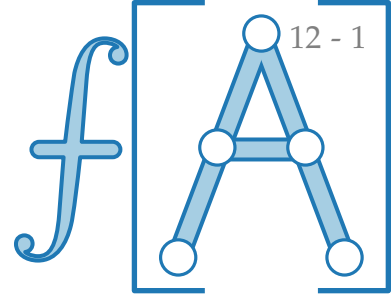
➔ Alle Segmente, die p enthalten, werden zurückgegeben.

Korrektheit (Fall 2)

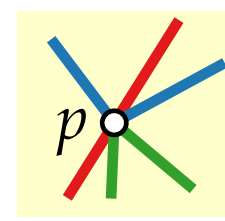
Fall 2. p ist ein innerer Punkt eines Segments.



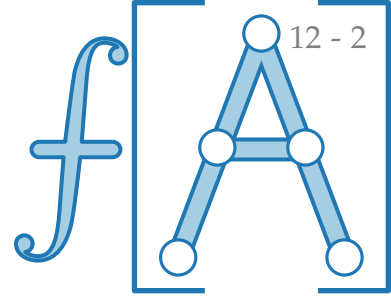
$C(p), L(p), U(p)$



Korrektheit (Fall 2)

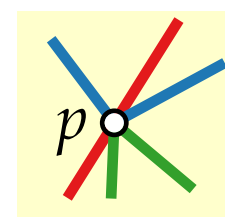


$C(p), L(p), U(p)$

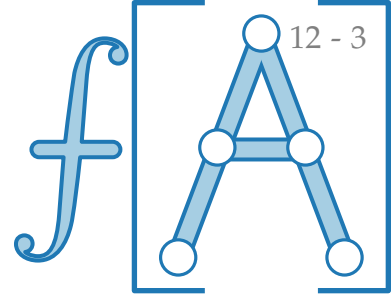


Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Korrektheit (Fall 2)



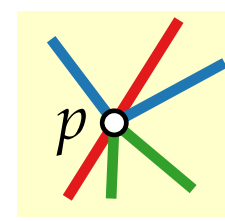
$C(p), L(p), U(p)$



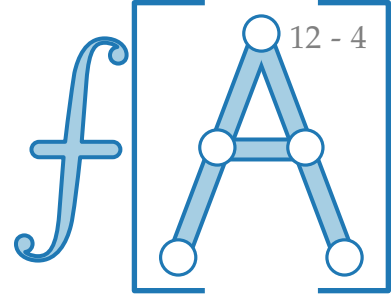
Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

Korrektheit (Fall 2)

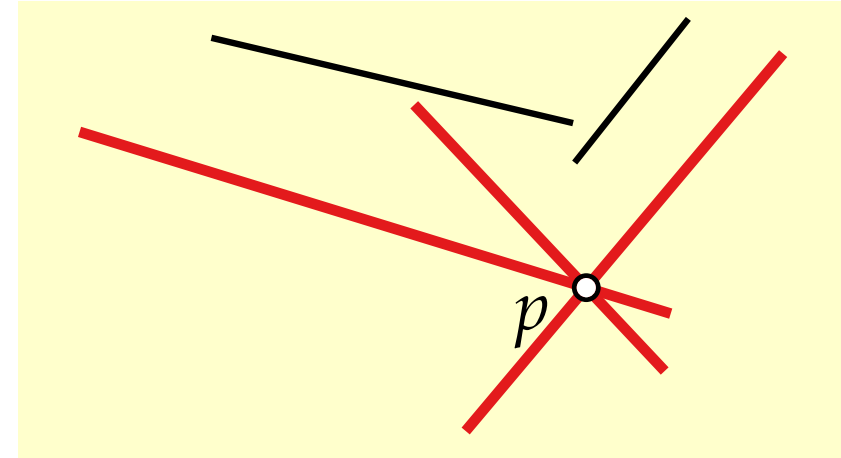


$C(p), L(p), U(p)$

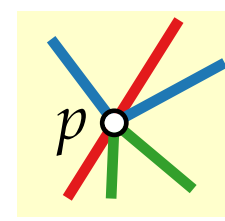


Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

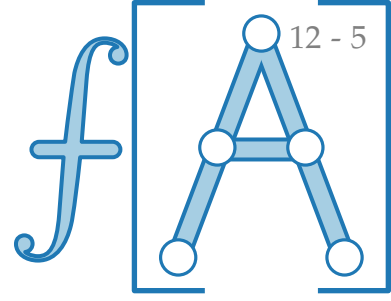
Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



Korrektheit (Fall 2)

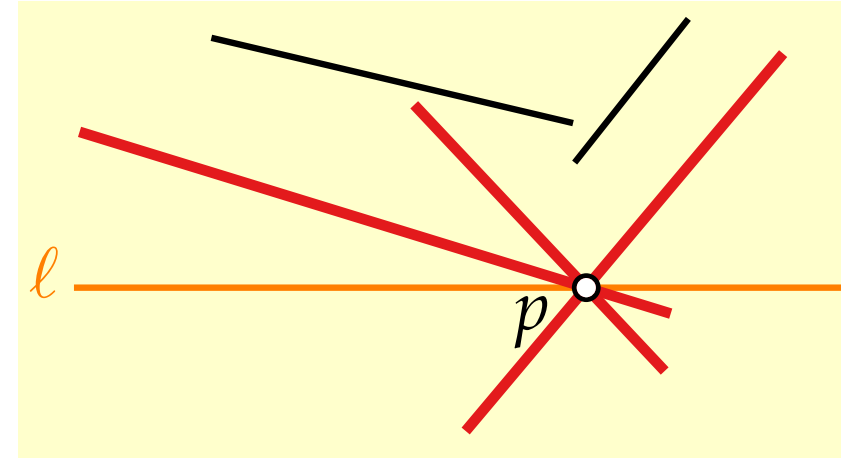


$C(p), L(p), U(p)$

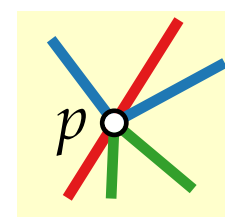


Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

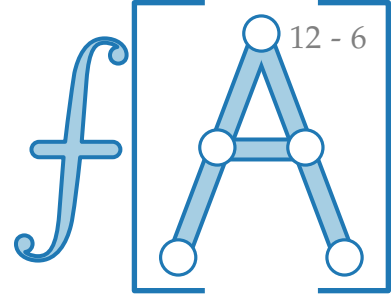
Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



Korrektheit (Fall 2)

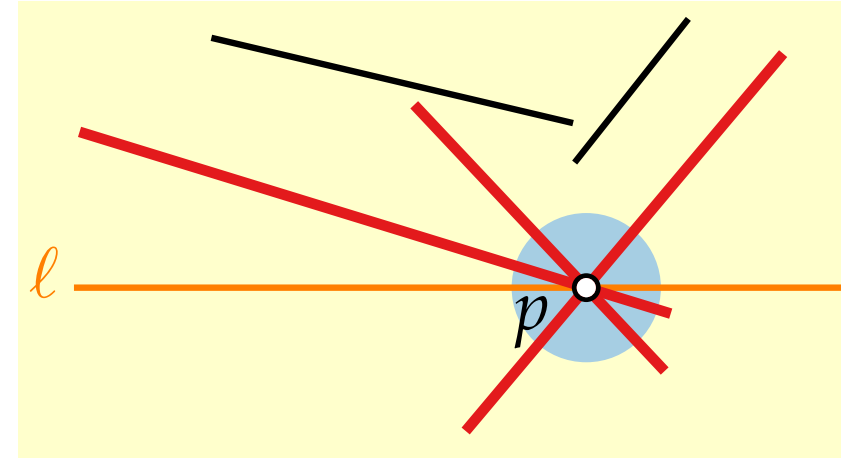


$C(p), L(p), U(p)$

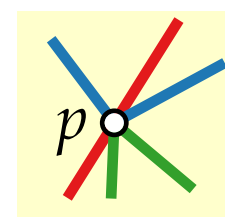


Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

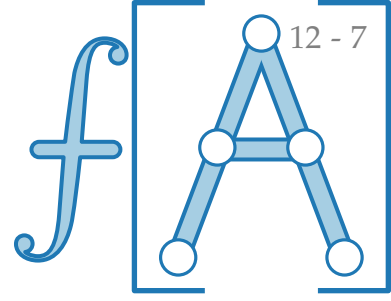
Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



Korrektheit (Fall 2)

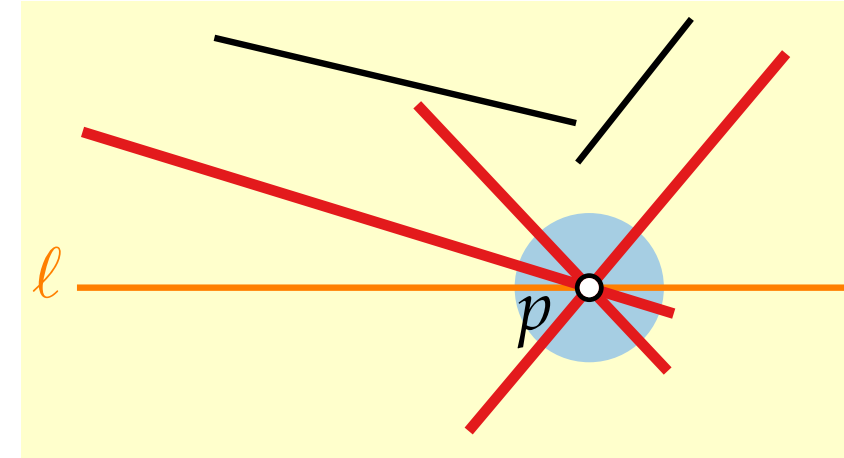


$C(p), L(p), U(p)$



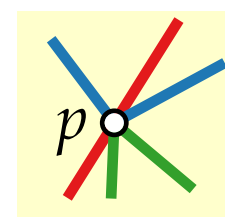
Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

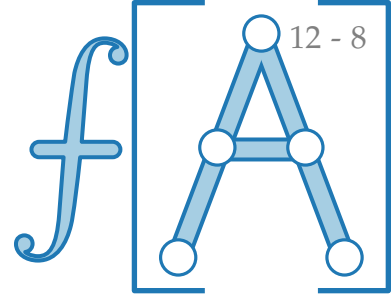


Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Korrektheit (Fall 2)

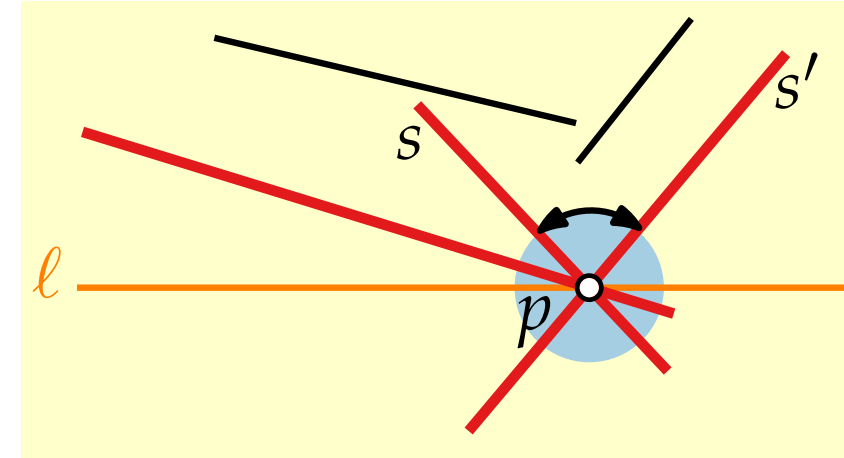


$C(p), L(p), U(p)$



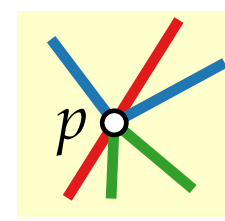
Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

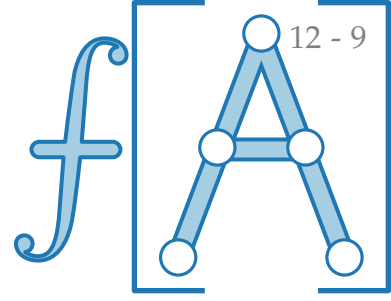


Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Korrektheit (Fall 2)

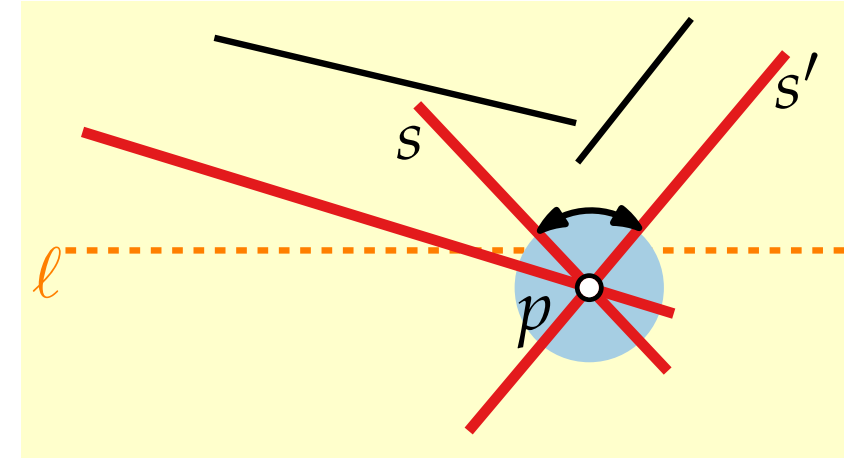


$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

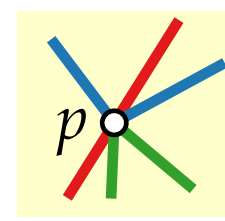
Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



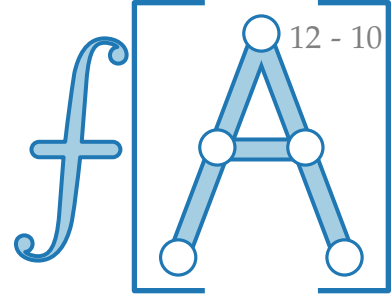
Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Wir bewegen ℓ etwas zurück.

Korrektheit (Fall 2)

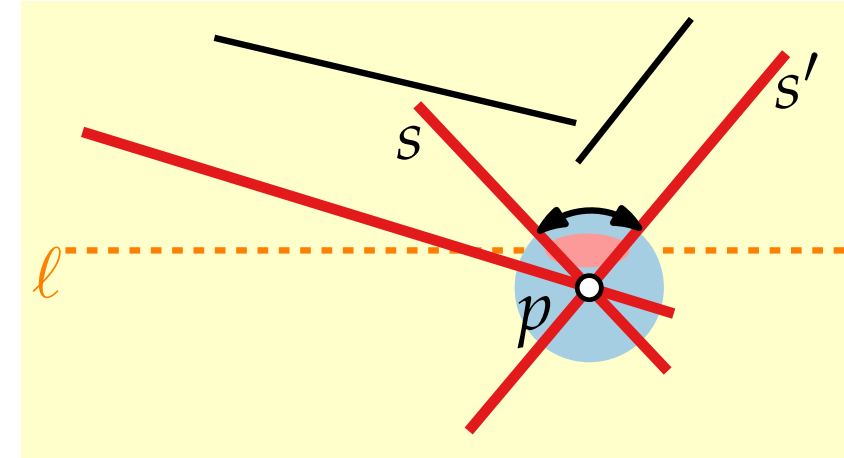


$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

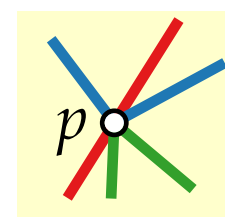


Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

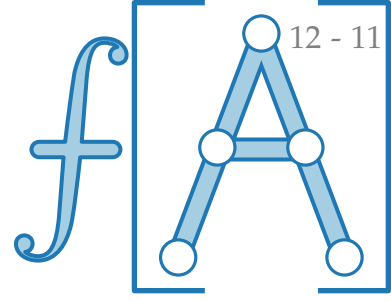
Wir bewegen ℓ etwas zurück.

Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

Korrektheit (Fall 2)

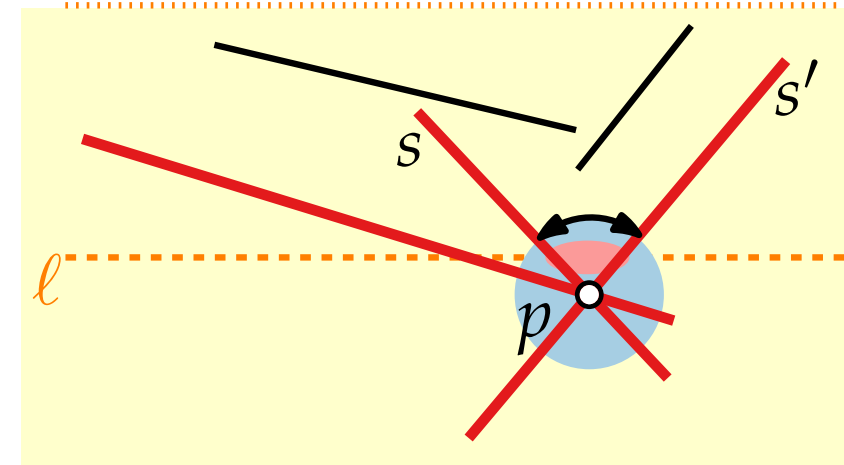


$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



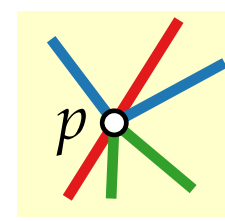
Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Wir bewegen ℓ etwas zurück.

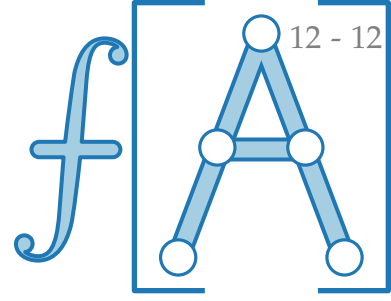
Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

Zu Beginn des Algorithmus waren sie keine Nachbarn in $\mathcal{T}$.

Korrektheit (Fall 2)

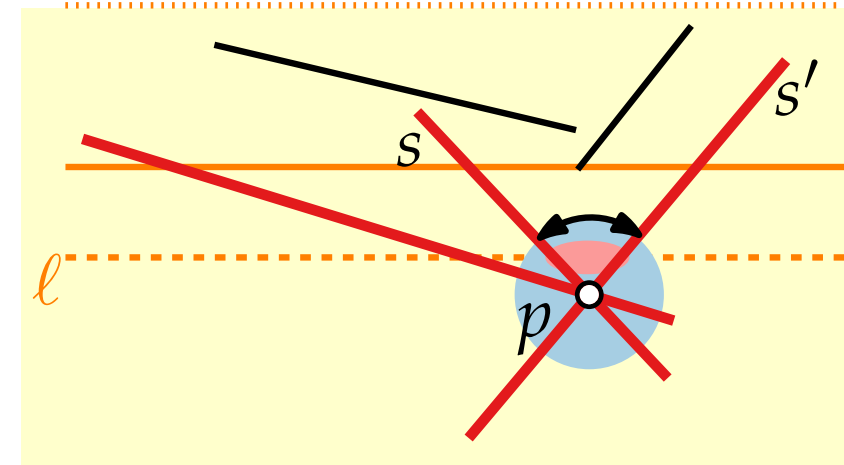


$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

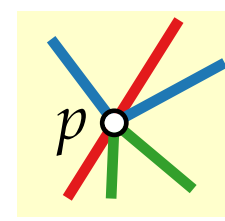
Wir bewegen ℓ etwas zurück.

Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

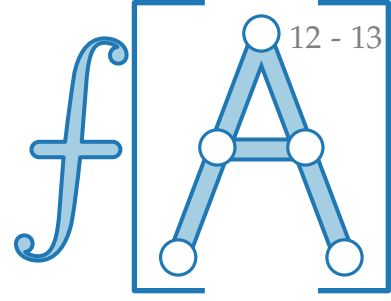
Zu Beginn des Algorithmus waren sie keine Nachbarn in $\mathcal{T}$.

➔ Es gab einen Zeitpunkt, zu dem sie Nachbarn wurden!

Korrektheit (Fall 2)

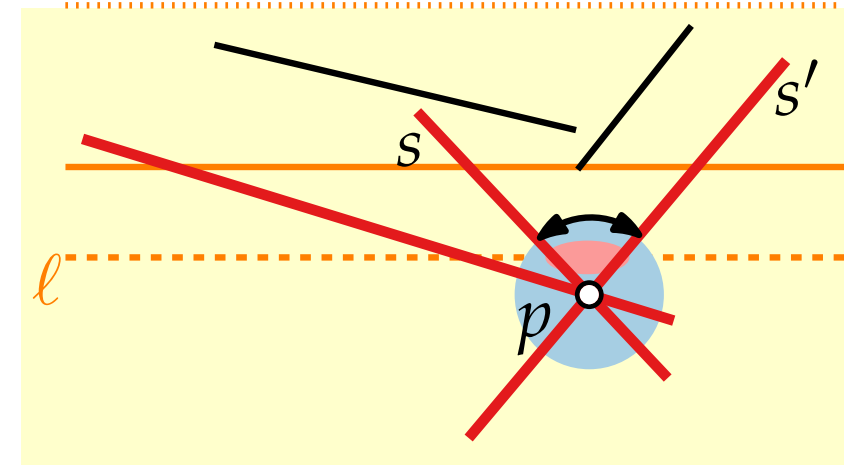


$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.



Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Wir bewegen ℓ etwas zurück.

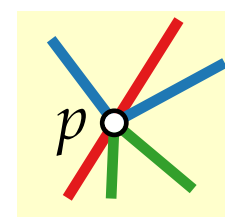
Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

Zu Beginn des Algorithmus waren sie keine Nachbarn in $\mathcal{T}$.

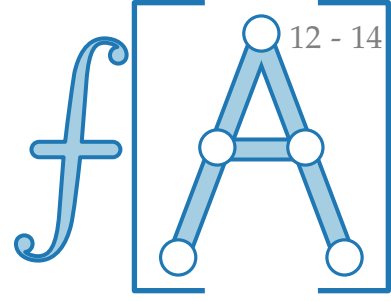
➔ Es gab einen Zeitpunkt, zu dem sie Nachbarn wurden!

Der Zeitpunkt war, wenn $\{p\} = s \cap s'$ in $\mathcal{Q}$ eingefügt wurde.

Korrektheit (Fall 2)



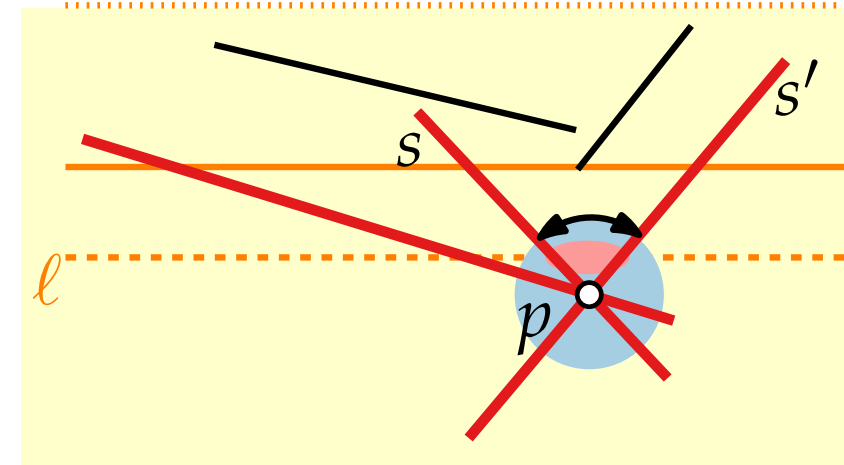
$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

Wir brauchen auch, dass **jedes** Segment mit p als innerem Punkt zu $C(p)$ hinzugefügt wurde



Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Wir bewegen ℓ etwas zurück.

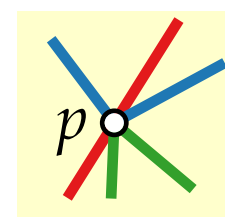
Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

Zu Beginn des Algorithmus waren sie keine Nachbarn in $\mathcal{T}$.

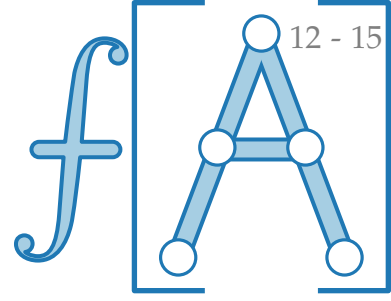
→ Es gab einen Zeitpunkt, zu dem sie Nachbarn wurden!

Der Zeitpunkt war, wenn $\{p\} = s \cap s'$ in $\mathcal{Q}$ eingefügt wurde.

Korrektheit (Fall 2)



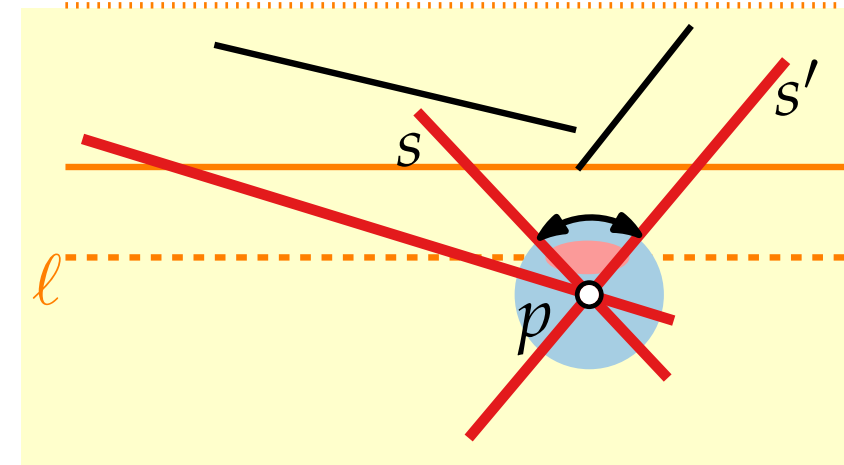
$C(p), L(p), U(p)$



Fall 2. p ist ein innerer Punkt eines Segments, d.h., $C(p) \neq \emptyset$.

Wenn p kein Endpunkt ist, müssen wir sicherstellen, dass p in $\mathcal{Q}$ eingefügt wird, bevor ℓ den Punkt p erreicht.

Wir brauchen auch, dass **jedes** Segment mit p als innerem Punkt zu $C(p)$ hinzugefügt wurde



Seien $s, s' \in C(p)$ Nachbarn in $C(p) \cup \{\ell\}$ in der Reihenfolge um p im Uhrzeigersinn.

Wir bewegen ℓ etwas zurück.

Dann waren s, s' Nachbarn auf ℓ (in $\mathcal{T}$).

Zu Beginn des Algorithmus waren sie keine Nachbarn in $\mathcal{T}$.

→ Es gab einen Zeitpunkt, zu dem sie Nachbarn wurden!

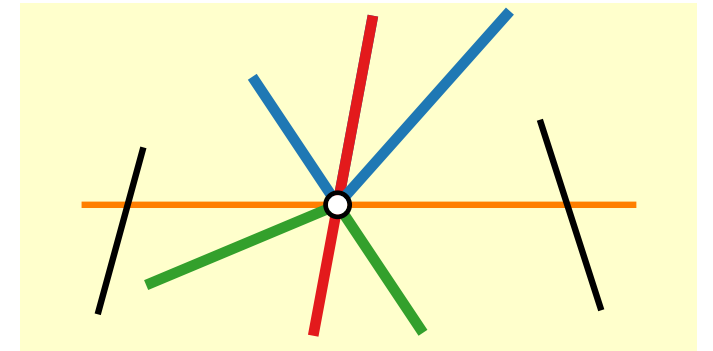
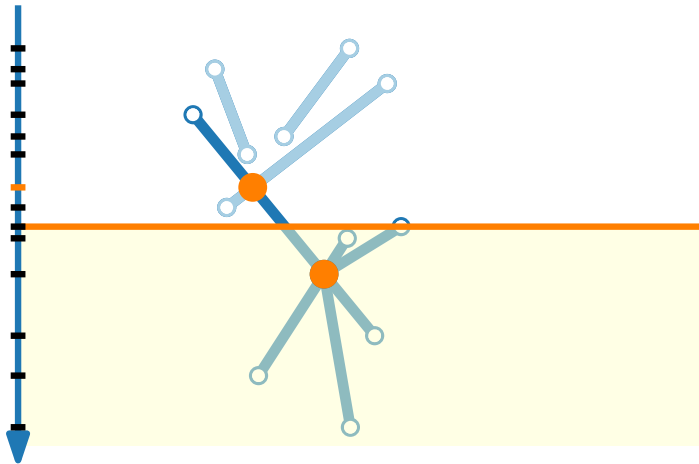
Der Zeitpunkt war, wenn $\{p\} = s \cap s'$ in $\mathcal{Q}$ eingefügt wurde. $\square$

Fortgeschrittene Algorithmen

13. Vorlesung

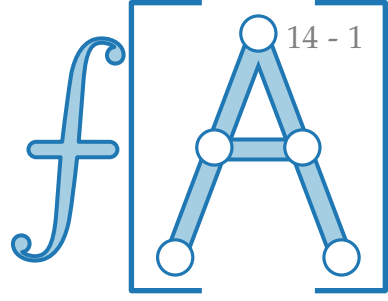
Algorithmische Geometrie II: Sweep-Line Algorithmus

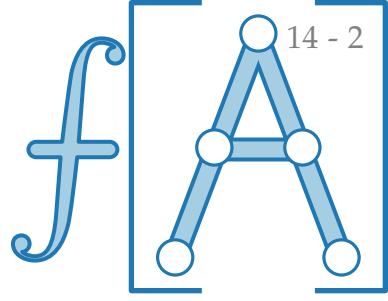
Teil V:
Laufzeit



```
 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente  
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$   
  foreach Endpunkt  $p$  von  $s$  do  
    if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$   
    if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$   
    if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$   
while  $Q \neq \emptyset$  do  
   $p = Q.\text{NEXTEVENT}()$   
   $Q.\text{DELETE}(p)$   
   $\text{HANDLEEVENT}(p)$ 
```

Laufzeit?





```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do

```

```

    foreach Endpunkt  $p$  von  $s$  do

```

```

        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 

```

```

        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 

```

```

        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do

```

```

     $p = Q.\text{NEXTEVENT}()$ 

```

```

     $Q.\text{DELETE}(p)$ 

```

```

     $\text{HANDLEEVENT}(p)$ 

```

```

 $\text{HANDLEEVENT}(\text{Event } p)$ 

```

```

if  $|U(p) \cup L(p) \cup C(p)| > 1$  then

```

```

    | gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !

```

```

    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein

```

```

if  $U(p) \cup C(p) = \emptyset$  then

```

```

     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 

```

```

     $\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p)$ 

```

```

else

```

```

     $s_{\text{left}}/s_{\text{right}}$  = linkestes / rechtestes Segment in  $U(p) \cup C(p)$ 

```

```

     $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 

```

```

     $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 

```

```

     $\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$ 

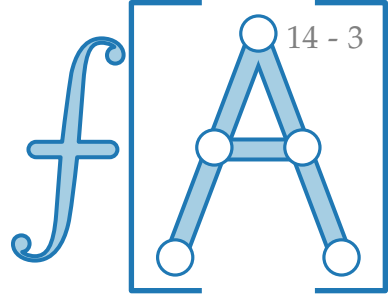
```

```

     $\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$ 

```

Laufzeit?



$Q = \emptyset$; $\mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p)$; $L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$

while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

 gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

 füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$

if $x \notin Q$ **then** $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$

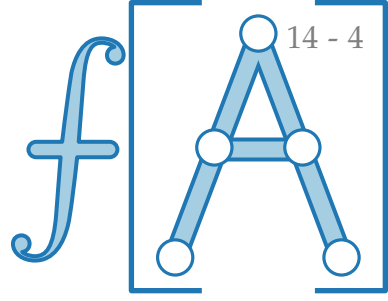
$b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$

$b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do

```

```

    foreach Endpunkt  $p$  von  $s$  do

```

```

        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 

```

```

        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 

```

```

        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do

```

```

     $p = Q.\text{NEXTEVENT}()$ 

```

```

     $Q.\text{DELETE}(p)$ 

```

```

     $\text{HANDLEEVENT}(p)$ 

```

```

HANDLEEVENT(Event  $p$ )

```

```

if  $|U(p) \cup L(p) \cup C(p)| > 1$  then

```

```

    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !

```

```

    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein

```

```

if  $U(p) \cup C(p) = \emptyset$  then

```

```

     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 

```

```

     $\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$ 
    if  $x \notin Q$  then  $Q.\text{insert}(x)$ 

```

```

else

```

```

     $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 

```

```

     $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 

```

```

     $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 

```

```

     $\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$ 

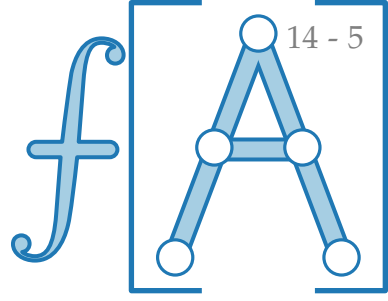
```

```

     $\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$ 

```

Laufzeit?



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do                        $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

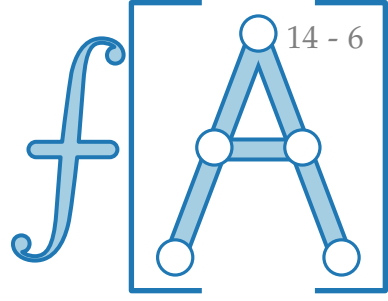
```

Laufzeit?

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
        if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do                        $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

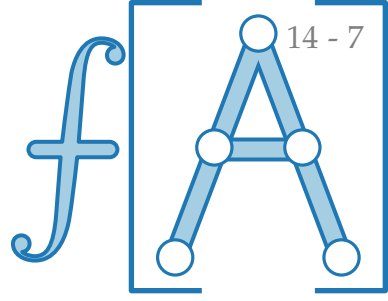
```

Laufzeit?

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
if  $U(p) \cup C(p) = \emptyset$  then
     $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
    if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
else
     $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
     $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
     $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
    FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do                        $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$             $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

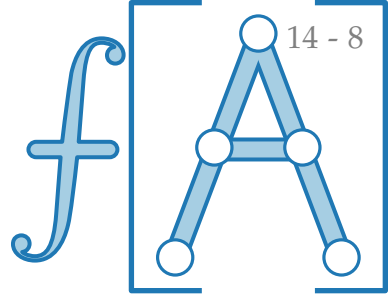
```

Laufzeit?

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
if  $U(p) \cup C(p) = \emptyset$  then
     $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
    if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
else
     $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
     $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
     $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
    FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do  $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$   $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

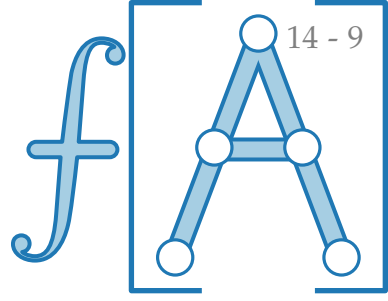
```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
if  $U(p) \cup C(p) = \emptyset$  then
     $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
    if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
else
     $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
     $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
     $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
    FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$$|Q| \leq$$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
  foreach Endpunkt  $p$  von  $s$  do  $2n \cdot$ 
    if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$   $\mathcal{O}(\log |Q|)$ 
    if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
    if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
   $p = Q.\text{NEXTEVENT}()$ 
   $Q.\text{DELETE}(p)$ 
   $\text{HANDLEEVENT}(p)$ 

```

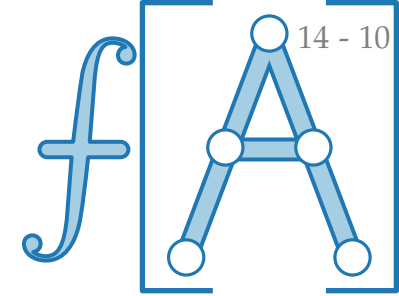
Laufzeit?

$$|Q| \leq 2n + k$$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
  | gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
  lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
  füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
if  $U(p) \cup C(p) = \emptyset$  then
   $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
  FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
  if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
else
   $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
   $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
   $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
  FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
  FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do  $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$   $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

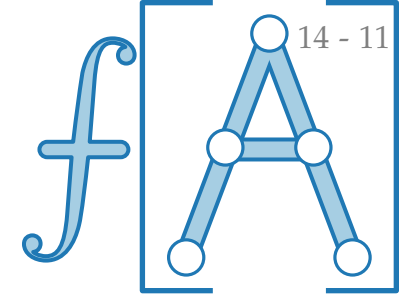
```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
     $\lfloor$  gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
if  $U(p) \cup C(p) = \emptyset$  then
     $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
    if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
else
     $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
     $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
     $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
    FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
    FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2}$$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do  $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$   $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

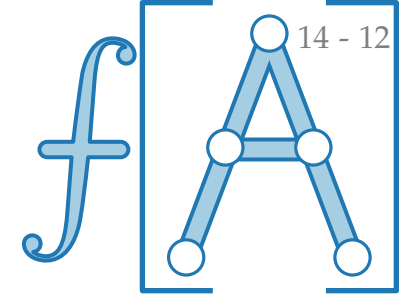
```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
         $\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \begin{cases} \{x\} = s \cap s' \\ \text{if } x \notin Q \text{ then } Q.\text{insert}(x) \end{cases}$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
         $\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$ 
         $\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$ 

```

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log |Q|)$

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

```

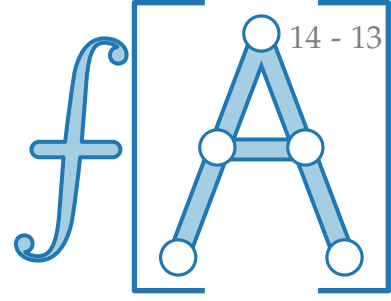
HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

$\{x\} = s \cap s'$
 if $x \notin Q$ then $Q.\text{insert}(x)$

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do                        $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$             $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

```

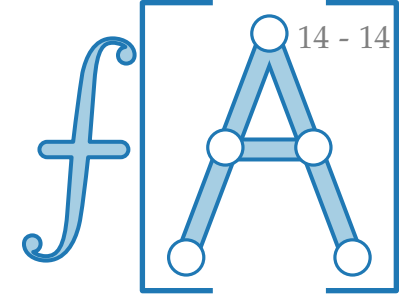
HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \begin{cases} \{x\} = s \cap s' \\ \text{if } x \notin Q \text{ then } Q.\text{insert}(x) \end{cases}$ 
    else
         $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
         $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
         $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do                        $2n \cdot$ 
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$             $\mathcal{O}(\log |Q|)$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

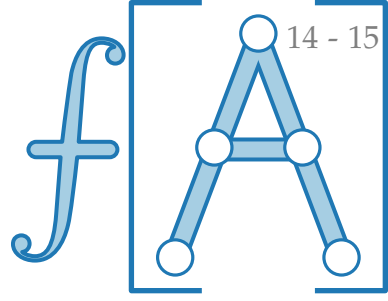
```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \begin{cases} \{x\} = s \cap s' \\ \text{if } x \notin Q \text{ then } Q.\text{insert}(x) \end{cases}$ 
    else
         $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
         $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
         $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$



$Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$

$2n \cdot n$
 $\mathcal{O}(\log |Q|)$

while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$
if $x \notin Q$ **then** $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

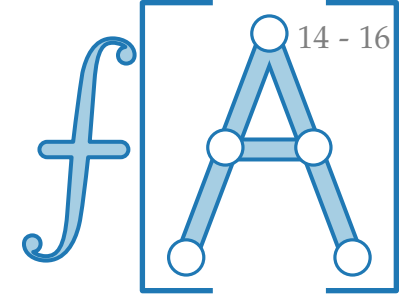
$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$$



$Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$

$2n \cdot$

n

$\mathcal{O}(\log |Q|)$

while $Q \neq \emptyset$ **do**

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$
if $x \notin Q$ **then** $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

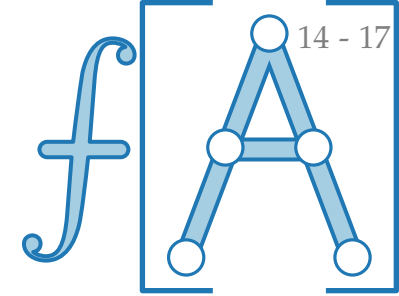
$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$$



$Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$

$2n \cdot$

n

$\mathcal{O}(\log |Q|)$

while $Q \neq \emptyset$ **do** $(2n + k) \cdot$

$p = Q.\text{NEXTEVENT}()$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$
if $x \notin Q$ **then** $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

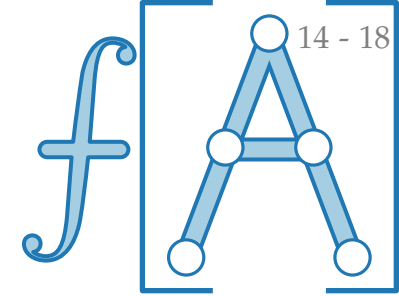
$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log |Q|)$

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

Laufzeit?

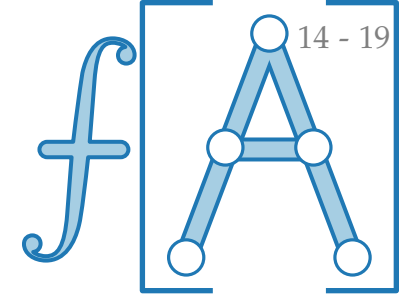
$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
         $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
         $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

$\{x\} = s \cap s'$
 if $x \notin Q$ then $Q.\text{insert}(x)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log |Q|)$
 n

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

Laufzeit?

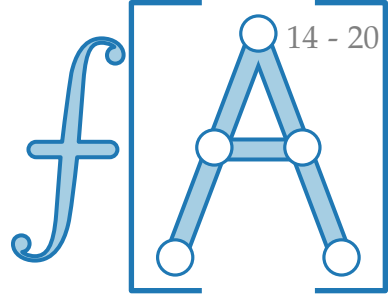
$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}}$  = linker/rechter Nachbar von  $p$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}}$  = linkstes / rechtestes Segment in  $U(p) \cup C(p)$ 
         $b_{\text{left}}$  = linker Nachbar von  $s_{\text{left}}$  in  $\mathcal{T}$ 
         $b_{\text{right}}$  = rechter Nachbar von  $s_{\text{right}}$  in  $\mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

$\{x\} = s \cap s'$
 if $x \notin Q$ then $Q.\text{insert}(x)$



$Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
foreach $s \in S$ **do** // initialisiere Eventschlange Q

foreach Endpunkt p von s **do**

if $p \notin Q$ **then** $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s **then** $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s **then** $U(p).\text{APPEND}(s)$

$2n \cdot$
 $\mathcal{O}(\log |Q|)$

while $Q \neq \emptyset$ **do** $(2n + k) \cdot$
 $p = Q.\text{NEXTEVENT}()$ $\mathcal{O}(\log n)$
 $Q.\text{DELETE}(p)$
 $\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ **then**

gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus
 lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ **then**

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$
if $x \notin Q$ **then** $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

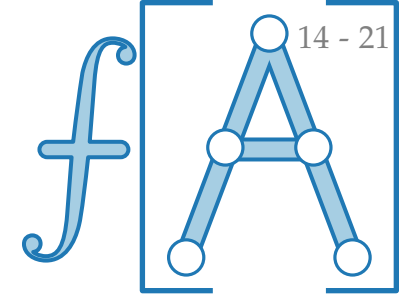
$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$$



$Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$ // Dummy Segmente
 foreach $s \in S$ do // initialisiere Eventschlange Q

foreach Endpunkt p von s do

if $p \notin Q$ then $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$

if p unterer Endpunkt von s then $L(p).\text{APPEND}(s)$

if p oberer Endpunkt von s then $U(p).\text{APPEND}(s)$

$2n \cdot$

n

$\mathcal{O}(\log |Q|)$

while $Q \neq \emptyset$ do $(2n + k) \cdot$

$p = Q.\text{NEXTEVENT}()$ $\mathcal{O}(\log n)$

$Q.\text{DELETE}(p)$

$\text{HANDLEEVENT}(p)$

$\text{HANDLEEVENT}(\text{Event } p)$

if $|U(p) \cup L(p) \cup C(p)| > 1$ then

gebe Schnittpunkt p und Segmente in $U(p) \cup L(p) \cup C(p)$ aus

lösche $L(p) \cup C(p)$ aus $\mathcal{T}$ // aufeinanderfolgend in $\mathcal{T}$!

füge $U(p) \cup C(p)$ in $\mathcal{T}$ in der Reihenfolge knapp unter ℓ ein

if $U(p) \cup C(p) = \emptyset$ then

$b_{\text{left}}/b_{\text{right}}$ = linker/rechter Nachbar von p in $\mathcal{T}$

$\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$
 if $x \notin Q$ then $Q.\text{insert}(x)$

else

$s_{\text{left}}/s_{\text{right}}$ = linkstes / rechtestes Segment in $U(p) \cup C(p)$

b_{left} = linker Nachbar von s_{left} in $\mathcal{T}$

b_{right} = rechter Nachbar von s_{right} in $\mathcal{T}$

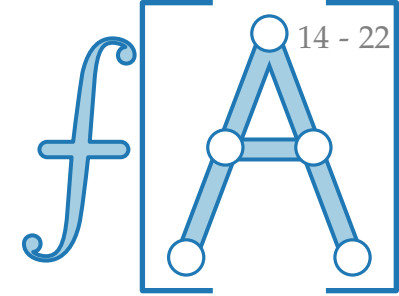
$\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$

$\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$

Laufzeit?

$$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$$

$$\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log \cancel{n})$

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

Laufzeit?

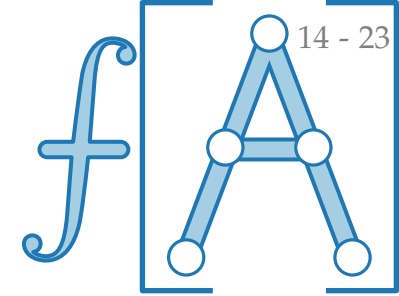
$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

$\{x\} = s \cap s'$
 if $x \notin Q$ then $Q.\text{insert}(x)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot$ $\mathcal{O}(\log \cancel{n})$

```

while  $Q \neq \emptyset$  do                                $(2n + k) \cdot$ 
     $p = Q.\text{NEXTEVENT}()$                                 $\mathcal{O}(\log n)$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

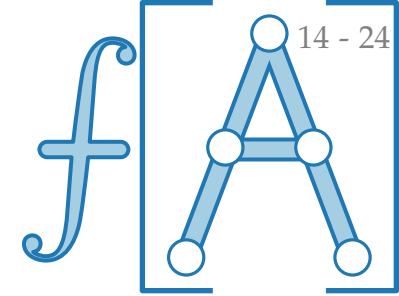
```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$   $\mathcal{O}(\log |\mathcal{T}|)$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
        if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot$ $\mathcal{O}(\log \cancel{n})$

```

while  $Q \neq \emptyset$  do                                 $(2n + k) \cdot$ 
     $p = Q.\text{NEXTEVENT}()$                                  $\mathcal{O}(\log n)$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

```

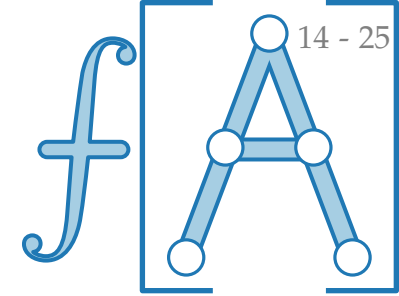
HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$   $\mathcal{O}(\log |\mathcal{T}|)$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
        if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

$|\mathcal{T}| \leq n$



```

 $\mathcal{Q} = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin \mathcal{Q}$  then  $\mathcal{Q}.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log \cancel{n})$

```

while  $\mathcal{Q} \neq \emptyset$  do
     $p = \mathcal{Q}.\text{NEXTEVENT}()$ 
     $\mathcal{Q}.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

Laufzeit?

$|\mathcal{Q}| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|\mathcal{Q}|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

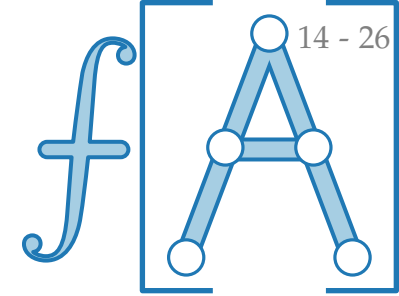
$|\mathcal{T}| \leq n$
 $\rightarrow \log(|\mathcal{T}|) \in \mathcal{O}(\log n)$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

$\{x\} = s \cap s'$
 if $x \notin \mathcal{Q}$ then $\mathcal{Q}.\text{insert}(x)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do                                     // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot$ $\mathcal{O}(\log \cancel{n})$

```

while  $Q \neq \emptyset$  do                                 $(2n + k) \cdot$ 
     $p = Q.\text{NEXTEVENT}()$                                  $\mathcal{O}(\log n)$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

```

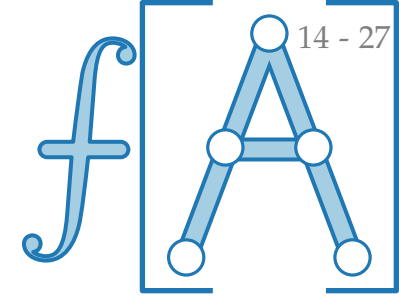
HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$   $\mathcal{O}(\log \cancel{n})$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )  $\rightarrow \{x\} = s \cap s'$ 
        if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

$|\mathcal{T}| \leq n$
 $\rightarrow \log(|\mathcal{T}|) \in \mathcal{O}(\log n)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \mathcal{O}(\log \cancel{n})$

```

while  $Q \neq \emptyset$  do
     $p = Q.\text{NEXTEVENT}()$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

$(2n + k) \cdot \mathcal{O}(\log n)$

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, b_{\text{right}}, p$ )
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
        FINDNEWEVENT( $b_{\text{left}}, s_{\text{left}}, p$ )
        FINDNEWEVENT( $b_{\text{right}}, s_{\text{right}}, p$ )

```

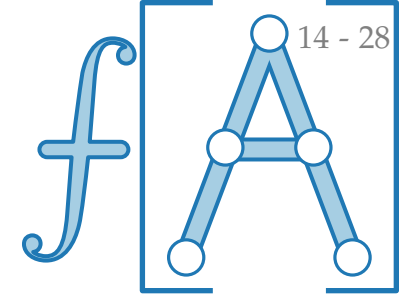
$\mathcal{O}(\log \cancel{n})$

$\{x\} = s \cap s'$
if $x \notin Q$ then $Q.\text{insert}(x)$

Laufzeit?

$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

$|\mathcal{T}| \leq n$
 $\rightarrow \log(|\mathcal{T}|) \in \mathcal{O}(\log n)$



```

 $Q = \emptyset; \mathcal{T} = \langle \text{vertikale Geraden } x = -\infty \text{ und } x = +\infty \rangle$  // Dummy Segmente
foreach  $s \in S$  do // initialisiere Eventschlange  $Q$ 
    foreach Endpunkt  $p$  von  $s$  do
        if  $p \notin Q$  then  $Q.\text{INSERT}(p); L(p) = U(p) = C(p) = \emptyset$ 
        if  $p$  unterer Endpunkt von  $s$  then  $L(p).\text{APPEND}(s)$ 
        if  $p$  oberer Endpunkt von  $s$  then  $U(p).\text{APPEND}(s)$ 

```

$2n \cdot \quad n$
 $\mathcal{O}(\log |Q|)$

```

while  $Q \neq \emptyset$  do  $(2n + k) \cdot$ 
     $p = Q.\text{NEXTEVENT}()$   $\mathcal{O}(\log n)$ 
     $Q.\text{DELETE}(p)$ 
     $\text{HANDLEEVENT}(p)$ 

```

```

HANDLEEVENT(Event  $p$ )
if  $|U(p) \cup L(p) \cup C(p)| > 1$  then
    gebe Schnittpunkt  $p$  und Segmente in  $U(p) \cup L(p) \cup C(p)$  aus
    lösche  $L(p) \cup C(p)$  aus  $\mathcal{T}$  // aufeinanderfolgend in  $\mathcal{T}$ !
    füge  $U(p) \cup C(p)$  in  $\mathcal{T}$  in der Reihenfolge knapp unter  $\ell$  ein
    if  $U(p) \cup C(p) = \emptyset$  then
         $b_{\text{left}}/b_{\text{right}} = \text{linker/rechter Nachbar von } p \text{ in } \mathcal{T}$   $\mathcal{O}(\log |T|)$ 
         $\text{FINDNEWEVENT}(b_{\text{left}}, b_{\text{right}}, p) \rightarrow \{x\} = s \cap s'$ 
        if  $x \notin Q$  then  $Q.\text{insert}(x)$ 
    else
         $s_{\text{left}}/s_{\text{right}} = \text{linkestes / rechtestes Segment in } U(p) \cup C(p)$ 
         $b_{\text{left}} = \text{linker Nachbar von } s_{\text{left}} \text{ in } \mathcal{T}$ 
         $b_{\text{right}} = \text{rechter Nachbar von } s_{\text{right}} \text{ in } \mathcal{T}$ 
         $\text{FINDNEWEVENT}(b_{\text{left}}, s_{\text{left}}, p)$ 
         $\text{FINDNEWEVENT}(b_{\text{right}}, s_{\text{right}}, p)$ 

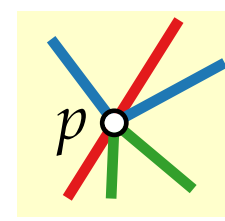
```

Laufzeit?

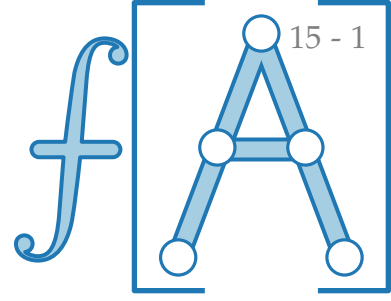
$|Q| \leq 2n + k \leq 2n + \frac{n(n-1)}{2} \in \mathcal{O}(n^2)$
 $\rightarrow \log(|Q|) \in \mathcal{O}(\log n^2) = \mathcal{O}(\log n)$

$|\mathcal{T}| \leq n$
 $\rightarrow \log(|\mathcal{T}|) \in \mathcal{O}(\log n)$

Laufzeit



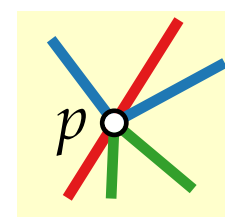
$C(p), L(p), U(p)$



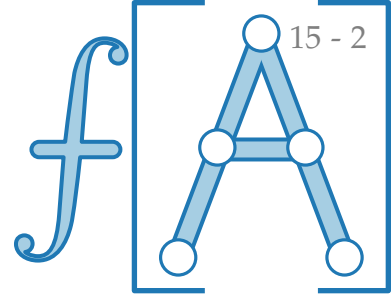
Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$

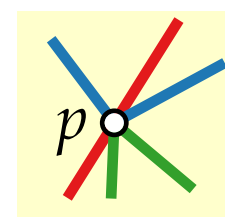


Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

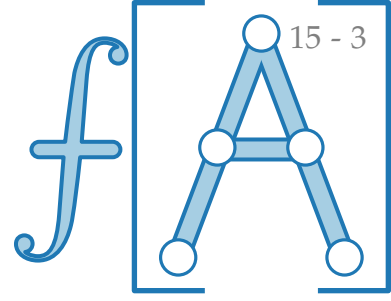
Beweis. Sei p ein Eventpunkt,

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$

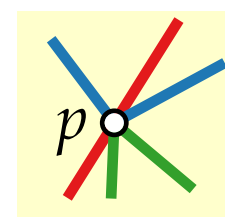


Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

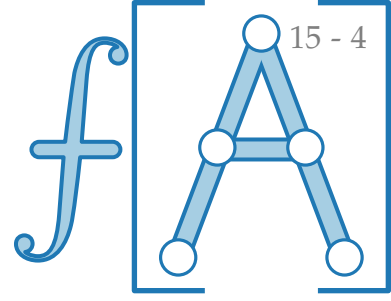
Beweis. Sei p ein Eventpunkt,
$$m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$$

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$

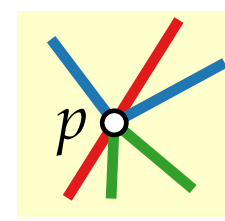


Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

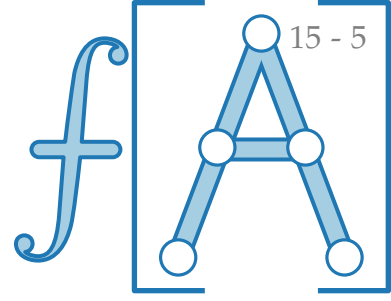
Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$



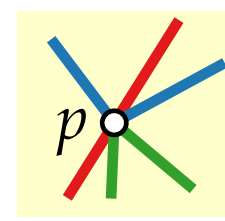
Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

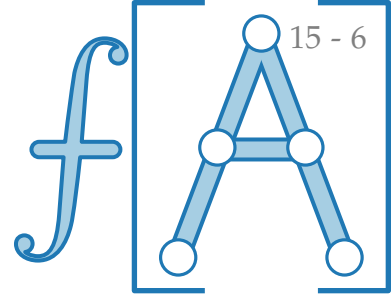
Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$



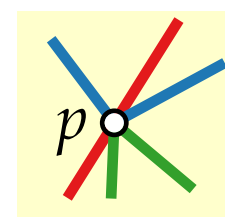
Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

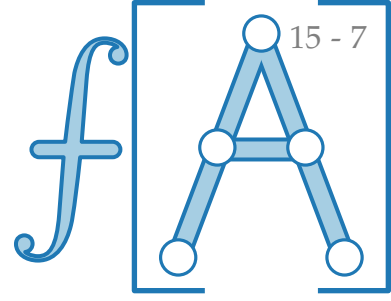
Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.
Wir zeigen, dass $m \in \mathcal{O}(n + k)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

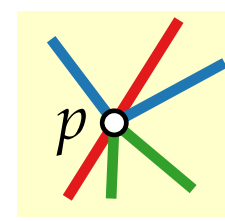
Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

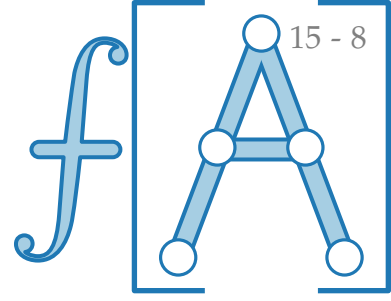
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Laufzeit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

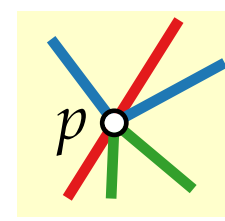
lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

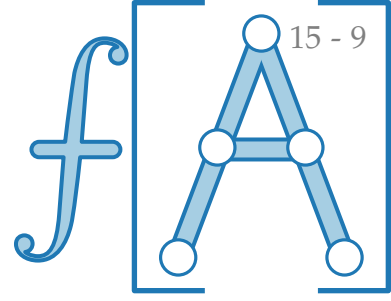
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \}$

Laufzeit



$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

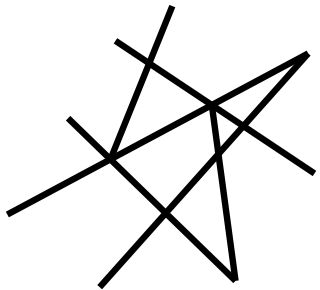
Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

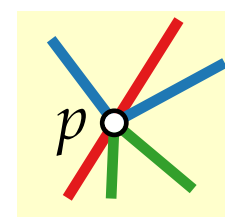
Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

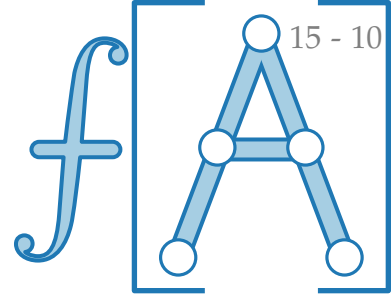
Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \}$



Laufzeit



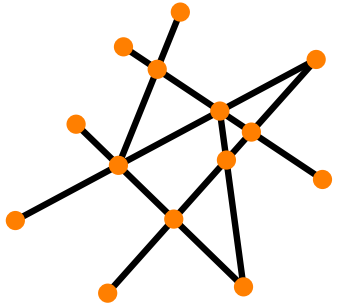
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

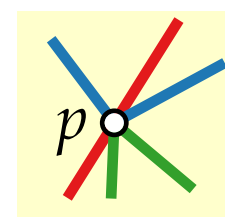


Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

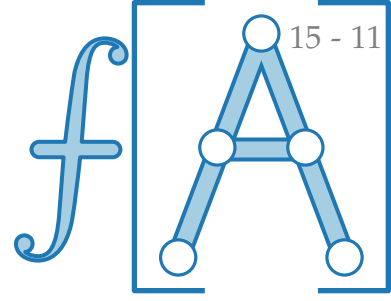
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \}$

Laufzeit



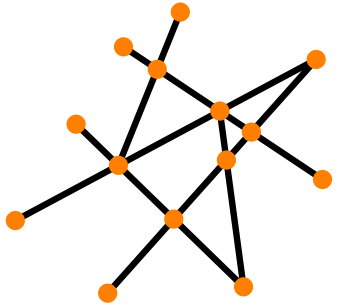
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein

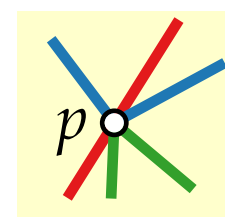


Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

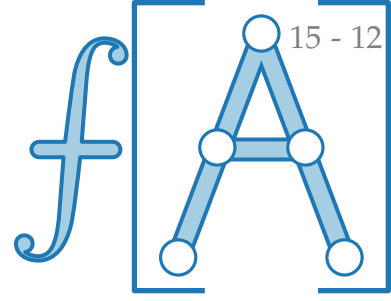
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Laufzeit



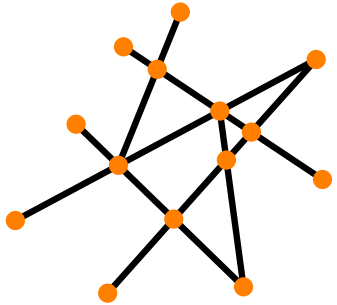
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



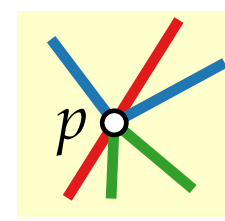
Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

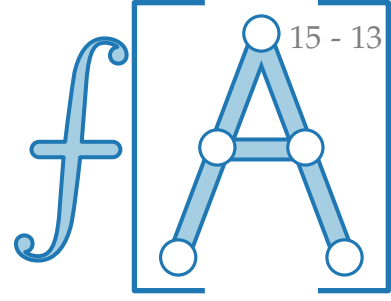
Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

Laufzeit



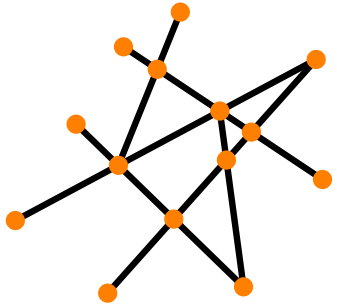
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

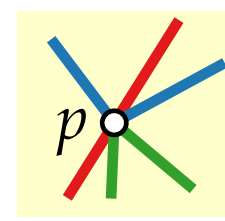
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

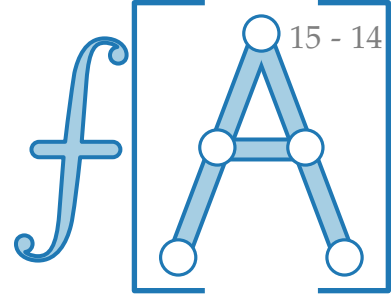
Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

$$\rightarrow m = \sum_p \deg(p) = 2|E|$$

Laufzeit



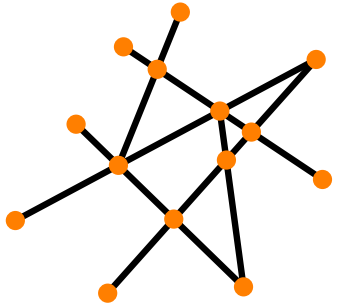
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

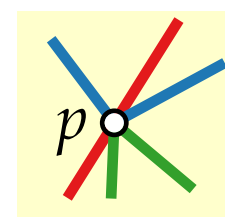
Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

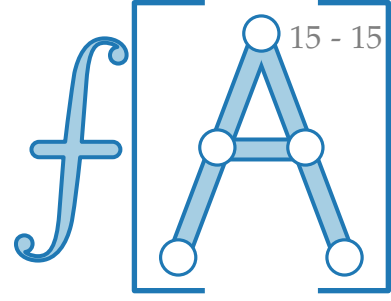
Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

$\Rightarrow m = \sum_p \deg(p) = 2|E| \leq$

Laufzeit



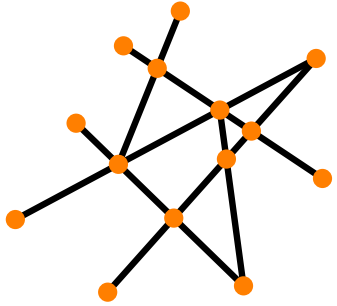
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

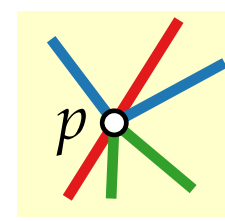
Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

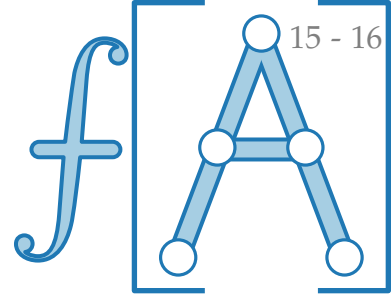
$\Rightarrow m = \sum_p \deg(p) = 2|E| \leq$

Euler (G ist planar!)

Laufzeit



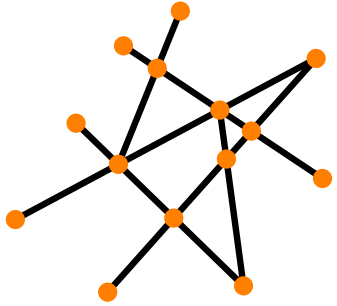
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

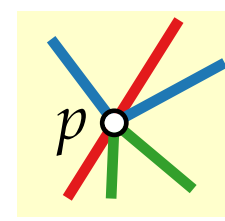
Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

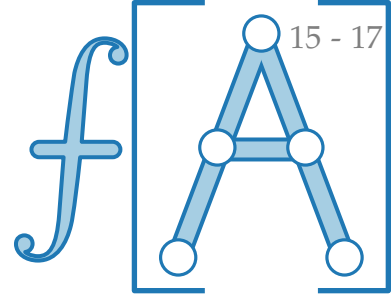
$$\rightarrow m = \sum_p \deg(p) = 2|E| \leq 2 \cdot (3|V| - 6)$$

Euler (G ist planar!)

Laufzeit



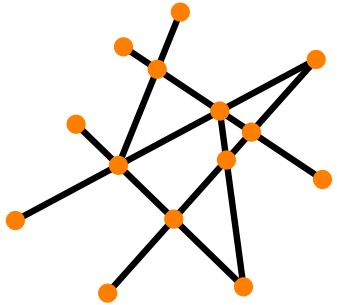
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

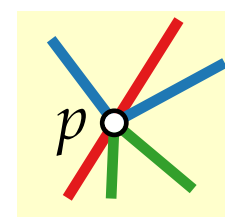
Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

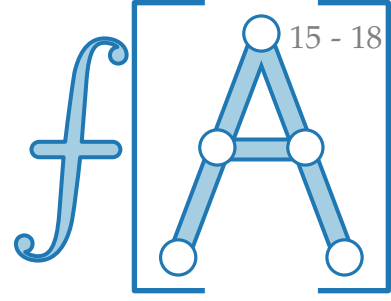
$$\rightarrow m = \sum_p \deg(p) = 2|E| \leq 2 \cdot (3|V| - 6) \in \mathcal{O}(\quad)$$

Euler (G ist planar!)

Laufzeit



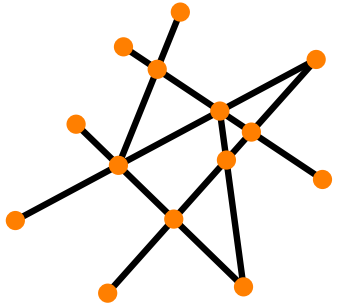
$C(p), L(p), U(p)$



Lemma. FINDINTERSECTIONS() findet k Schnittpunkte unter n nicht-überlappenden Liniensegmente in $\mathcal{O}((n + k) \log n)$ Zeit.

Beweis. Sei p ein Eventpunkt,
 $m(p) = |L(p) \cup C(p)| + |U(p) \cup C(p)|$
und $m = \sum_p m(p)$.

lösche $L(p) \cup C(p)$
füge $U(p) \cup C(p)$ ein



Dann ist die Laufzeit $\mathcal{O}((n + k) \log n + m \log n)$.

Wir zeigen, dass $m \in \mathcal{O}(n + k)$. ($\Rightarrow$ Lemma)

Definiere (geometrischen) Graph $G = (V, E)$ mit
 $V = \{ \text{Endpunkte, Schnittpunkte} \} \Rightarrow |V| \leq 2n + k$.

Für jeden Punkt $p \in V$: $m(p) = \deg(p)$.

$$\rightarrow m = \sum_p \deg(p) = 2|E| \leq 2 \cdot (3|V| - 6)$$

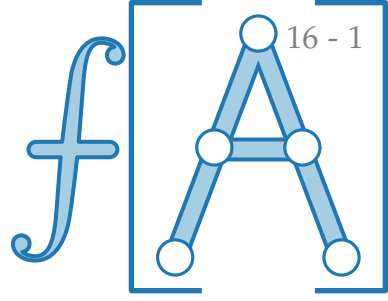
Euler (G ist planar!)

$\in \mathcal{O}(n + k)$



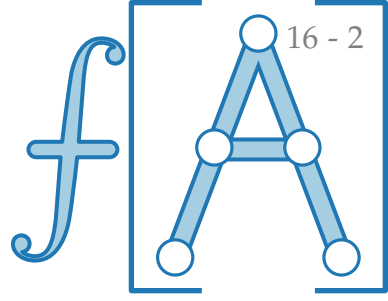
Resultat

Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.



Resultat

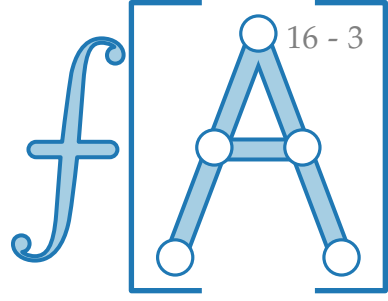
Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.



Resultat

Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

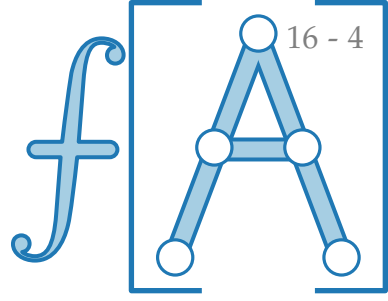
Sicher?



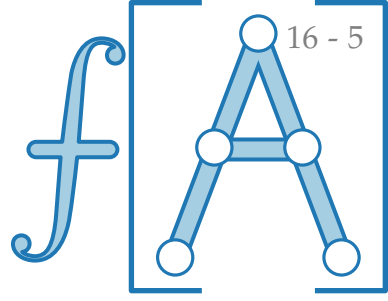
Resultat

Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange Q enthält



Resultat

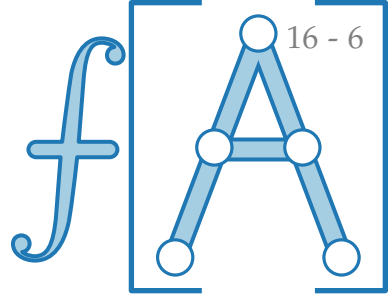


Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**

Resultat

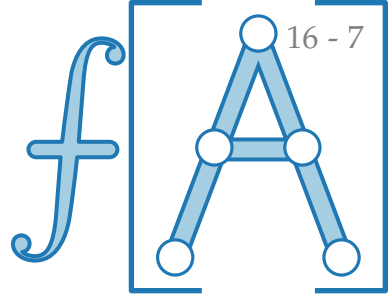


Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

Resultat



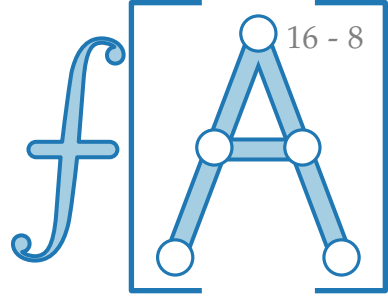
Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

➔ (worst-case) Platzverbrauch $\in$

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

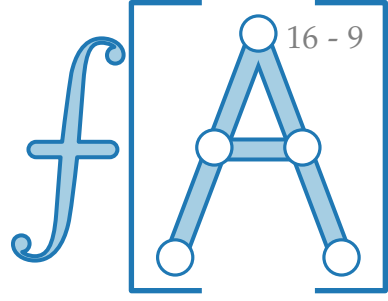
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

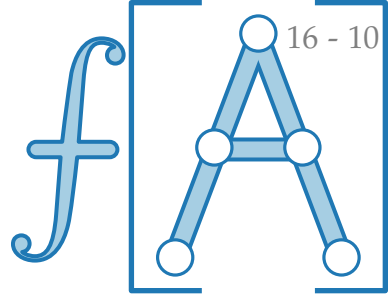
Sicher? Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

➔ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?

Resultat



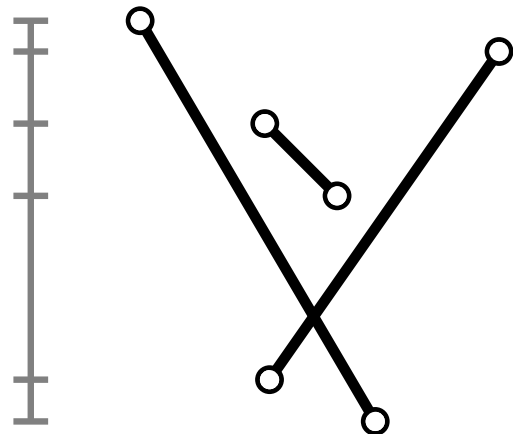
Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange $\mathcal{Q}$ enthält

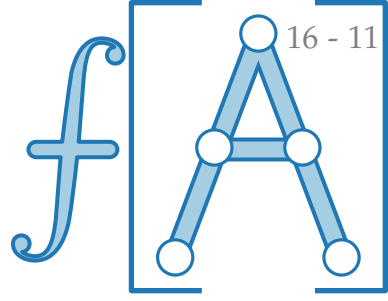
- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

➔ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

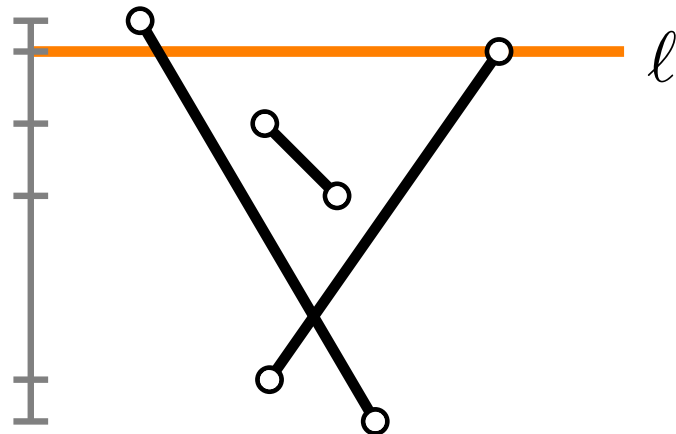
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

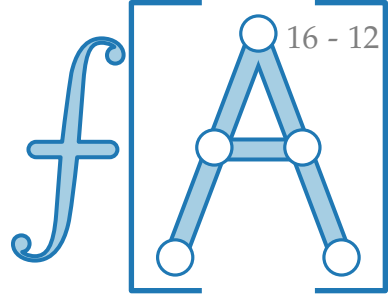
- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

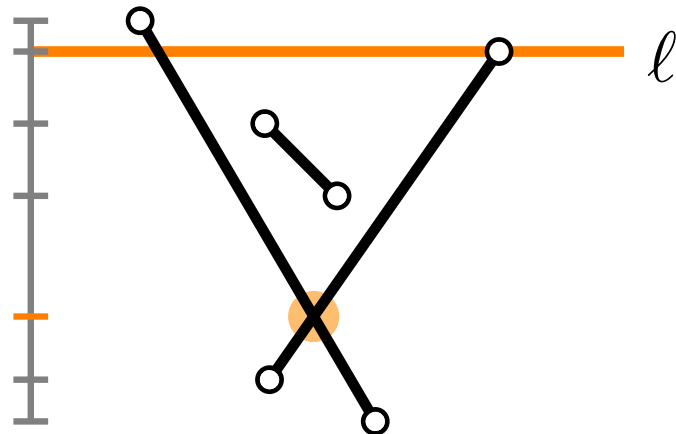
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

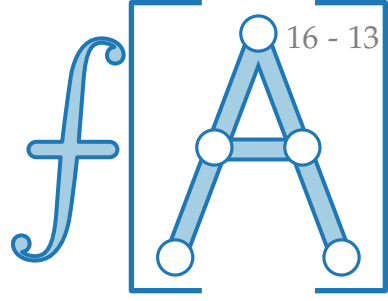
→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



- füge $s \cap s'$ in $\mathcal{Q}$ ein

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

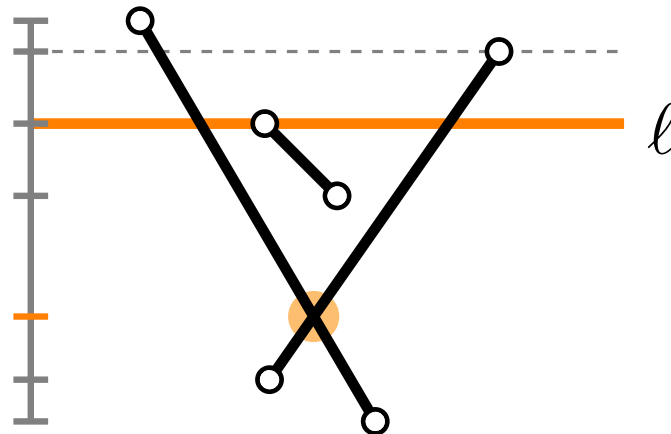
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

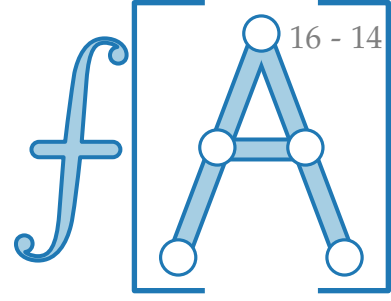
→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



- füge $s \cap s'$ in $\mathcal{Q}$ ein

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

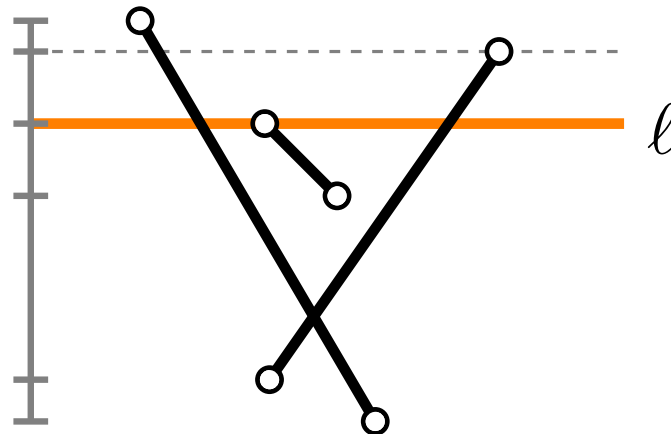
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

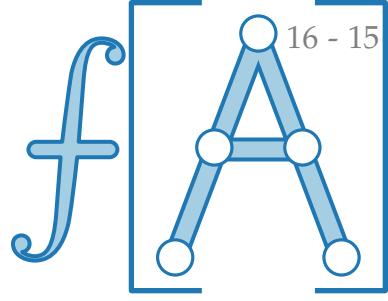
→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



- füge $s \cap s'$ in $\mathcal{Q}$ ein
- entferne $s \cap s'$ aus $\mathcal{Q}$

Resultat



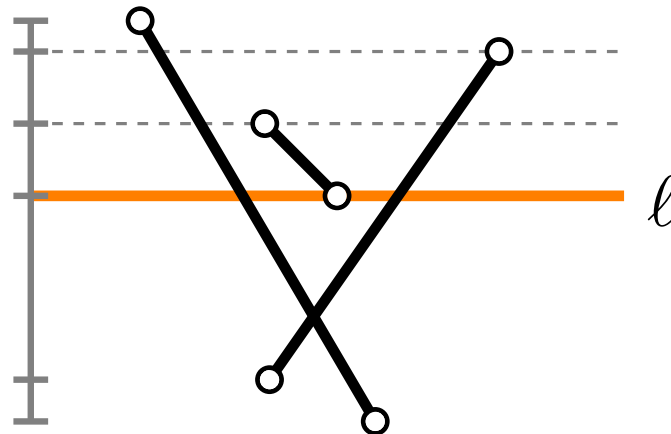
Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Sicher? Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

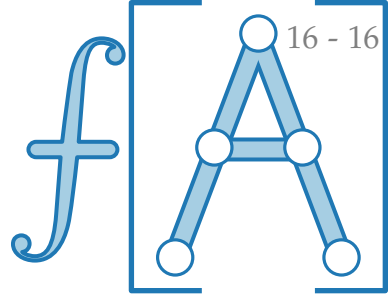
→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



- füge $s \cap s'$ in Q ein
- entferne $s \cap s'$ aus Q

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

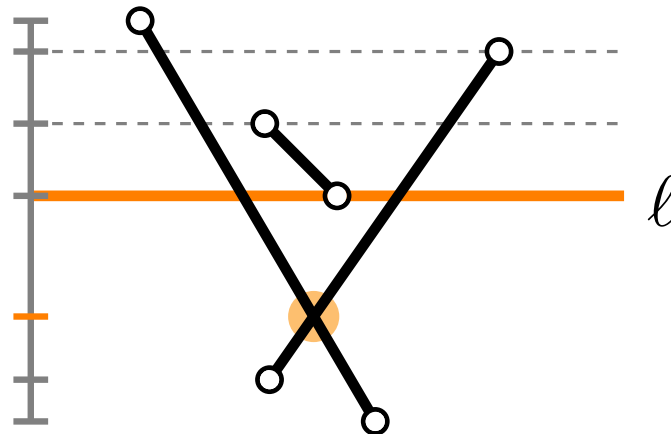
Sicher?

Die Eventschlange $\mathcal{Q}$ enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

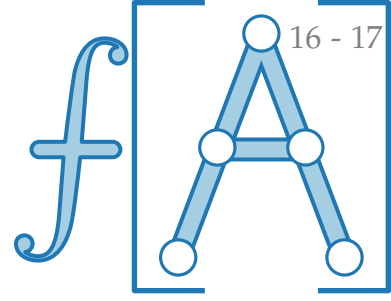
→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



- füge $s \cap s'$ in $\mathcal{Q}$ ein
- entferne $s \cap s'$ aus $\mathcal{Q}$
- füge $s \cap s'$ wieder in $\mathcal{Q}$ ein

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

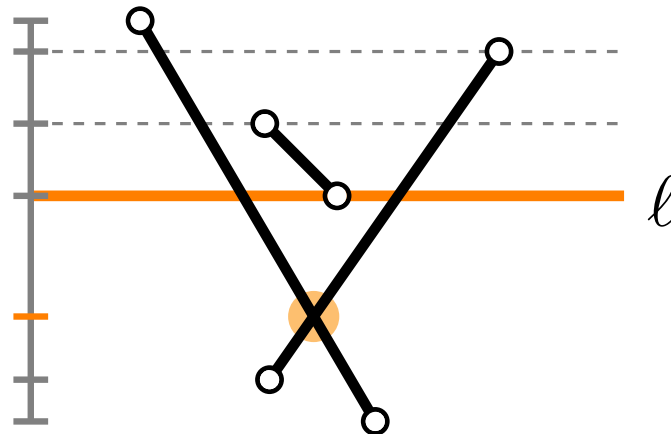
Sicher?

Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

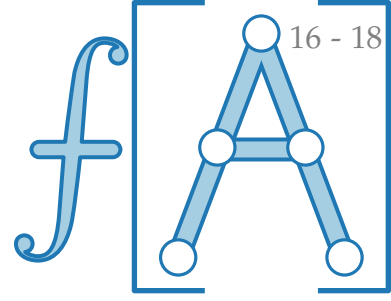
Geht das besser?



- füge $s \cap s'$ in Q ein
- entferne $s \cap s'$ aus Q
- füge $s \cap s'$ wieder in Q ein

→ brauchen nur $\mathcal{O}(n)$ Platz;
(asymptotische) Laufzeit ändert sich nicht

Resultat



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

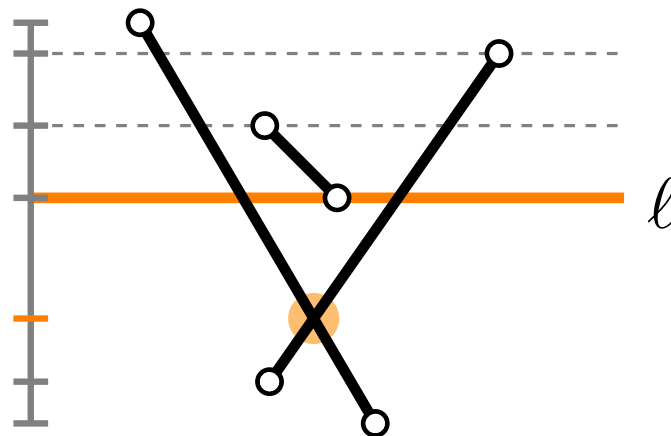
Sicher?

Die Eventschlange Q enthält

- Alle Endpunkte **unter der Sweep-Line**
- Alle Schnittpunkte **unter der Sweep-Line**

→ (worst-case) Platzverbrauch $\in \Theta(n + k)$ 😞

Geht das besser?



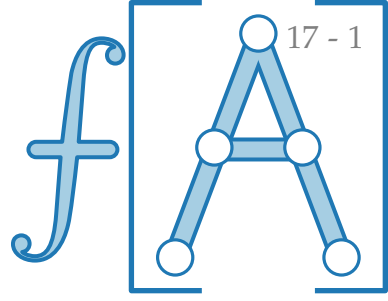
- füge $s \cap s'$ in Q ein
- entferne $s \cap s'$ aus Q
- füge $s \cap s'$ wieder in Q ein

→ brauchen nur $\mathcal{O}(n)$ Platz;
(asymptotische) Laufzeit ändert sich nicht

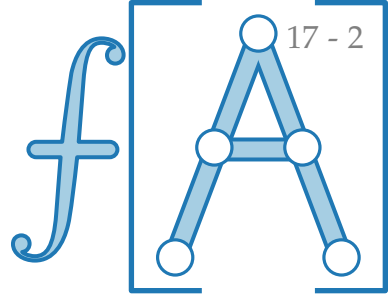


Verbesserungen

Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.



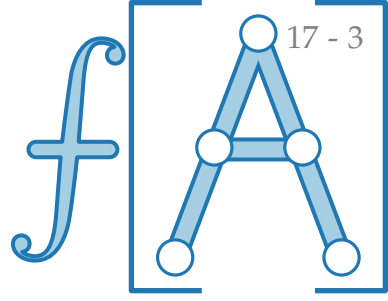
Verbesserungen



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Satz. [Chazelle & Edelsbrunner 1992]
Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}(n \log n + k)$ Zeit und $\mathcal{O}(n + k)$ Speicherplatz berechnen.

Verbesserungen



Satz. Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}((n + k) \log n)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.

Satz. [Chazelle & Edelsbrunner 1992]
Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}(n \log n + k)$ Zeit und $\mathcal{O}(n + k)$ Speicherplatz berechnen.

Satz. [Balaban 1995]
Wir können alle k Schnittpunkte von n nicht-überlappenden Liniensegmenten in der Ebene und die Segmente, die die Schnittpunkte enthalten, in $\mathcal{O}(n \log n + k)$ Zeit und $\mathcal{O}(n)$ Speicherplatz berechnen.